Costa Rica Institute of Technology

School of Mechatronics Engineering



An MBSE-Based Framework for the Design and Analysis of Nanosatellite Electrical Power Systems

Graduation Project Report submitted for the degree of

Licentiate in Mechatronics Engineering

Anthony José Arguedas Rodríguez

Cartago, November 13, 2026

I declare that the present Graduation Project and its corresponding final report are of my authorship and contain a substantive original intellectual contribution. I have developed them under my judgement and responsibility, with the appropiate academic guidance, through the reference and usage of relevant sources and the addition of original analyses, decisions, and contributions.

All sources, materials, and contributions from third parties used in the present work have been properly recognized or cited, as applicable.

In case of having used artificial generative intelligence tools during the development of the project or this report, I declare that I have personally reviewed and validated the entirety of the content produced or modified with their aid.

In consequence, I assume full academic responsibility for the exactness, integrity, originality, pertinence, and proper attribution of the realized work and the content of its final report.

Anthony José Arguedas Rodríguez

Montreal, Quebec, November 13, 2026

ID: 2-0851-0418

Abstract

For the ASCEND research group at the École de technologie supérieure, evidence fragmentation during the early lifecycle phases of nanosatellite electrical power systems hinders the traceability between needs and requirements, architecture decisions, and the analytical evidence that supports them. Furthermore, it hinfers the reproducibility of analyses and the capability to detect design inconsistencies before advacing to physical verification and validation phases. This project mitigates this problem by developing a model-based systems engineering framework oriented at the early justification of nanosatellite electrical power systems through traceable and reproducible evidence. The framework is implemented using Python and Github, at a positive economic value, and fulfills requirements such as retaining parametric requirements, identifying impacts of a proposed mission, requirement, component, or model change, proposing COTS EPS components from available requirements, and recommending and executing a set of EPS analyses and verification methods. Two case studies compared the analysis outputs of the framework and those of BIRDS-4 and OreSat publications under the same mission and EPS inputs, and found consistent values and findings. User trials showed a significant decrease in the time taken to perform controlled early justification tasks for stakeholders leveraging the framework. The framework should be extended to other nanosatellite subsystems, and the release of the entire codebase as a public repository enables this.

Keywords: CubeSat, nanosatellite, electrical power system, model-based systems engineering, framework

A quienes me esperan de vuelta

Agradecimientos

Agradezco profundamente a mi padre, a mi madre, a mi novia, y a todo el resto de mi familia que me ha hecho sentir en casa a pesar de la distancia. A mi asesor en Canadá, el Dr. Jesús David González Llorente, y al Dr. Juan José Rojas Hernández por confiarme un reto tan grande y tan enriquecedor. A mi asesor en Costa Rica, MSc.-Ing. Roger Meléndez Poltronieri, por su discernimiento, paciencia, y dirección constante. Y a todas las personas que trabajan para el Tecnológico de Costa Rica y la École de technologie supérieure, por contribuir al pilar social que es la educación pública.

Anthony José Arguedas Rodríguez

Montreal, Quebec, November 13, 2026

Contents

List of Figures v

List of Tables vii

List of Abbreviations ix

1 Introduction 1

2 Theoretical Framework 5
- 2.1 NASA engineering lifecycle . . . 5
- 2.2 Concept of Operations . . . 5
- 2.3 Verification and validation . . . 5
- 2.4 Model-based Systems Engineering . . . 6
- 2.5 The Object-Oriented Systems Engineering Methodology . . . 6
- 2.6 Stakeholders. . . . 6
- 2.7 Operational domain as-is and to-be, and enterprise. . . . 7
- 2.8 Normative documents applicable to CUBICS proposal . . . 7
- 2.9 Generic models of EPS requirements and architecture . . . 8
- 2.10 Generic models of EPS components . . . 9
- 2.11 EPS development lifecycle . . . 10

3 Methodology 11
- 3.1 Stakeholder Analysis . . . 11
  - 3.1.1 As-is capabilities . . . 11
  - 3.1.2 Stakeholder concerns . . . 12
  - 3.1.3 Potential to-be capabilities . . . 13
- 3.2 System Requirements Analysis . . . 13
  - 3.2.1 Measures of performance . . . 15
- 3.3 Logical Architecture . . . 19
- 3.4 Logical Components . . . 20
- 3.5 Synthesize Candidate Physical Architectures . . . 22
- 3.6 Logical Component Alternatives . . . 22
- 3.7 Synthesized Candidates . . . 24
  - 3.7.1 Candidate scores and rationales . . . 25

4 Design Proposal 29

4.1 Global MBSE model and user interfaces . . . 29
4.2 L01: MBSE Workspace . . . 31
4.3 L02: COTS Database . . . 33
4.4 L03: Simulation and analysis . . . 37
4.4.1 Mass and occupied volume (FR-12) . . . 37
4.4.2 Orbit and eclipse durations (FR-14) . . . 37
4.4.3 Power budgets and preliminary sizing (FR-14) . . . 38
4.4.4 Array alternatives (FR-15 and FR-16) . . . 39
4.4.5 Photovoltaic characteristics (FR-18) . . . 40
4.4.6 DET and MPPT operating points (FR-19) . . . 41
4.4.7 Array area and illumination (FR-14) . . . 42
4.4.8 Battery capacity and configuration (FR-14 to FR-16) . . . 44
4.4.9 Bus voltage and regulation choices (FR-17) . . . 45
4.4.10 Protection and electrical extrema (FR-20) . . . 45
4.5 L04: Configuration Management . . . 46

5 Results and Analysis **49**
5.1 User trials . . . 49
5.2 Catalog inspection . . . 51
5.3 EPS case studies . . . 52
5.3.1 Verification setup . . . 52
5.3.2 Results and discussion . . . 52
5.4 Economic Analysis . . . 53
5.4.1 Base case and included costs . . . 53
5.4.2 Review effort and maintenance . . . 54
5.4.3 Net present value . . . 54
5.4.4 Sensitivity and interpretation . . . 55

6 Conclusions **57**

A User Trial Evidence **79**
A.1 Task presentations . . . 80
A.2 Submitted states and elapsed times . . . 81
A.3 Evidence detail . . . 87

B Automatic EPS Requirements **89**

C COTS Database **91**

D EPS Case Study Evidence **99**
D.1 Verification basis . . . 99
D.2 Results by requirement . . . 100
D.2.1 FR-12: mass, volume and allocation margins . . . 101
D.2.2 FR-14: energy budgets and BOL/EOL sizing . . . 101
D.2.3 FR-15: solar technology and battery chemistry . . . 102

D.2.4 FR-16: series and parallel arrangements 102
D.2.5 FR-17: bus rails and converter compatibility 103
D.2.6 FR-18: PV characteristics and sampled MPP 103
D.2.7 FR-19: matched DET and ideal MPPT paths 104
D.2.8 FR-20: normal extrema and protection comparisons 104

E Framework Models and Analysis Scripts **107**

List of Figures

2.1 Simplified systems engineering technical processes. 7

3.1 Logical architecture. 21

4.1 CSRM mission and segment structure. 30
4.2 CSRM CubeSat subsystems. 30
4.3 EPS properties in Python. 31
4.4 The same battery properties in the GUI. 31
4.5 Python calculation and algebraic expression. 31
4.6 Calculated properties and rendered requirement prose. 32
4.7 Requirement, component and analysis links. 33
4.8 Persistent analysis input and output mappings. 34
4.9 Verification before and after a parameter change. 35
4.10 Automatically created EPS requirements. 35
4.11 COTS filtering in Python and the GUI. 36
4.12 Configuration change and verification impact. 47

A.1 Enlarged requirement submission from MOP-01. 87
A.2 MOP-06 intermediate change views. 88

E.1 Model and analysis folder structure. 107
E.2 `models/__init__.py`. 108
E.3 `models/elements.py`. 109
E.4 `models/eps.py`. 112
E.5 `models/functions.py`. 121
E.6 `models/geometry.py`. 123
E.7 `models/mission.py`. 124
E.8 `models/modes.py`. 126
E.9 `models/objects.py`. 128
E.10 `models/properties.py`. 133
E.11 `models/recommendations.py`. 138
E.12 `models/requirements.py`. 140
E.13 `models/spacecraft.py`. 144
E.14 `models/stakeholders.py`. 146
E.15 `scripts/illumination.py`. 147
E.16 `scripts/sizing.py`. 151

List of Tables

3.1 Framework requirements. 14
3.2 Framework measures of performance. 16
3.3 Framework requirement and interfaces allocations to the logical components. 20
3.4 Five physical implementation alternatives for each logical component. Descriptions include proposed custom extensions and external tools. 22
3.5 Candidate scores and rationales relative to the as-is baseline C5. Criterion scores range from −2 to +2, with higher scores preferred. 25

5.1 First user trial task. 50
5.2 COTS product counts by catalog category. 51
5.3 Economic value by period in constant 2026 Canadian dollars. Mission years follow the four-month internship. Present values use monthly discounting, and totals use unrounded values. 55

A.1 Illustrative inputs shared by the prepared sizing tasks. Changes for each task are given in Table A.2. Groups are illustrative. 79
A.2 Task inputs and requested submissions. Each role receives a separate copy of the same initial mission for each task. 80
A.3 Catalogue values supplied to MOP-11. The supplied energy values are minimum ratings and the masses are maximum ratings. Every candidate has nominal voltage 3.6 V. 81
A.4 User trial submissions. 82

B.1 Requirements automatically assigned to EPS elements. 89

C.1 COTS products and source documents. 91

List of Abbreviations

Abbreviations

AIV	Assembly, integration and verification
API	Application programming interface
BM25	Lexical ranking method for text retrieval
BOL	Beginning of life
CAD	Computer-aided design
CDR	Critical design review
cFS	core Flight System
ConOps	Concept of operations
COTS	Commercial off-the-shelf
CPU	Central processing unit
CSRM	CubeSat System Reference Model
CW	Continuous wave
DC	Direct current
DET	Direct energy transfer
DFN	Doyle-Fuller-Newman battery model
DoD	Depth of discharge
DSM	Design structure matrix
DUT	Device under test
EMC	Electromagnetic compatibility
EOL	End of life
EPS	Electrical power system
FAB	Front access board

FMEA	Failure modes and effects analysis
FMECA	Failure modes, effects and criticality analysis
FMI	Functional Mock-up Interface
FMU	Functional Mock-up Unit
FN	False negative
FP	False positive
FPP	F Prime Prime (F Prime modeling language)
FR	Framework requirement (system requirement for the EPS engineering framework)
FRR	Flight readiness review
FSW	Flight software
FW	Framework (action-label prefix)
GNSS	Global navigation satellite system
GPS	Global Positioning System
GPU	Graphics processing unit
HDDL	Hierarchical domain definition language
HIL	Hardware in the loop
HTN	Hierarchical task network
ID	Identifier
IF	Interface
IMU	Inertial measurement unit
INCOSE	International Council on Systems Engineering
KiBaM	Kinetic battery model
LED	Light-emitting diode
MBSE	Model-based systems engineering
MCR	Mission concept review
MDR	Mission definition review
MIL	Model in the loop
MOE	Measure of effectiveness
MOP	Measure of performance

MPPT	Maximum power point tracking
MR	Mission requirement
NASA	National Aeronautics and Space Administration
NE	Non-EPS engineer (stakeholder role)
OBC	On-board computer
OMG	Object Management Group
OOSEM	Object-Oriented Systems Engineering Method
PCB	Printed circuit board
PCI	Peripheral Component Interconnect
PDR	Preliminary design review
PIL	Processor in the loop
PM	Project manager (stakeholder role)
RAM	Reliability, availability and maintainability
REST	Representational state transfer
RMSE	Root mean square error
SaaS	Software as a service
SC	Scenario
SDR	System definition review
SE	Systems engineering. Systems engineer when naming a stakeholder role.
SIL	Software in the loop
SIR	System integration review
SOH	State of health
SPM	Single particle battery model
SRR	System requirements review
SysML	Systems Modeling Language
TBR	To be resolved
TP	True positive
UC	Use case
UHF	Ultra high frequency

USD	Universal Scene Description (scene representation)
V&V	Verification and validation
YAML	YAML Ain't Markup Language

Chapter 1

Introduction

We are said to be living in a *New Space* era, where access to space is open to more countries, companies, universities, and people than ever. This democratization of space has been driven by progressively decreasing barriers to entry for space systems development and space launches in terms of time horizon, technological feasibility, and economic cost. The factors at play include reusable launch platforms, standardization of space system and payload architectures, the integration of commercial-off-the-shelf (COTS) components, and the adoption of more efficient and effective design and development approaches.

The CubeSat format within the nanosatellite class is a prime example of architecture standardization. These systems consist of modular units of 10-by-10-by-10 centimeter cubes at around 1.33 kilograms, which have been proven to be integratable at up to 16 units [1] and have supported a plethora of scientific and commercial payloads ranging from orbital laboratories to internet network nodes.

Unfortunately, CubeSats historically suffer from a high failure rate, and thus not only fall short of fulfilling mission needs but become a worrying source of space debris at the crowded low earth orbits that they usually launch to. The causes can be traced back to a lack of expertise, coordination, or consistency in the engineering teams behind the mission, often based at universities that are new entrants to the sector as opposed to incumbents with space heritage, or, at a technical level, attributed to electrical power systems (EPS) as a centralized point of unrecoverable failure.

This project takes place at the (ASCEND) research group at the École de technologie supérieure in Montréal, Canada. Dr. Jesús González Llorente, ASCEND lead an academic advisor of this project, pursues a line of research concerned with reliable space systems, and has authored multiple academic publications targeting the design and validation of nanosatellite electrical power systems.

ASCEND is pursuing participation in a CubeSat Initiative in Canada for STEM (CUBICS) mission co-developed across two universities and involving mostly student volunteers in engineering and coordination roles. This endeavor requires a wide and deep exploration of solution candidates, coupled with robust tradeoff analysis, verification, and validation of

said candidates to deliver a reliable CubeSat. Furthermore, the CUBICS financial and time budget requires that any requirement non-conformities need to be detected as early as possible, preferrably prior to hardware fabrication which is one of the costliest phases of the entire engineering lifecycle, and is associated with a large iteration period. All of the above needs to achieved by a highly motivated and capable yet inexperienced cohort of student volunteers.

The current project is intended as a precedent for the CUBICS ambition, which is referred to as capability development. It covers the electrical power subsystem in its lifecycle phases prior to commissiong or developing hardware, and is founded in the experience gained in previous EPS design efforts at ASCEND. The problem that motivated the current project is:

> Evidence fragmentation during the early lifecycle phases of nanosatellite electrical power systems hinders the traceability between needs and requirements, architecture decisions, and the analytical evidence that supports them. Furthermore, it hinfers the reproducibility of analyses and the capability to detect design inconsistencies before advacing to physical verification and validation phases.

This project applied the Object-Oriented Systems Engineering Method to the above problem. The as-is EPS design and development practices of the stakeholders and their shortcomings were assessed, and a to-be set of capabilities was derived from best practices identified in the academic literature. After multiple discussions with stakeholders, a set of concerns about the as-is capabilities and the potential transition to the to-be capabilities was constructed. Identified concerns relate to the high effort to define verification activities for EPS component requirements, the high effort to maintain traceability from requirements to design decisions, the high cost and effort of redesign after hardware fabrication, and several other factors.

The main objective of this project is:

> To develop a model-based systems engineering framework oriented at the early justification of nanosatellite electrical power systems through traceable and reproducible evidence.

An MBSE framework is defined following Buruso et al. [2] as the integration of MBSE methods, MBSE languages, and tools within and throughout the relevant lifecycle phases. The prescription of MBSE in the main objective is a constraint imposed by ASCEND expertise and capability development interests. Therefore, multiple methods, languages, and tools from within the ample MBSE literature were considered as candidate building blocks for the framework.

The secondary objectives of this project are: (1) to characterize the needs, requirements, interfaces and technical criteria of nanosatellite EPS; (2) to define the operational flow of modeling, analysis, and evidence management of the MBSE framework; (3) to model a generic EPS architecture with functions, logical and physical components, interfaces, modes

and states, and requirements; (4) to characterize early EPS verification and validation within the EPS framework; (5) to evaluate the framework on two contrasting EPS case studies. Therefore, the current project does not aim to only design or develop an EPS for ASCEND, but to improve the EPS design and development capabilities at the research group, so that they can be leveraged to bring the future CUBICS mission to fruition.

The framework will be verified against requirements such as retaining forward and reverse links between requirements and component allocations, enabling the reconstruction of mission analyses, and identifying change impacts. Verification will combine catalogue inspection, two mission case studies, and two individual user trials. The BIRDS-4 and OreSat case studies will use published power budgets and component data to check EPS mass and volume estimates, power and energy budgets, sizing, component and architecture alternatives, and electrical limits against independently prepared references. The user trials will cover requirement and allocation traceability, analysis records and reproduction, change impact assessment, engineering guidance, revision control, COTS recommendations, and mission input management. An ASCEND PhD candidate and a student club undergraduate will each perform 11 tasks, with each task required to take less than 60/11 min per participant.

The remainder of this document is structured as follows: Chapter 2 presents the theoretical background, with particular attention to normative documents and state-of-the-art MBSE research. Chapter 3 describes the OOSEM analysis of stakeholder needs and their formalization as verifiable framework requirements that can be fulfilled by multiple architecture alternatives. Chapter 4 details the design of the chosen alternative, in terms of its CubeSat and EPS models, orbit characterization, and power budget, component selection, and verification method implementations. Chapter 5 covers results and analysis, obtained via two case studies as well as user trials. Chapter 6 states the conclusions and future work.

The main engineering contribution of this project is the publication of the full codebase [3] of an MBSE framework that can mitigate evidence fragmentation during the early lifecycle phases of nanosatellite electrical power systems owing to it enabling traceable and reproducible evidence. The framework imposes no operation costs as it is implemented using Python and Github, and has been verified against BIRDS-4 and OreSat with regard to the correctness of its analyses. User trials showed that the stakeholders became more agile when managing EPS design evidence when using the framework, and it has potential to be extended to other nanosatellite subsystems.

Chapter 2

Theoretical Framework

This section presents concepts and academic works that form the theoretical foundation of the document. Special attention is given to normative documents and state-of-the-art MBSE research that characterize the needs, requirements, interfaces and technical criteria of nanosatellite EPS, as per the first objective of the project.

2.1 NASA engineering lifecycle

The NASA reference lifecycle progresses from Pre-Phase A (Concept Studies), through A (Concept and Technology Development) and B (Preliminary Design and Technology Completion), to C (Final Design and Fabrication). Requirements, design and technical management processes recur as products mature. Phase C normally includes detailed design, hardware fabrication and software coding. CDR occurs during C before deliverable coding. SIR occurs at its end to assess integration readiness [4, Secs. 2.2–2.3, 3.3–3.6 and Table 6.7-1].

2.2 Concept of Operations

A Concept of Operations is a high-level description of how a system will be used to meet stakeholder expectations, usually in a time sequence. It provides an operational perspective that guides requirements and architecture development [4, Sec. 4.1.1.2.4 and Appendix B].

2.3 Verification and validation

Verification establishes conformity to applicable requirements. Validation assesses suitability for stakeholder expectations and ConOps in the intended environment. Validation may use models under representative simulated conditions. Analysis is a valid validation method. Requirements validation also checks correctness, feasibility, consistency, verifiability and stakeholder suitability. Model calibration adjusts parameters against data. Independent

validation data, assumptions, uncertainty and the intended validity domain are needed to assess a model's credibility [4, Secs. 2.4, 4.2 and 5.3–5.4][5, pp. 7–9].

2.4 Model-based Systems Engineering

Buruso et al. describe model-based systems engineering (MBSE) as the use of a shared system model across engineering disciplines and throughout the system lifecycle, from identifying stakeholder needs to development and operations [2]. Such models can capture functional, logical, and physical characteristics at different levels of design detail. They can also support simulation, comparison of candidate solutions, and data exchange across tools and engineering roles [2]. Potential benefits of the adoption of MBSE include lower documentation effort and development cost, better system understanding and collaboration, earlier fault detection, and reduced technical risk [2].

2.5 The Object-Oriented Systems Engineering Methodology

OOSEM is a top-down, scenario-driven MBSE method that serves to analyze, specify, design, and support the verification of systems. Its primary output is an integrated system model with consistent views of behavior, structure, properties, and requirements traceability. Its specification and design activities progress from stakeholder needs and black-box system requirements to logical and physical architectures. Engineering analysis, trade studies, and traceability span these activities. The activities are tailored and iterated rather than executed as fixed sequential phases [6].

Figure 2.1 summarizes the systems engineering technical process. Stakeholder needs inform system specification and design. Component requirements guide component design, implementation, and test. Verified components feed system integration and test. Feedback from component work and integration returns to system specification and design [6, Fig. 1.1]. OOSEM implements system specification and design and supports integration and verification through model-based test cases. The development process can be applied recursively at successive levels of the system hierarchy [6, Sec. 17.1.2].

2.6 Stakeholders.

In OOSEM, a stakeholder is a person or organization with an interest in the system across its lifecycle. A stakeholder role identifies whose concerns must be addressed [6, Secs. 17.3.2 and 17.3.5].

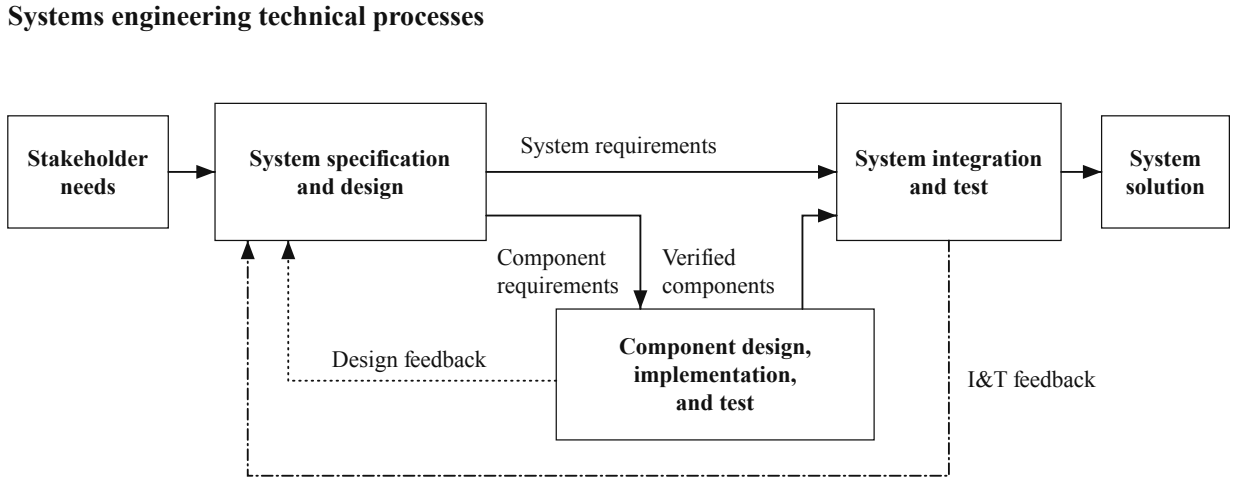


Figure 2.1: Simplified systems engineering technical processes. Solid arrows carry forward exchanges. Dotted arrows carry design feedback. Dash-dot arrows carry integration and test feedback. Adapted from Friedenthal et al. [6, Fig. 1.1].

2.7 Operational domain as-is and to-be, and enterprise.

An operational domain provides the context for the system of interest. It contains the operational system, the users and external systems with which it interacts, and the relevant environment. An as-is domain characterizes the current context sufficiently to understand stakeholder concerns. A to-be domain describes the intended context after the mission need is addressed. Within a domain, an enterprise is an aggregation of systems and users that collaborate to achieve a set of mission objectives. It contains the system of interest and the supporting participants needed to accomplish those objectives, while other actors and environmental elements may remain external to it [6, Sec. 17.3.2].

2.8 Normative documents applicable to CUBICS proposal

The basic normative references are the CUBICS announcements of opportunity, first released in 2022 and most recently in 2026 [7, 8]. These documents define proposal eligibility and the evaluation process used to select recipients of CSA funding [8, Secs. 3 and 5]. The 2022 call specified design and test reviews adapted from the NASA Systems Engineering Handbook for college and university payload or CubeSat projects [7, App. B.7]. The 2026 call allows flexibility in development strategies while evaluating the documented project plan, milestones and schedule [8, FAQ 24 and App. C.2.1]. The Canadian Space Agency also published the Systems Engineering Mission Tailoring Guidelines (CSA-SE-GDL-0001), which provide guidance on tailoring systems engineering requirements by mission class, particularly

for smaller Class D missions like academic nanosatellites and personnel training [9, Secs. 1.1–1.3].

By applying CSA-SE-GDL-0001 to the NASA systems engineering lifecycle, a mission treated as CSA Class D can retain the progression from concept studies through design, fabrication, integration, operations and closeout. Engineering effort is scaled to the mission's objectives, resources and accepted risk of mission failure [4, Secs. 3 and 3.11][9, Sec. 3.2]. Plans can be consolidated into a concise project plan while retaining essential requirements, interfaces, resource budgets, risks and verification evidence [4, Sec. 3.11.4][9, Secs. 4.2–4.4 and App. B].

Class D design assurance can favor commercial components, limited redundancy and fewer separate physical development and qualification units where the consequences of failure permit. Frequent engineering discussions and peer reviews provide oversight, supported by recorded tailoring decisions, residual risks and required approvals [9, Sec. 3.4.1][4, Sec. 3.11.3].

Normative documents provide requirements directly applicable to the EPS [10, Sec. 2.3]. They also constrain other subsystems, whose interfaces and operating needs lead to derived EPS requirements [4, Secs. 4.2–4.3]. United Nations guidance and United States regulations provide comparative references for battery safety, passivation and disposal [11, 12]. Canadian licensing requires five-year disposal and at least 90% probability of disposal success [13, Decisions D10–D11].

2.9 Generic models of EPS requirements and architecture

The CubeSat Design Specification and EXOpod Nova manual provide preliminary guidance on power isolation, inhibits and charging access [10, 14]. Other subsystems contribute derived EPS requirements through their supply voltages, power consumption and operating schedules [15, Secs. 2–4][16, Sec. 2].

The Object Management Group's CSRM Profile 1.1 specification complements these engineering sources with standardized categories for mission, spacecraft, subsystem and component requirements [17, Sec. 6.1]. It relates requirements to the architectural elements that satisfy them and to verification activities with recorded outcomes [17, Secs. 6.3–6.4]. These relationships support tracing normative and derived EPS requirements through design and verification. The surveyed literature identifies it as mature, with 18 papers in its core research lineage covering the Radio Aurora Explorer predecessors, reference model development, and companion methods for behavior, technical measurement, validation and mission engineering [18–35].

2.10 Generic models of EPS components

Luccisano et al. model shared component identity, manufacturer, mass, power by operating mode and margins per component. Their distinction between aggregate and detailed component budgets supports explicit accounting boundaries [36, Secs. 3.1–3.2]. Ceglarek and Bertrand contribute a catalog built from manufacturer datasheets with controlled parameter meanings and units. Their experience with inconsistent datasheets motivates retaining the original value and its reviewed interpretation [37, Sec. 3]. The proposed schema will extend these contributions with source revisions, operating conditions, uncertainty, supported configurations and interface descriptions. These extensions will make catalog comparisons assessable.

El Hachimi et al. supply the solar maximum power point and EOL properties, battery energy and cycle-life criteria, and examples of named cells and their arrangements [15, Secs. 3 and 6.2]. Casado et al. contribute solar current–voltage curves with irradiance and temperature conditions, battery discharge curves and internal resistance [38, Secs. 2.1 and 2.3]. Boan contributes the series/parallel configuration and mission illumination needed to assess a selected assembly [16, Sec. 3]. These sources motivate separate cell and assembly records, conditional electrical characteristics and installation geometry. Wijaya et al. contribute fixed commercial product properties alongside sized alternatives. Purchased panels and packs will retain their specified dimensions, mass and ratings [39, Sec. III.A–B, Appendix].

El Hachimi et al. contribute conditioning and bus alternatives, converter input compatibility, independently controlled outputs, protection thresholds, timing and autonomous protection functions [15, Secs. 4 and 7.1–7.6]. Casado et al. contribute regulator operating modes and transitions, protected output rails, and distribution efficiency estimates based on measurements [38, Secs. 2.2 and 2.4]. The models will therefore distinguish conversion capabilities, electrical operating limits, loss data, channel organization and protection behavior. A controller integrated circuit record will retain its required external circuit. Its ratings alone will establish no complete converter or protected output.

In complementary approach, digital twin research supports obtaining verification evidence without fabricating the complete EPS. Casado et al. use models validated against physical measurements to simulate EPS behavior over a complete orbital cycle [38, Secs. 3–4]. Related battery digital twin work by Zhu et al. (2025) describes virtual testing and software-in-the-loop verification without physical prototypes, followed by hardware-in-the-loop testing [5, p. 3]. This is consistent with the tailored lifecycle, since NASA recognizes modeling and simulation as verification by analysis and recommends selecting methods according to project constraints and accepted risk [4, Sec. 5.3.1.2]. The CSRM specification likewise includes verification under simulated or modeled conditions within its definition of analysis [17, Sec. 6.6.8]. Selected requirements can therefore be verified before fabrication using credible models, while retaining the physical safety and qualification tests required by the mission and launcher [4, Sec. 5.3.1.2][10, Secs. 2.3 and 3].

2.11 EPS development lifecycle

EPS development workflows link mission inputs and power budgeting to architecture, design, integration and verification within the spacecraft lifecycle [15, 16]. These workflows can be understood as an EPS development lifecycle embedded within the tailored spacecraft lifecycle, progressing from applicable normative and derived requirements to a design validated against them. El Hachimi et al. cover the progression through functional and environmental qualification, while Boan demonstrates the preliminary stages through iterative sizing and energy balance simulations [15, Secs. 2–5][16, Secs. 2–4].

Analysis of system models provides verification evidence during preliminary design. Strobbe et al. link an EPS MBSE model to power budget calculations and propagate parameter changes to identify violated requirement constraints [40, Secs. III–IV]. Jbara automates power and other spacecraft budget calculations and accepts or rejects candidate component selections against numerical limits [41, Secs. 4–5]. Wijaya et al. generate alternative architectures, couple subsystem sizing calculations and compare their mass, power and dimensional results with mission requirements [42, Sec. 5.2]. Their simplified sizing models support preliminary trends and relative trade-offs [42, Sec. 6.1].

Chapter 3

Methodology

OOSEM structures the development of the framework as the system of interest. Stakeholder analysis first allows an understanding of the as-is and to-be states of stakeholder capabilities. Then, this chapter presents the direct derivation of framework requirements, followed by their allocation to logical components and candidate physical architectures [6, Chap. 17].

3.1 Stakeholder Analysis

The needs of stakeholders are analyzed by comparing the as-is state of their capabilities against a to-be (desired) state derived from a review of external references and the state-of-the-art. This is a preliminary discussion that will be later formalized into verifiable requirements. The analysis characterizes the needs, requirements, interfaces and technical criteria of nanosatellite EPS, both as observed within ASCEND and in the consulted references, and thus represents the deliverable of the first objective of this project.

3.1.1 As-is capabilities

For the CUBICS mission, a collaboration between ASCEND researchers and members of a student club at ÉTS is being proposed. The present work is aimed at serving them both.

ASCEND researchers are familiar with systems engineering and even MBSE, but the nature of their position does not allow them to contribute significant time to the practical aspect of CubeSat design and development. In contrast, ASCEND-associated students who wish to participate in CUBICS have years of experience in building systems for student engineering competitions but follow no systems engineering method.

The students have built CubeSat-format payloads for rocketry competitions, for which they verified payload power requirements only after fabrication. This verification was observed to take the form of reusing a legacy battery pack for which the original calculations have been lost, connecting all loads, and performing mission operations repeatedly until the stored energy was depleted. If the time until energy depletion was greater than the duration of one

operations cycle as defined by the competition rules, the power requirements were taken as verified. These experiments were reported to result in battery lifespans several times greater that one operations cycle, with the students opining that the battery pack was oversized.

In contrast, the student club members have ample experience in verifying requirements related to rocket mass, maximum height, and maximum speed before fabrication, via inspection and analysis in simulation. These requirements are given as numerical targets by each rocketry competition, or derivated from the top-place results of previous competitions. These competition requirements are always expressed in terms of measurable quantities, which is essential for competition grading.

ASCEND researchers report an EPS design that has been in progress for nine months. Unclear definition of requirements has been a recurring issue, and it has forced redesigns when it is discovered that the existing requirement list is incomplete. Requirement changes force the researchers to wait at least one week, until the next one hour-long weekly meeting, to assess their impact, because the traceability from requirements to design has to be established manually each time.

3.1.2 Stakeholder concerns

Following OOSEM, stakeholder discussions identified concerns about the cost and effort of closing capability gaps. These concerns inform framework requirements and measures of performance.

A project manager and systems engineer at ASCEND is concerned with maintaining verifiable requirements, redesign after fabrication, and tools suited to the team's mission scale and resources [15, Secs. 4–5][38, Sec. 1][2, Sec. 5.1]. That stakeholder is also concerned with requirement traceability, communicating verification and validation evidence, and updating EPS and other subsystem power requirements as designs change. Maintaining consistency across requirements, models and verification evidence also demands effort [2, Secs. 4.4 and 5.1][43, Secs. 3–4][44, Sec. V].

An EPS engineer is concerned with repeating verification after requirement changes, tracing design decisions, revising designs after failed verification, updating component requirements and communicating designs during technical reviews. Further concerns include verifying safe and effective charging and power management [5, pp. 1–3 and 6][15, Sec. 3][45, Secs. 5.1 and 6]. Engineers responsible for other subsystems share the concern about verification cost and effort.

Taken together, the aforementioned as-is capability gaps and associated concerns can be formulated as a problem that motivates the current project:

> Evidence fragmentation during the early lifecycle phases of nanosatellite electrical power systems hinders the traceability between needs and requirements, architecture decisions, and the analytical evidence that supports them. Furthermore, it hinfers the reproducibility of analyses and the capability to detect design

inconsistencies before advacing to physical verification and validation phases.

3.1.3 Potential to-be capabilities

After observing as-is stakeholder capabilities and recording them objectively, the OOSEM method suggests analyzing the capabilities of existing external enterprises and systems. From this, capability gaps or the lag behind the state-of-the-art can be identified and later formalized into the requirements for the systems and processes that will improve these capabilities, in this case the proposed framework. Sources for this comparison include the academic literature focused on the state-of-the-art in MBSE, as ASCEND is a research group with MBSE interests, and normative documents. Said sources were discussed in Chapter 2, to which the reader is referred in order to avoid repetition.

3.2 System Requirements Analysis

System Requirements Analysis defines how the to-be (desired) stakeholder capabilities can be obtained with support from the system of interest, the framework in this case. The requirements define what the framework must accomplish for the stakeholders constrained by acceptable ranges of measurable system variables (Measures of Performance in OOSEM), which make them verifiable.

Table 3.1 gives the full list of framework requirements. Overall, the requirements tackle the core problem that motivates this work by covering from needs, requirements, and architecture decisions to analytical evidence in a single framework, which reduces fragmentation. Furthermore, analysis reproducibility and the detection of design inconsistencies with respect to defined verification criteria are directly considered in the requirements.

FR-01 and FR-02 establish the base unit of engineering knowledge as a requirement with forward and reverse links and quantitative values, which are used in MBSE approaches to construct a model representation of the mission. FR-06 enables the capability to assess the impact of a change in a quantitative requirement, which is desired by the stakeholders in order to explore alternatives and keep requirements up to date in an agile manner. FR-03 establishes that whenever an analysis is executed its inputs and outputs have to be contextualized by the mission requirements. FR-09 establishes a controlled evolution of mission requirements and the architecture choices that depend on them, while FR-04 specifies that this evolution must be replicable without loss of information.

FR-07 and FR-08 aim to lessen the workload of inexperienced engineers, who understand EPS development from technical specifications but may struggle to follow a systems engineering line of work from high-level mission requirements, with the necessary verification measures at every step. FR-10 and FR-11 support progress along the EPS engineering lifecycle, where early designs with ideal or assumed values must mature into evaluating the properties of real COTS. FR-05 states that the model of what an EPS consists of, which de-

limits which COTS are relevant for this framework within the inmense entirety of commercial offerings, must adhere to the CSRM Profile 1.1 specification.

FR-12 to FR-20 cover fundamental analyses that are considered in the literature to be the minimum set that verifies an early EPS design and the COTS choices made for it from Phase 0 to Phase B in the NASA Reference Lifecycle. This is what the problem statement of this projects refers to as early lifecycle phases. They cover the component types defined by CSRM and common tradeoff analyses performed at this level of design maturity.

Table 3.1: Framework requirements.

ID	System requirement
FR-01	The framework shall retain each requirement with its identifier, statement, parents and children, rationale, conditions or quantities, applicable units, verification method, and resolution status, including reasons for unresolved fields.
FR-02	The framework shall retain forward and reverse links among requirements and component allocations.
FR-03	The framework shall associate each analysis result with its requirement/expectation, method, scenario, parameters, raw outputs, and execution status.
FR-04	The framework shall provide the inputs, dependencies and instructions to reconstruct all analyses of a mission.
FR-05	The framework shall allocate EPS requirements among components covering generation, storage, conversion, regulation, protection, distribution, and control components as per the CSRM Profile 1.1 specification.
FR-06	The framework shall identify impacts of a proposed mission, requirement, component, or model change on requirements and EPS design.
FR-07	The framework shall recommend next engineering activities and from the current activity and inputs. identifying unmet prerequisites.
FR-08	The framework shall recommend analysis, verification and validation methods or models on edits or requests using available inputs.
FR-09	The framework shall retain retrievable mission states that resolve exact requirement, mission, design, and model revisions with revision history and the authorized decisions that control them.
FR-10	The framework shall estimate electrical and dynamic EPS model parameters from quality-checked data.
FR-11	The framework shall propose COTS EPS components from available requirements.
FR-12	The framework shall calculate EPS mass and occupied volume estimates from aggregate or component data and assess margins against allocations, retaining sources, units, and unknown values.
FR-13	The framework shall retain mission orbit, attitude, duration, environmental conditions, subsystem modes, and load schedules as inputs to EPS requirements and analysis.
FR-14	The framework shall calculate state and mode powers, sunlight and eclipse energy budgets, and array and battery sizing at supplied beginning and end of life conditions, reporting integer cell counts, array area, and battery capacity.

ID	System requirement
FR-15	The framework shall compare explicitly supplied solar and battery cell technologies through applicable power, capacity, electrical, installation and charger compatibility checks, retaining raw measures, limits, verdicts and unknown values.
FR-16	The framework shall compare explicitly supplied array and battery series and parallel configurations through common power, capacity, electrical and array installation checks, retaining raw measures, limits, verdicts and unknown values.
FR-17	The framework shall compare explicitly selected bus voltage and buck/boost converter alternatives against linked input voltage intervals, output voltage tolerances and output current limits, retaining raw measures and unknown values.
FR-18	The framework shall calculate sampled photovoltaic P–V characteristics and maximum power points from supplied I–V samples and check assembled array ratings against linked electrical and component limits.
FR-19	The framework shall compare operating points and bus powers for explicitly selected direct energy transfer and MPPT paths under common array, battery, illumination and load conditions against linked electrical limits.
FR-20	The framework shall calculate voltage and current extreme values for supplied normal operating cases and compare them with declared undervoltage, overvoltage and overcurrent trip thresholds.

3.2.1 Measures of performance

Table 3.2 assigns Measures of Performance to the framework requirements [6, Sec. 17.3.3] to make them verifiable. Each MOP is evaluated through one of two verification scenarios.

The first scenario combines catalogue inspection for FR-10 with mission case studies for FR-12 and FR-14 to FR-20. The catalogue inspection counts COTS products with source datasheet references for each generic EPS component type. The case studies use BIRDS-4 and OreSat component and mission data, including their published power budgets [46, 47], to define representative verification configurations. Explicit supplementary inputs complete these configurations. The comparisons assess the implemented calculations under those conditions, with no claim of reproducing either flown mission in full. The sources cited in Table 3.2 supply component data, architecture alternatives, solar characteristics and protection settings. Each case study MOP is evaluated separately for BIRDS-4 and OreSat. Both missions must meet its desired range.

For FR-12 and FR-14 to FR-20, an evaluator prepares reference results independently of the framework using the equations and analysis scope in Chapter 4. Inputs come from each mission's published conditions and component data. A published output is compared directly when its conditions, included components and circuit location match the calculation. Other expected outputs are calculated independently from those inputs. Each reference identifies its source and whether it is published or calculated. Supplementary inputs and alternative configurations are declared verification assumptions, distinct from the published mission design.

Each of these protocols requires complete numerical cases and applicable checks for unknown and invalid inputs. Unavailable inputs remain unknown in the mission baseline. Complete cases use explicitly stated supplementary values. For each nonzero numerical reference r, the framework output x must satisfy $|x - r|/|r| \leq 0.01$ after unit conversion. The 1% tolerance applies to each value in each mission, without averaging. Zero references require zero outputs. Counts, arrangements, verdicts, retained records, expected unknowns and evaluation errors require exact agreement. Missing required outputs fail the comparison. Source inconsistencies are reported without changing published values. These comparisons verify the implemented calculations at their stated conditions.

The second scenario verifies FR-01 to FR-09, FR-11 and FR-13 through two individual user trials, one with a PhD candidate from ASCEND and one with an undergraduate from the student club. Each participant performs the 11 tasks in Table 3.2 from prepared initial states. The trials use the GUI and a prepared Git environment for FR-09. Each trial allocates 60 min of task time equally among the tasks, giving each task a limit of 60/11 min (approximately 5 min 27 s). Each user trial MOP consists only of the two elapsed times from task presentation to participant submission. Both times must be below 60/11 min and are evaluated separately, without averaging or a correctness score. A task that reaches its time limit is recorded at that limit.

Table 3.2: Framework measures of performance.

MOP/FR	Verification protocol	Desired range
Catalogue inspection		
MOP-10/FR-10	Group the added COTS entries by the generic EPS component type they represent. Count distinct part numbers with a reference to the source manufacturer datasheet in each group. Report the count for every generic EPS component type defined by the framework, including types with no entries. Catalogue product categories are mapped to these generic types before counting.	≥ 5 sourced COTS per generic EPS component type
BIRDS-4 and OreSat case studies		
MOP-12/FR-12	For each mission, enter its documented EPS components with sourced masses, occupied volume estimates and allocations. Use BIRDS-4 component records [48] and OreSat's 252 g solar and 416 g battery module entries [49, Sec. D.2] at their stated boundaries. Compare component sums, aggregate substitutions and remaining allocations with independent calculations. Check that integrated products contribute once, nonnegative margins pass, and unknown contributions leave affected totals unresolved. Inspect retained sources and units.	$\leq 1\%$ per mission Exact records and verdicts

MOP/FR	**Verification protocol**	**Desired range**
MOP-14a/FR-14	For each mission, compare state and mode powers, orbital period, sunlight/eclipse durations, interval load energies and minimum panel power with independent calculations from its published inputs and stated model assumptions. Compare matching published load and generation rows separately. Check supplied energy against interval demand. Use external mean generation for BIRDS-4's free rotation. Exercise circular and eccentric orbits and cases without eclipse as explicit variations.	$\leq 1\%$ per mission Exact verdicts
MOP-14b/FR-14	Calculate each mission's array and battery sizing at stated BOL and EOL conditions using complete component data and declared supplementary inputs. Compare fractional and integer cell counts, cycle demand, selected DoD, required battery energy, pack energy and charge capacity, retained capacity and array areas with independent calculations. Check packing, installation area, projected illumination, source generation and power and energy verdicts for the entered configuration. Apply each degradation or loss factor once.	$\leq 1\%$ per mission Exact counts and verdicts
MOP-15/FR-15	For each mission, enter at least two solar cell technologies and two battery chemistries as separate configurations under common mission, load and illumination conditions. Use BIRDS-4's proposed hybrid study [50, Sec. III] and OreSat's solar and battery studies [51, 52] to define alternatives. Compare power, area, capacity, electrical and charger compatibility measures and verdicts with independent references. Retain explicit selections and unknown properties.	$\leq 1\%$ per mission Exact verdicts and unknowns
MOP-16/FR-16	Enter each mission's published array and battery arrangements and at least one supplied alternative for each. Include BIRDS-4's two-cell solar strings and 3S2P, 3800 mAh battery [48, Sec. 2.2, Table 5] and OreSat's two independent 2S packs, 37.4 Wh per card [53, p. 6]. Compare counts, array ratings and area, pack energy, charge capacity, voltage and current ratings and feasibility verdicts with independent calculations. Keep independent packs and their circuit connections distinct.	$\leq 1\%$ per mission Exact counts, arrangements and verdicts

MOP/FR	Verification protocol	Desired range
MOP-17/FR-17	For each mission, connect explicit converter products to supplied source intervals and load rails. Include BIRDS-4's 3.3 and 5 V rails [48, Sec. 2.2.3] and OreSat's bus voltage alternatives and SPV1040 rejection after its 1S to 2S change [52, 54]. Compare input interval containment, nominal output voltage tolerance, output current limits and terminal currents including characterized losses with independent references. Check supplied product conditions and retained unknowns.	$\leq 1\%$ per mission Exact verdicts and unknowns
MOP-18/FR-18	Compare available labelled BIRDS-4 Maya-2 power products [48, Figs. 13–14] and OreSat's tabulated solar products [55] at their stated conditions. Use explicitly declared complete verification curves to compare sampled MPP, efficiency and scaled array ratings with independent calculations. Retain the distinction between published samples and assumed curve segments. Check linked electrical limits, scalar fallback, partial or invalid curve rejection and purchased-panel handling.	$\leq 1\%$ per mission Exact verdicts and statuses
MOP-19/FR-19	Using the mission curves from MOP-18, define paired DET and ideal MPPT verification configurations for each mission under common array, battery, illumination and load conditions. Independently calculate operating voltage and current, source power, operating efficiency and bus power with the specified drops and converter efficiency. Compare these measures and linked electrical limit verdicts. Test exact samples, explicitly enabled interpolation and out-of-range errors. Treat fixed input-voltage regulation as a separate case.	$\leq 1\%$ per mission Exact verdicts and errors
MOP-20/FR-20	Use each mission's normal voltage and current cases and selected detector thresholds, including BIRDS-4's 2, 3 and 4 A line thresholds [56, Sec. 6.3] and OreSat's 30 W load current cases and hardware protection settings [57]. Compare extrema, current bounds from powers and voltages at the same terminals, and strict no-trip verdicts with independent references. Exercise values below, at and above each declared threshold. Check detector selection, cell voltage sensing, units, unknowns and invalid samples.	$\leq 1\%$ per mission Exact verdicts and statuses
User trials		
MOP-01/FR-01	Given a fully defined mission requirement, create another requirement with the FR-01 fields and a simple verification condition and submit its retained fields and condition result.	Both times $< 60/11$ min

MOP/FR	Verification protocol	Desired range
MOP-02/FR-02	Given prepared requirement links and component allocations, retrieve forward and reverse links and submit the associated requirement and component identifiers.	Both times $< 60/11$ min
MOP-03/FR-03	Given a completed analysis and a deliberately failed run, open their result records and submit the linked requirement, method, scenario, parameters, available outputs and execution status.	Both times $< 60/11$ min
MOP-04/FR-04	Given retained mission inputs, dependencies and instructions in a prepared environment, reconstruct the mission and rerun its assigned analyses using the retained input and output bindings. Submit the rerun results.	Both times $< 60/11$ min
MOP-05/FR-05	Given a prepared EPS and requirements for the seven FR-05 functions, allocate each requirement to a component and submit the allocation view.	Both times $< 60/11$ min
MOP-06/FR-06	Apply supplied mission, requirement, component and model parameter edits separately, and submit the updated requirements and EPS results after each edit.	Both times $< 60/11$ min
MOP-07/FR-07	Given an EPS component with an omitted input, use its automatically assigned requirements to identify the next quantity to supply, enter the provided value and submit the updated requirement view.	Both times $< 60/11$ min
MOP-08/FR-08	Given an EPS component and supporting inputs, inspect its automatically assigned requirements and submit the assigned analysis and verification methods or models and their input mappings.	Both times $< 60/11$ min
MOP-09/FR-09	Given a prepared mission repository, save a baseline with its author and authorized decision rationale, change one input, retrieve the baseline and submit its requirement, mission, design and model revisions and decision history.	Both times $< 60/11$ min
MOP-11/FR-11	Given a generic EPS component, supplied requirements and compatible COTS alternatives, run the component query and submit the recommended part numbers.	Both times $< 60/11$ min
MOP-13/FR-13	Given a mission input sheet, enter and save orbit, attitude, duration, environmental conditions, modes and load schedules, reopen the mission and submit its retained inputs.	Both times $< 60/11$ min

3.3 Logical Architecture

Following OOSEM, the framework requirements are allocated to logical components, which are abstract units of behavior independent of a particular implementation technology [6,

Sec. 17.3.4]. Table 3.3 allocates the FRs to four logical components. Figure 3.1 shows their exchanges in one view, with users outside the framework boundary and simulation and analysis inside it.

3.4 Logical Components

Table 3.3: Framework requirement and interfaces allocations to the logical components.

Logical component	FR numbers	Interfaces
L01 MBSE Workspace	01–08, 11, 13	Stakeholders submit requirements and parameters. Responses use an underlying MBSE model to return validity checks of the supplied requirements and their parameters, evaluated values for computable parameters, and the updated state of the requirements and recommended next activity after a submission. L01 forwards any computation requests to L03, and links the obtained results to the parameters of the MBSE model.
L02 COTS Database	10	L01 submits requests to link COTS parameters to the MBSE model, and receives identifiers for said parameters. L03 requests the linked parameters as reported in the MBSE model and receives their values.
L03 Simulation and Analysis	12, 14–20	Simulation and analysis scripts receive spacecraft MBSE models, parameters and states. They return identifiers to numerical analysis results.
L04 Configuration Management	09	Users submit change requests and receive impact findings. Users submit change history requests and receive revision histories, retrievable baselines.

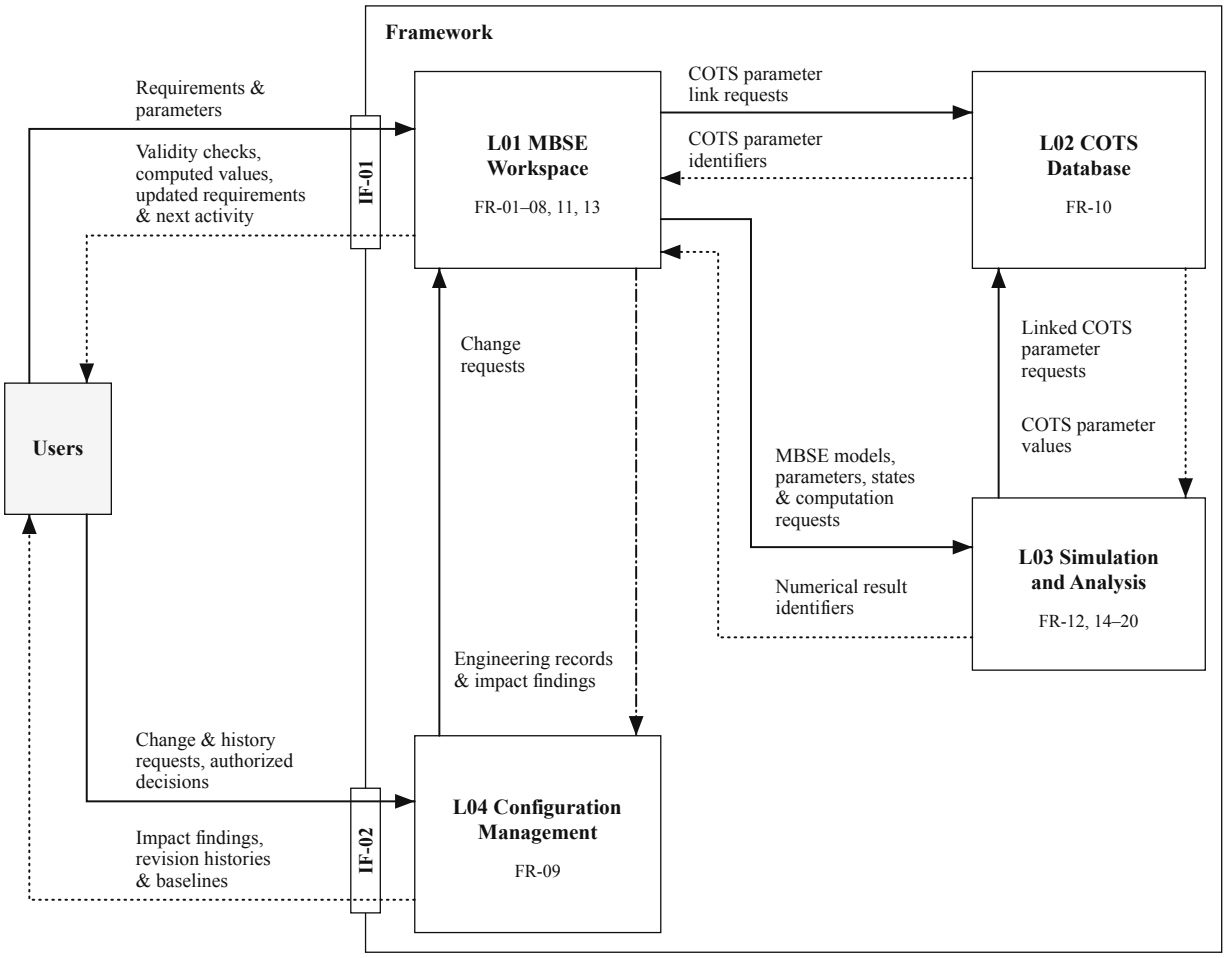


Figure 3.1: Logical architecture of the framework. Users exchange engineering inputs and responses with L01 through interface IF-01, and configuration requests, decisions and responses with L04 through IF-02. Solid arrows carry requests and analysis inputs. Dotted arrows carry identifiers, parameter values and user responses. Dash-dot arrows carry engineering records and impact findings.

3.5 Synthesize Candidate Physical Architectures

3.6 Logical Component Alternatives

Logical components break down the required framework behavior into four building blocks. Table 3.4 gives five alternatives to implement each logical component. For each alternative, a concise description of how it implements each logical component is given. Valispace Requirements Manager and CATIA Stimulus were studied as dedicated MBSE alternatives at the request of the stakeholders, and are considered for all logical components given that they are integrated solutions with ample functionality. An analysis of Microsoft Excel, Python, MATLAB, and Github was also requested for the applicable logical components. The rest of the alternatives were chosen from a review of the state of the art in MBSE, consistent with the focus stated in the main objective of this project.

Table 3.4: Five physical implementation alternatives for each logical component. Descriptions include proposed custom extensions and external tools.

ID	Alternative	Implementation
L01 MBSE Workspace		
L01-A	Valispace Requirements Manager	Store linked requirements and computed parameters in Altium Requirements Portal. Configure the Engineering Assistant with EPS AI Skills for requirement review and verification criteria [58, 59]. A custom checker reads project data through the REST/Python API, evaluates the EPS activity rules and returns recommendations for user review [60].
L01-B	CATIA Stimulus	Simulate formalized behavioral requirements to expose inconsistencies [61, 62]. Mission inputs, component records, result links, and activity guidance would have to be implemented externally.
L01-C	Microsoft Excel	Represent requirements, components and parameters in linked worksheets. Use formulas for checks and custom macros for activity guidance, analysis requests and result links.
L01-D	Python	Build a custom model editor with requirement and component links, input checks, activity rules and analysis calls. Python model exchange in the Nanospace studies provides a precedent [36, 63], but its code is unavailable.
L01-E	COMET	Use shared requirements, engineering parameters and numerical links as the MBSE model. Add external EPS selection rules, activity guidance and analysis [64].

ID	Alternative	Implementation
L02 COTS Database		
L02-A	Valispace Requirements Manager	Populate component hierarchies with catalogue parameters and supplier references. A custom API adapter would have to supply catalogue identifiers and exchange values with L01 and L03 [60, 65].
L02-B	CATIA Stimulus	Load component properties into model parameters through a custom catalogue adapter. Use an external database for persistent part records, datasheets and identifiers.
L02-C	Microsoft Excel	Maintain one catalogue row per part with identifiers, units, limits and datasheet links. Lookup formulas or custom adapters supply selected values to the MBSE model [64].
L02-D	Python	Provide a custom catalogue database with parameter identifiers, units, sources and queries. Interfaces resolve MBSE component references and return numerical values to L03, extending the Nanospace data exchange approach [63] (original code unavailable).
L02-E	COMET	Use component catalogue entries and datasheet-linked parameters in a shared engineering model. Add external catalogue revision tracking and mappings from part properties to mission parameters [64].
L03 Simulation and Analysis		
L03-A	Valispace Requirements Manager	Evaluate static budgets using linked formulas. Send dynamic EPS and fault analyses to external solvers through a custom API connector, then link their results to model parameters [60, 65].
L03-B	CATIA Stimulus	Generate constrained test scenarios and check requirement behavior with external EPS models coupled through the Functional Mock-up Interface. Retain numerical outputs, test traces and requirement coverage [61, 66].
L03-C	Microsoft Excel	Calculate mass, volume, mode power, eclipse energy and array and battery sizing with formulas. Add scenario tables for concept comparisons and external solvers for dynamic electrical and fault analyses [64].
L03-D	Python	Execute custom scripts for budgets, sizing, concept comparisons and electrical and fault models. Return raw outputs and execution status. Quantitative Python component models provide a demonstrated reference without code [45].
L03-E	MATLAB/Simulink	Run sizing and parameter studies in MATLAB and dynamic EPS and fault models in Simulink. A custom connector imports model parameters and returns numerical results.

ID	Alternative	Implementation
L04 Configuration Management		
L04-A	Valispace Requirements Manager	Use requirement and parameter histories and change notifications to review revisions. Add and external archives for exact mission, design, model and analysis revisions and authorized decisions [60, 65].
L04-B	CATIA Stimulus	Treat executable requirements, simulation models and test records as controlled engineering items [61]. An external configuration system provides revision history, approved baselines and analysis reconstruction instructions.
L04-C	GitHub	Version text models, catalogue records, scripts and analysis settings in Git repositories. Use reviewed change requests and tagged releases with baseline records, dependency versions and reconstruction instructions.
L04-D	SeaLion approach	Keep model inputs and generated engineering views under Git as demonstrated by SeaLion. Extend release records with approvals, exact analysis dependencies and reconstruction instructions [67].
L04-E	Nanospace	Maintain engineering records and dependency links in a shared database. Add immutable revision records, baseline snapshots, authorized change decisions and analysis reconstruction packages [63].

3.7 Synthesized Candidates

Valispace Requirements Manager (C1) and CATIA Stimulus (C2) were compared as integrated solutions spanning all logical components. Other two candidates were synthesized by combining different alternatives: Version-controlled Python (C3) consists of using native Python for L01-L03, and Github for L04, while Version-controlled Excel with Python adapters (C4) uses Microsoft Excel for L01-L02, Python for L03, and Github for L04. A fifth candidate representing the as-is capabilities (C5) was introduced to serve as the baseline.

The candidates were compared based on a set of weighted criteria, which have value zero if they match the baseline, +1 to +2 if they improve them, and -1 to -2 if they worsen them. Greater is better. These criteria are not Measures of Performance, as all considered candidates are assumed to be capable of attaining the target MOP values, but merely a decision-making aid informed by the stakeholder analysis. The criteria were development complexity (20%), cost per year (30%), learning curve (30%), evidence centralization (10%) and extensibility (10%).

3.7.1 Candidate scores and rationales

Table 3.5 records each score and its rationale. Lower development complexity, costs and learning effort count as improvements. Greater evidence centralization and extensibility also count as improvements. The weighted total for candidate j is $S_j = \sum_{i=1}^{5} w_i s_{ij}$, where s_{ij} is the criterion score and w_i is its percentage weight expressed as a fraction. C5 has zero for every criterion and a weighted total of zero.

Table 3.5: Candidate scores and rationales relative to the as-is baseline C5. Criterion scores range from -2 to $+2$, with higher scores preferred.

Criterion	Weight	Score	Rationale
C1 Valispace Requirements Manager			
Development complexity	20%	-2	Requires custom extensions to cover all requirements, and its interfaces are propietary and not well documented.
Yearly cost	30%	-2	CA$1418 yearly. Included in Altium Develop, a professional subscription service.
Learning curve	30%	0	Includes AI assistance, but the array of controls is large.
Evidence centralization	10%	+1	Depends on the degree of integration allowed by the propietary interfaces.
Extensibility	10%	0	Requires a long-term commitment to an expensive product whose usability depends on propietary interfaces.
Weighted total	**100%**	**-0.90**	

Criterion	Weight	Score	Rationale
C2 CATIA Stimulus			
Development complexity	20%	-2	Requires custom extensions to cover all requirements, and its interfaces are propietary and not well documented.
Yearly cost	30%	-1	CA$249 via an INCOSE individual membersip, for private or academic use only.
Learning curve	30%	0	Includes AI assistance, but the array of controls is large.
Evidence centralization	10%	+1	Depends on the degree of integration allowed by the propietary interfaces.
Extensibility	10%	0	Requires a long-term commitment to an expensive product whose usability depends on propietary interfaces. 3DXPERIENCE integration is limited for unauthorized developers.
Weighted total	**100%**	**-0.60**	
C3 Version-controlled Python			
Development complexity	20%	-1	A widely-used object-oriented programming language. Several publications provide reference patterns, even if code is unavailable.
Yearly cost	30%	0	No cost to develop, operate, and maintain.
Learning curve	30%	0	Only the included requirements are developed. Python files can be read and edited by AI agents to provide assistance. Custom assistance can be tailored to user trials.
Evidence centralization	10%	+2	Covers all requirements natively. Includes database management, object-oriented modeling via classes, numerical analysis, and version control with online uploads and authorship traceability.
Extensibility	10%	+2	Ample ecosystem of open-source libraries.
Weighted total	**100%**	**+0.20**	

Criterion	Weight	Score	Rationale
C4 Version-controlled Excel with Python adapters			
Development complexity	20%	-1	A widely-used and programmable spreadsheet software. Documentation on its use for systems engineering is scarce.
Yearly cost	30%	-1	CA$115 Yearly cost of a Microsoft 365 subscription.
Learning curve	30%	0	Excel is widely used, but macro programming is an specialist topic.
Evidence centralization	10%	+1	Macros require a completely difference interface for development, usage, and configuration management.
Extensibility	10%	+1	The core spreadsheet is propietary software. The files it generates aren't easily readably for version contr
Weighted total	**100%**	**-0.30**	

According to the weighted scores, the winner concept is C3: Version-controlled Python. The design of this concept is presented in detail in Chapter 4, while a detailed economic analysis can be found in Section 5.4.

Chapter 4

Design Proposal

The framework is implemented as a set of Python classes which represent the MBSE model of a CubeSat mission according to CSRM Profile 1.1, where each class models an element like a requirement, a subsystem, a component or, more specifically, an EPS component like a battery or a solar array. The reader is referred to Appendix E for the full set of Python files that fully describe the framework's behavior, models and analyses. The code that defines how models can be utilized, for example, to add, edit, update, verify, and delete requirements, represents the deliverable of the second objective of this project, whereas the provided EPS models represent the deliverable of the third objective. Furthermore, Appendix B contains the list of all requirement statements that are automatically assigned to EPS components as the deliverable of the fourth objective.

The complete project codebase, including the present document, the GUI, user documentation, examples, and short explanatory videos for how to execute all framework requirements on the GUI, is available as a public Github repository [3]. The present section will present a narrative overview of said codebase guided by the requirements it satisfies.

4.1 Global MBSE model and user interfaces

Figures 4.1–4.2 show the mission structure, including a parametrized orbit (FR-13) and the nine CubeSat subsystems in CSRM. Mission teams tailor these logical categories to their missions. Detailed, mission-tailored design then establishes physical quantities and implementation choices.

The model classes can be imported into a Python environment, as shown in Figure 4.3. A graphical user interface (GUI) exposes the same models for users unfamiliar with Python (Figure 4.4). Besides classification, the models assign engineering properties to components, including the EPS component properties required by FR-05. These properties represent COTS datasheet quantities and provide inputs to the analyses. The figures use a synthetic battery allocation to demonstrate the same parameter values in both interfaces.

Analyses are Python functions that consume EPS properties and return results that can be

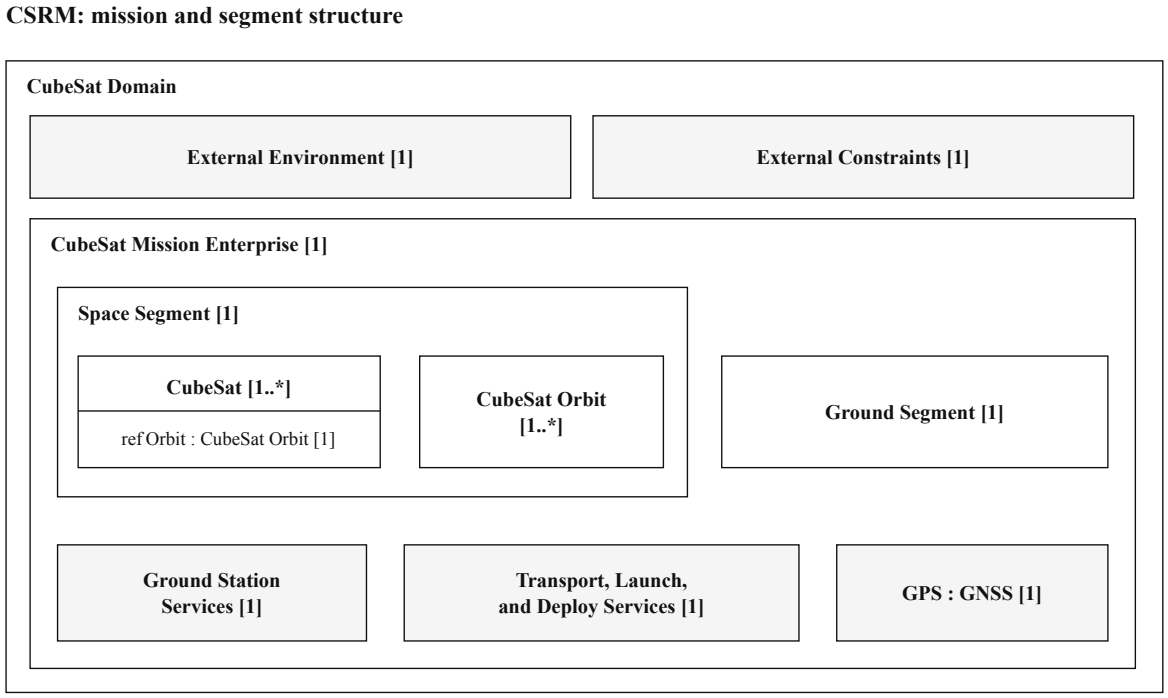


Figure 4.1: CSRM mission and segment structure, as a graphical representation of the corresponding Python classes. Adapted from the CSRM reference model, copyright 2022 INCOSE Space Systems Working Group and OMG Space Domain Task Force, CC BY 4.0 [68].

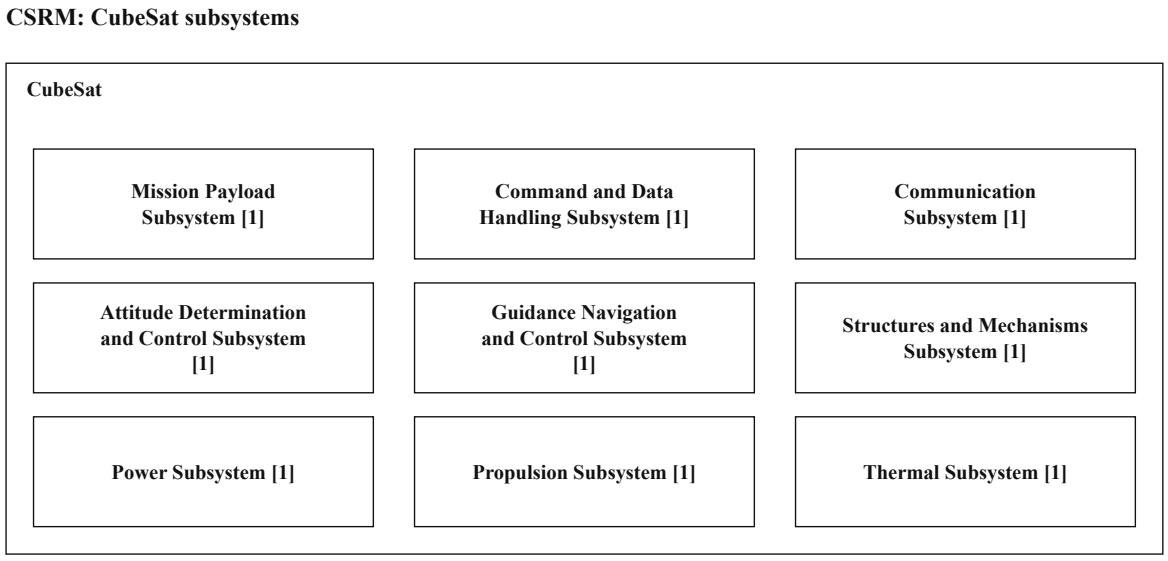


Figure 4.2: CSRM CubeSat subsystems. The reference architecture contains one part for each of nine subsystem categories. Power Subsystem is the EPS. Mission-specific models can omit, combine or refine categories to meet mission needs [32]. Adapted from the CSRM reference model [68].

(a) Battery class (selected lines)

```
class Battery(EPSComponent):
    rated_energy = Property(
        "self.cell_count.value * "
        "self.cell_rated_energy.value", "Wh")
    bol_energy = Property(
        "self.rated_energy.value * 3600", "J")
```

(b) Interactive Python

```
>>> from framework.models import (
...     Mission, CubeSatBus)
>>> mission = Mission()
>>> bus = mission.add(CubeSatBus("BUS"))
>>> battery = bus.EPS.Battery
>>> battery.rated_energy = 20.35
>>> battery.rated_energy.value
20.35
>>> battery.bol_energy.value
73260.0
```

Figure 4.3: Selected lines from the Battery class and an interactive Python session. The default `rated_energy` expression multiplies cell count by cell energy in Wh. A complete pack rating can be assigned directly, as in the illustrative 20.35 Wh allocation shown here. The `bol_energy` property converts this rating to 73260 J at beginning of life.

(a) Rated energy entered in Wh

rated_energy	Default	Wh	20.35	20.35

(b) Beginning-of-life energy calculated in J

bol_energy	Default	J	self.rated_energy.value * 3600	73260

Figure 4.4: Cropped views of the Battery parameter editor show the same 20.35 Wh rating and calculated 73260 J value as Figure 4.3. Each row retains its expression, unit and evaluated value.

assigned to other properties. The functions follow the equations and algorithms cited in this chapter. Figure 4.5 compares a calculation with its algebraic expression.

(a) Battery sizing calculation

```
energy = (eclipse_energy / depth_of_discharge
          * (1 + degradation_margin))
_finite(energy)
return Energy(energy)
```

(b) Algebraic expression

$$E_{\text{Battery}} = \frac{E_e}{\text{DoD}}(1 + M)$$

`eclipse_energy`	E_e (J)
`depth_of_discharge`	DoD
`degradation_margin`	M

$$\frac{6000\ \text{J}}{0.5}(1 + 0.2) = 14400\ \text{J}$$

Figure 4.5: The battery energy calculation implements Equation (4.21). An eclipse demand of 6000 J, a discharge depth of 0.5 and a degradation margin of 0.2 require 14400 J of beginning-of-life energy. The excerpt shows the calculation and return value. The full function also checks finite inputs and their permitted ranges.

4.2 L01: MBSE Workspace

This section describes the behavior of logical component L01: MBSE Workspace. For FR-01, the framework represents requirements, quantities and verification conditions as Python

classes. Inspired by the property expressions of SysML v2 [69], a Property stores a Python expression that can reference other properties, arithmetic operations and function calls. Figure 4.6 shows a calculated property and the GUI rendering of a requirement statement. Units accompany values as annotations. The framework performs no automatic unit conversion or dimensional checking.

(a) Property class (selected lines)

```
class Property:
    def __init__(self, source=None, unit=None):
        self.expression = source
        self.unit = unit

    @property
    def value(self):
        return self.evaluate()
```

(b) Calculated requirement property

```
required = Property(
    "sizing.battery_energy("
    "eclipse_energy=self.eclipse_energy, "
    "depth_of_discharge=self.depth_of_discharge, "
    "degradation_margin=self.degradation_margin"
    ").joules", "J")
```

(c) GUI statement after evaluation

The battery shall provide at least 14400 J of BOL energy (available: 73260 J).

Figure 4.6: Selected Property class lines and a requirement property that calls the battery sizing function. Its named inputs are other properties, and the returned `joules` field supplies the required energy. The GUI statement combines the calculated 14400 J target with the available 73260 J. The class excerpt omits internal initialization and evaluation details.

For FR-02 and FR-03, each verification condition links a requirement to a component property through its measure and to an analysis result through its target expression (Figure 4.7). To satisfy FR-04, the requirement's `verify()` method evaluates its stored conditions and their input expressions. Figure 4.8 shows these input and output mappings in the GUI. Python users call the evaluation method directly. The GUI evaluates expressions during editing and refreshes the requirement table after saving. This feedback exposes the impact of a parameter change for FR-06, as illustrated by the passing and failing evaluations in Figure 4.9.

The behavior to satisfy FR-07 and FR-08 is entirely deterministic, despite recommendation being a very open and subjective problem. Certain models, mainly subsystems and components, were given methods to automatically self-assign requirements when they are added to the mission. The requirements were chosen based on recurring constraints to EPS components in the literature and relevant normative documents, and can be read in full in B.

The GUI displays automatically created requirements and their unresolved inputs (Figure 4.10). For FR-07, these inputs identify the next engineering activity: supply the quantities needed to evaluate the requirement. The requirements already contain verification conditions, which become evaluable when their inputs are supplied, providing the verification activity recommendation required by FR-08.

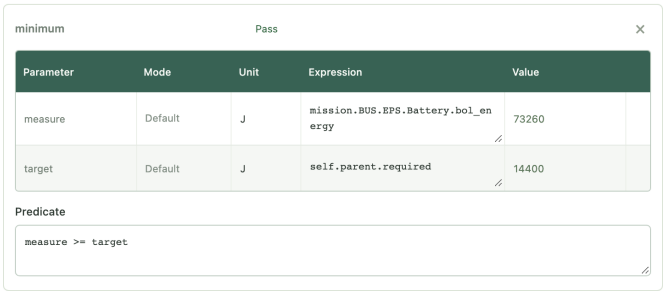
For FR-11, the framework evaluates an explicitly supplied set of compatible COTS alternatives against the generic component's directly owned requirements (Figure 4.11). Each trial substitutes the candidate's property definitions into a copy of the mission and retains the candidate only when every owned requirement passes. Unknown inputs, evaluation errors

(a) Stored verification links

```
class VerificationCondition(ModelObject):
    measure = Property()
    target = Property()

condition = VerificationCondition(
    "minimum", battery.bol_energy,
    Property("self.parent.required", "J"),
    "measure >= target")
```

(b) GUI verification condition



(c) Requirement evaluation method

```
def verify(self, *, mode=None):
    return VerificationReport(self.mission, tuple(
        condition.evaluate(mode=mode)
        for condition in self.conditions))
```

Figure 4.7: The verification condition stores a measure linked to the battery's `bol_energy` property and a target linked to the requirement's calculated `required` property. The GUI displays these same links and the comparison predicate. The evaluation method reuses all stored conditions.

and failed conditions exclude the candidate. The engineer selects applicable datasheet facts and conditions before defining the COTS alternatives. Requirements owned by the EPS or bus remain outside this component query.

4.3 L02: COTS Database

This database is said to estimate electrical and dynamic EPS model parameters (as per FR-10) because its information comes from datasheets and not empirical modeling. For the purposes of this project, a great volume of COTS property values taken from datasheets was deemed preferrable to a small volume of empirical data, given that the framework requirements cover the exploration of alternatives during early design stages and not the qualification of mature physical designs.

The database contains commercially available products in 18 catalog categories, for which the datasheets were obtained via the Mouser Canada search engine. The properties of the generic EPS components were defined based on the information found in the datasheets, and datasheets that didn't provide the necessary values to execute all EPS analyses were discarded unless they are used in one of the case studies (BIRDS-4 or OreSat). The reader is referred to Appendix C for the full list of COTS with their corresponding generic EPS component type, part identifier, and mass as one representative property. The full database is available in the project repository [3] as tab-separated tables.

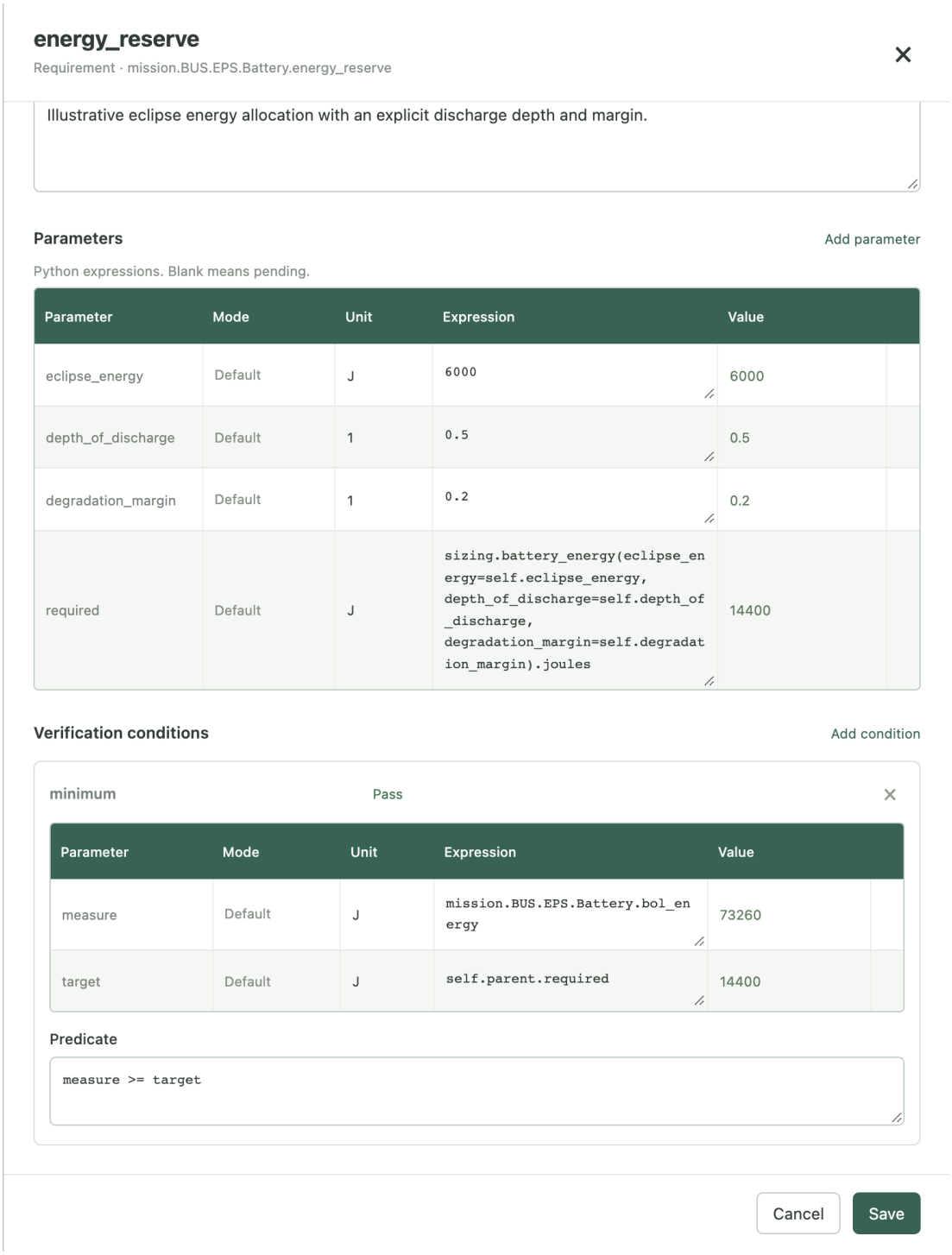


Figure 4.8: The requirement editor shows three function inputs, the `required` output expression and the measure and target bindings. The visible Save button retains these mappings in the Python session. Editing an input evaluates the draft automatically.

(a) Before: 20.35 Wh, Pass

Requirements +

Name	Requirement	Verified
energy_reserve	The battery shall provide at least 14400 J of BOL energy (available: 73260 J).	Pass

(b) After: 3.0 Wh, Fail

Requirements +

Name	Requirement	Verified
energy_reserve	The battery shall provide at least 14400 J of BOL energy (available: 10800 J).	Fail

Figure 4.9: Changing the battery rating from 20.35 Wh to 3.0 Wh reduces available energy from 73260 J to 10800 J. The target remains 14400 J. After saving the change, the same requirement changes from Pass to Fail without redefining its condition or analysis mappings.

Requirements +

Name	Requirement	Verified
sunlight_energy	The EPS shall deliver at least 18359.33042884411 J during sunlight per orbit.	Pending
eclipse_energy	The EPS shall deliver at least 6387.57239376584 J during eclipse per orbit.	Pending
solar_power	The solar array shall provide at least 8.926305241641142 W at end of life.	Pass
solar_cell_count	The solar array shall contain at least 9 cells.	Pass
battery_energy	The battery shall have at least {minimum.target} of beginning-of-life energy.	Pending

Figure 4.10: The first five of 18 requirements created when the EPS is added to the mission. Known inputs produce numerical targets for sunlight energy, eclipse energy, solar power and cell count. The battery energy statement retains its variable placeholder because mission lifetime and cycle fit inputs are unset. Supplying these quantities is the next engineering activity. The two energy delivery checks also require evidence of available energy, so a numerical target alone does not establish a passing result.

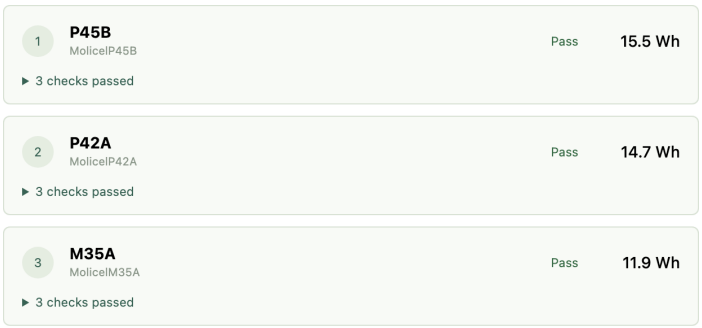
(a) Candidate acceptance

```
reports = tuple(
    requirement.verify(mode=selected_mode)
    for requirement in owner.requirements)
if any(report.verified is not True
       for report in reports):
    continue
```

Python query

```
matches = battery.recommend_cots(
    cots_catalog,
    sort_by="rated_energy",
    reverse=True, limit=3)
```

(b) GUI recommendations



(c) Expanded verification for P45B

Requirement	Measure	Target	Predicate
energy_screen check	15.5 Wh	11 Wh	`measure >= target`
mass_screen check	0.07 kg	0.075 kg	`measure <= target`
voltage_screen check	3.6 V	3.6 V	`measure == target`

Figure 4.11: The implementation rejects a candidate when any directly owned requirement is unverified. The GUI applies the same query to five sourced Molicel cells, requiring at least 11 Wh of rated energy, at most 0.075 kg of mass and a nominal voltage of 3.6 V. P45B, P42A and M35A pass and are ranked by decreasing rated energy. The cropped GUI results show these matches on the right and the expanded P45B measures, targets and predicates below.

4.4 L03: Simulation and analysis

L03 will implement FR-12 and FR-14 to FR-20 through Python expressions and numerical analyses referenced by requirements. L01 will retain the mission inputs required by FR-13 and supply them to L03, following Table 3.3. Each subsection identifies the FRs supported by its equations and algorithms. Variables will correspond to properties of MBSE model elements. Analyses will return numerical measures, comparison limits and verification results. Missing inputs will remain unknown, and invalid inputs will produce an evaluation error.

4.4.1 Mass and occupied volume (FR-12)

For FR-12, the EPS estimate will sum the contributions of its immediate components. An aggregate estimate can replace a component sum when the included components are identified. With component masses m_i in kg and occupied-volume estimates V_i in m^3, the totals and remaining allocations will be:

$$m_{\mathrm{EPS}} = \sum_i m_i, \qquad V_{\mathrm{EPS}} = \sum_i V_i, \tag{4.1}$$

$$\Delta m = m_{\mathrm{alloc}} - m_{\mathrm{EPS}}, \qquad \Delta V = V_{\mathrm{alloc}} - V_{\mathrm{EPS}}. \tag{4.2}$$

Each allocation check will pass when its remaining allocation is nonnegative. Each physical item will contribute once, including integrated COTS products represented under a generic component. Sources and units will accompany the supplied estimates. An unknown contribution will leave the affected total unresolved.

4.4.2 Orbit and eclipse durations (FR-14)

These calculations will supply the durations used by the FR-14 energy and sizing analyses from the orbit retained under FR-13. Preliminary sizing will follow El Hachimi et al. [15, Secs. 2–3, Eqs. (1)–(10)]. The eclipse model will include beta angle through the circular relation of Freimann et al. [70, Sec. IV, Eq. (6)]. Eccentric orbits will use the entry and exit mean anomaly method described by Amato [71, Sec. 3.3.5, p. 59].

The orbital model will assume Keplerian motion, a spherical Earth, a cylindrical shadow and a fixed Sun direction during each orbit. Let a_o be the semimajor axis in km, e the eccentricity and ν the true anomaly. With Earth radius $R_e = 6378$ km and gravitational parameter $\mu = 398601\ \mathrm{km}^3/\mathrm{s}^2$, the domain is $0 \le e < 1$ with $a_o(1-e) > R_e$. All angles are in radians. The orbital radius r, mean motion n and period T_o are:

$$r(\nu) = \frac{a_o(1-e^2)}{1 + e\cos\nu}, \tag{4.3}$$

$$n = \sqrt{\frac{\mu}{a_o^3}}, \tag{4.4}$$

$$T_o = \frac{2\pi}{n}. \tag{4.5}$$

The beta angle $\beta \in [-\pi/2, \pi/2]$ measures the Sun direction relative to the orbital plane. For $|\beta| < \pi/2$, let λ be the angle from perigee to the Sun's projection onto that plane, measured in the direction of motion. Shadow boundaries will be found numerically from:

$$\lambda_{\text{eff}} = \begin{cases} 0, & e = 0 \text{ or } |\beta| = \pi/2, \\ \lambda, & \text{otherwise}, \end{cases} \tag{4.6}$$

$$r(\nu)^2 \left[1 - \cos^2\beta \, \cos^2(\nu - \lambda_{\text{eff}})\right] = R_e^2, \qquad \cos\beta \, \cos(\nu - \lambda_{\text{eff}}) < 0. \tag{4.7}$$

The inequality selects the night side. Entry and exit boundaries will be paired along arcs where the left side of the equality is less than R_e^2. At each boundary, the eccentric anomaly $\mathcal{E}$ and mean anomaly $\mathcal{M}$ are:

$$\mathcal{E}(\nu) = \operatorname{atan2}\left(\sqrt{1-e^2}\sin\nu, \; e + \cos\nu\right), \tag{4.8}$$

$$\mathcal{M}(\nu) = \mathcal{E}(\nu) - e\sin\mathcal{E}(\nu), \tag{4.9}$$

$$\frac{dt}{d\mathcal{E}} = \frac{1 - e\cos\mathcal{E}}{n}. \tag{4.10}$$

Boundary anomalies will be unwrapped in the direction of motion so that each difference $\Delta\mathcal{M}_j = \mathcal{M}(\nu_{j,\text{out}}) - \mathcal{M}(\nu_{j,\text{in}})$ between entry and exit is positive. For N_e eclipse intervals per orbit, the eclipse and sunlight durations (s) are:

$$T_e = \frac{1}{n}\sum_{j=1}^{N_e} \Delta\mathcal{M}_j, \tag{4.11}$$

$$T_s = T_o - T_e. \tag{4.12}$$

An orbit without an eclipse has $N_e = 0$ and $T_e = 0$, including $|\beta| = \pi/2$. The circular limit uses $e = 0$, $a_o = R_e + h$ and any common angular reference for ν and λ, where h is altitude. At $\beta = 0$, this limit recovers El Hachimi et al.'s eclipse relation.

4.4.3 Power budgets and preliminary sizing (FR-14)

For FR-14, each component state will calculate power from its voltage and current. The engineer will bind component and subsystem powers for each assessed spacecraft mode. For the loads included in the power budget:

$$P_i = V_i I_i, \qquad P_L(t) = \sum_i P_i(t). \tag{4.13}$$

Mode allocations, duty cycles and load schedules will be explicit inputs. The analysis will use supplied mean powers $\overline{P}_{L,s}$ and $\overline{P}_{L,e}$ over the complete sunlight and eclipse intervals, including duty cycles and the EPS consumption counted in the budget:

$$E_s = \int_{\text{sunlight}} P_L(t)\, dt = \overline{P}_{L,s} T_s, \tag{4.14}$$

$$E_e = \int_{\text{eclipse}} P_L(t)\, dt = \overline{P}_{L,e} T_e. \tag{4.15}$$

The numerical calculation will multiply each supplied mean power by its duration. It will infer no schedule or allocation normalization. These expressions cover one orbit and give energy in J for power in W and time in s. Separate inputs for energy delivered to these loads will be checked against E_s and E_e. An absent interval will contribute zero energy. With corresponding energy path efficiencies X_s and X_e, the minimum solar panel power for FR-14 will be:

$$P_{\mathrm{sp}_{\min}} = \frac{\frac{E_s}{X_s} + \frac{E_e}{X_e}}{T_s}. \tag{4.16}$$

For FR-14, solar cell power at end of life (subscript EOL) will use maximum power point voltage $V_{\mathrm{mp}_{\mathrm{EOL}}}$ and current $I_{\mathrm{mp}_{\mathrm{EOL}}}$. The fractional cell count will be N_{sc}, and the reported minimum integer count will be $\lceil N_{\mathrm{sc}} \rceil$:

$$P_{\mathrm{sc}_{\mathrm{EOL}}} = V_{\mathrm{mp}_{\mathrm{EOL}}} I_{\mathrm{mp}_{\mathrm{EOL}}}, \tag{4.17}$$

$$N_{\mathrm{sc}} = \frac{P_{\mathrm{sp}_{\min}}}{P_{\mathrm{sc}_{\mathrm{EOL}}}}. \tag{4.18}$$

For FR-14, the preliminary battery cycle estimate will assume one cycle per orbit, with $N_{\mathrm{orbit/day}} = 86400\ \mathrm{s}/T_o$. For mission lifetime N_{lifetime} in years and $N_{\mathrm{day/year}}$ days per year, the required cycle count will constrain the depth of discharge DoD. Battery energy will include degradation margin M:

$$N_{\mathrm{cycle}} = N_{\mathrm{lifetime}} \cdot N_{\mathrm{day/year}} \cdot N_{\mathrm{orbit/day}}, \tag{4.19}$$

$$N_{\mathrm{life}}(\mathrm{DoD}) = a \cdot \mathrm{DoD}^b \geq N_{\mathrm{cycle}}, \tag{4.20}$$

$$E_{\mathrm{Battery}} = \frac{E_e}{\mathrm{DoD}}(1 + M). \tag{4.21}$$

The cycle fit will use supplied $a > 0$, $b < 0$ and a characterized interval $0 < \mathrm{DoD}_{\min} \leq \mathrm{DoD}_{\max} \leq 1$. The analysis will select the largest DoD within that interval satisfying Equation (4.20). A cycle demand exceeding the fit's capacity at $\mathrm{DoD}_{\min}$ will produce an error. BOL and EOL assessments will use data at their stated conditions, with each degradation factor applied once.

4.4.4 Array alternatives (FR-15 and FR-16)

For FR-15, the engineer will supply alternative cell technologies. For FR-16, the engineer will supply alternative series and parallel counts. Each alternative will have a separate mission configuration and use the same feasibility checks under common mission and load conditions. The comparison will retain raw measures, limits, verdicts and unknown values. Products, connections and counts will remain explicit selections. The framework will check each configuration against the power budget, electrical limits and installation constraints [16, Sec. 3].

For each array section g, let $N_{s,g}^{\mathrm{pv}}$ be the number of cells in series and $N_{p,g}^{\mathrm{pv}}$ the number of parallel strings. Both counts will be positive integers. For identical cells, the installed count

will be checked against Equation (4.18):

$$N_{\mathrm{pv}} = \sum_{g} N_{s,g}^{\mathrm{pv}} N_{p,g}^{\mathrm{pv}} \geq \lceil N_{\mathrm{sc}} \rceil \,. \tag{4.22}$$

For a single section, this is equivalent to checking $N_p^{\mathrm{pv}} \geq \lceil N_{\mathrm{sc}}/N_s^{\mathrm{pv}} \rceil$ for the entered series count. This preliminary count bound assumes the EOL cell power used in the power budget.

Qureshi gives the voltage sum for cells in series [72, Sec. 3.1.1, Eq. (3-1)]. For identical cells with matched illumination and temperature, ideal series/parallel scaling gives:

$$\begin{aligned} V_{\mathrm{mp},g} &= N_{s,g}^{\mathrm{pv}} V_{\mathrm{mp,cell},g}, \quad I_{\mathrm{mp},g} = N_{p,g}^{\mathrm{pv}} I_{\mathrm{mp,cell},g}, \\ V_{\mathrm{oc},g} &= N_{s,g}^{\mathrm{pv}} V_{\mathrm{oc,cell},g}, \qquad I_{\mathrm{sc},g} = N_{p,g}^{\mathrm{pv}} I_{\mathrm{sc,cell},g}. \end{aligned} \tag{4.23}$$

Here V_{mp} and I_{mp} describe the maximum power point, V_{oc} is open-circuit voltage and I_{sc} is short-circuit current. Voltages are in V and currents in A. Unequal illumination, shading and shared regulator inputs will require the combined current–voltage characteristic [38, Secs. 2.1–2.2].

For section g connected to MPPT input r, conservative extrema over the applicable temperature, irradiance and lifetime conditions will be checked against the receiver's tracking interval, absolute voltage limit and permissible source current:

$$[N_{s,g}^{\mathrm{pv}} V_{\mathrm{mp,cell},g}^{\mathrm{lo}}, N_{s,g}^{\mathrm{pv}} V_{\mathrm{mp,cell},g}^{\mathrm{hi}}] \subseteq [V_{\mathrm{track},r}^{\mathrm{lo}}, V_{\mathrm{track},r}^{\mathrm{hi}}], \tag{4.24}$$

$$N_{s,g}^{\mathrm{pv}} V_{\mathrm{oc,cell},g}^{\mathrm{hi}} \leq V_{\mathrm{abs},r}, \qquad N_{p,g}^{\mathrm{pv}} I_{\mathrm{sc,cell},g}^{\mathrm{hi}} \leq I_{\mathrm{source,allow},r}. \tag{4.25}$$

The receiving limits will be explicitly linked to the selected path. MPPT tracking intervals, absolute voltage limits and permissible source currents will remain distinct quantities. These checks will use the selected converter's supported operating conditions [15, Sec. 7.1].

4.4.5 Photovoltaic characteristics (FR-18)

For FR-18, the sizing calculation will accept an optional sampled I–V characteristic as an alternative to scalar MPP inputs. The engineer will select a coherent product, temperature, irradiance and degradation condition. For one cell, the samples (v_k, i_k), $k = 0, \ldots, K$, will have finite, nonnegative values and strictly increasing voltages. At least three samples will include short circuit $(0, I_{\mathrm{sc,cell}})$, a positive maximum power point and open circuit $(V_{\mathrm{oc,cell}}, 0)$. The calculation will form the P–V samples and extract:

$$p_k = v_k i_k, \qquad k^* = \arg\max_k p_k, \tag{4.26}$$

$$(V_{\mathrm{mp,cell}}, I_{\mathrm{mp,cell}}, P_{\mathrm{mp,cell}}) = (v_{k^*}, i_{k^*}, p_{k^*}), \tag{4.27}$$

$$(V_{\mathrm{oc,cell}}, I_{\mathrm{sc,cell}}) = (v_K, i_0), \qquad \eta_{\mathrm{PV}} = \frac{P_{\mathrm{mp,cell}}}{G_{\mathrm{ref}} A_{\mathrm{cell}}}. \tag{4.28}$$

Here $G_{\mathrm{ref}} > 0$ is the curve's reference irradiance in W/m^2 and $A_{\mathrm{cell}} > 0$ is active area in m^2, with $0 < \eta_{\mathrm{PV}} \leq 1$. The MPP will be the largest sampled product, so sparse data will include

the manufacturer's MPP. This extraction will perform no interpolation, extrapolation or diode fitting.

The curve will supply the MPP power used by the FR-14 cell-count calculation and the electrical values scaled by Equation (4.23). The assembled values will be checked against the assigned limits in Equations (4.24)–(4.25), supporting FR-18. Applicable EOL curves will provide EOL sizing inputs. An empty curve will preserve the scalar sizing calculation. A purchased panel represented as one item will use its panel characteristic without repeating its internal cell counts.

4.4.6 DET and MPPT operating points (FR-19)

For FR-19, each array section will reference its explicitly selected power path. DET will use the battery or bus voltage plus the blocking and wiring drop $\Delta V_g \geq 0$ at the assessed current. Ideal MPPT will use the section's reference MPP voltage. Fixed input-voltage regulation will use a separately supplied setpoint:

$$V_{\mathrm{op},g} = \begin{cases} V_{\mathrm{bus}} + \Delta V_g, & \text{DET}, \\ V_{\mathrm{mp},g}, & \text{ideal MPPT}, \\ V_{\mathrm{set},g}, & \text{fixed input-voltage regulation}. \end{cases} \tag{4.29}$$

The supplied cell curve will be scaled to the entered series and parallel counts. An exact sampled voltage will return its sampled current. When the engineer explicitly accepts linear interpolation between adjacent assembled samples (V_j, I_j) and (V_{j+1}, I_{j+1}), the operating current will be:

$$I_g(V) = I_j + \frac{V - V_j}{V_{j+1} - V_j}(I_{j+1} - I_j), \qquad V_j < V < V_{j+1}. \tag{4.30}$$

The default will accept exact samples only. Voltages outside the supplied characteristic will produce an error. Scalar MPP inputs will suffice when the operating voltage equals the reference MPP voltage. Other operating voltages will require the selected curve. For FR-19, the operating power and the efficiency used by the area and generation calculations will be:

$$P_{\mathrm{op},g} = V_{\mathrm{op},g} I_g(V_{\mathrm{op},g}), \tag{4.31}$$

$$\eta_{\mathrm{op},g} = \eta_{\mathrm{ref},g} \frac{P_{\mathrm{op},g}}{P_{\mathrm{mp},g}}. \tag{4.32}$$

Here $\eta_{\mathrm{ref},g}$ will be the supplied or calculated EOL efficiency for scalar inputs, or $\eta_{\mathrm{PV},g}$ from Equation (4.28) for curve inputs. These efficiencies will retain the constant-efficiency irradiance approximation and a fixed normalized I–V shape within each assessed case. Changing temperature, irradiance or battery voltage will require applicable curves or an externally established mean power.

For the simple DET path and an MPPT converter with characterized efficiency η_{conv}, the bus powers before additional consumption will be:

$$P_{\mathrm{bus,DET}} = V_{\mathrm{bus}} I_g(V_{\mathrm{op},g}), \qquad P_{\mathrm{bus,MPPT}} = \eta_{\mathrm{conv}} P_{\mathrm{op},g}. \tag{4.33}$$

The engineer will compare DET and MPPT under common array, battery, illumination and load conditions, using their respective path losses and the electrical checks above. Energy delivered to the loads will remain a separate input unless established by an applicable calculation. Ideal MPP extraction will establish a reference operating point. Actual tracker startup, tracking accuracy and conversion feasibility will require the selected controller's data. Fixed-voltage regulation will establish no automatic MPP search.

4.4.7 Array area and illumination (FR-14)

For FR-14, the array area and illumination algorithm will use the following body dimensions, mounting faces and section normals. The same calculations will support the FR-15 and FR-16 comparisons:

$$(\ell_x, \ell_y, \ell_z)_{\text{nom}} = 0.1 \text{ m} \begin{cases} (1, 1, N_U), & N_U \in \{1, 1.5, 2, 3\}, \\ (2, 1, 3), & N_U = 6, \\ (2, 2, 3), & N_U = 12, \end{cases} \tag{4.34}$$

$$\mathcal{F} = \{+X, -X, +Y, -Y, +Z, -Z\}, \qquad f_g \in \mathcal{F}, \tag{4.35}$$

$$\hat{\mathbf{n}}^B(\pm X) = \pm(1, 0, 0)^{\mathsf{T}}, \quad \hat{\mathbf{n}}^B(\pm Y) = \pm(0, 1, 0)^{\mathsf{T}}, \quad \hat{\mathbf{n}}^B(\pm Z) = \pm(0, 0, 1)^{\mathsf{T}}, \tag{4.36}$$

$$A_{\text{body},g} = \begin{cases} \ell_y \ell_z, & f_g = \pm X, \\ \ell_x \ell_z, & f_g = \pm Y, \\ \ell_x \ell_y, & f_g = \pm Z, \end{cases} \qquad \ell_x, \ell_y, \ell_z > 0, \tag{4.37}$$

$$\hat{\mathbf{n}}_g^B = \frac{\mathbf{d}_g^B}{\|\mathbf{d}_g^B\|}, \qquad \mathbf{d}_g^B \neq \mathbf{0}, \tag{4.38}$$

$$\mathbf{d}_g^B = \hat{\mathbf{n}}^B(f_g) \quad \text{(flush body mounting)}.$$

For a cell assembly with active area $A_{\text{cell},g}$ per cell, active array area and required panel area will be calculated separately:

$$A_{\text{active},g} = N_{s,g}^{\text{pv}} N_{p,g}^{\text{pv}} A_{\text{cell},g}, \qquad A_{\text{panel},g} = \frac{A_{\text{active},g}}{f_{\text{packing},g}} \leq A_{\text{usable},g}. \tag{4.39}$$

All areas are in m^2, with total active area $A_{\text{active}} = \sum_g A_{\text{active},g}$. The packing fraction $0 < f_{\text{packing},g} \leq 1$ is the active cell area divided by panel area. The usable area will exclude mounting and access clearances. Installation verification will also require a realizable cell layout within the available dimensions. Finished panels will use their specified active area and installation envelope.

For FR-14, normal-incidence area will follow Nogrady's relation [73, Sec. 4.3.4]. With assessed irradiance $G > 0$ in W/m^2, the comparison area will use the selected operating efficiency from Equation (4.32):

$$\eta_{\text{EOL}} = \eta_{\text{BOL}} L_d f_T, \qquad A_{\text{effective,min}} = \frac{P_{\text{sp}_{\text{min}}}}{G \eta_{\text{op}}}. \tag{4.40}$$

Here η_{BOL} is cell efficiency, L_d is lifetime retention and f_T is the temperature correction at the assessed condition. The first relation will apply when EOL efficiency is derived from BOL data. Supplied EOL efficiencies or curves will provide the reference efficiency directly. Efficiencies and packing fractions will be expressed as fractions. Supplied EOL data will already include their stated degradation and temperature effects. For an array assessed at normal incidence, the active area will be checked for $A_{\mathrm{active}} \geq A_{\mathrm{effective,min}}$.

For FR-14, prescribed Sun or nadir pointing will determine each section's projected illumination. Attitude inputs retained under FR-13 will specify the pointing face and clocking angle:

$$\hat{\mathbf{s}}^O(\nu) = \begin{bmatrix} \cos\beta\cos(\nu-\lambda_{\mathrm{eff}}) \\ -\cos\beta\sin(\nu-\lambda_{\mathrm{eff}}) \\ \sin\beta \end{bmatrix}, \qquad O=(R,T,N), \tag{4.41}$$

$$(R,T,N) = (\text{radial outward}, \text{along track}, \text{orbit normal}),$$

$$\hat{\mathbf{a}}^B = \hat{\mathbf{n}}^B(f_p), \qquad f_p \in \mathcal{F}, \tag{4.42}$$

$$\hat{\mathbf{b}}^B = \hat{\mathbf{e}}^B_{j(i)}, \qquad f_p = \pm i, \quad (i,j(i)) \in \{(X,Y),(Y,Z),(Z,X)\},$$

$$\hat{\mathbf{c}}^B = \hat{\mathbf{a}}^B \times \hat{\mathbf{b}}^B, \qquad \phi \in \mathbb{R} \quad (\text{clocking angle}), \tag{4.43}$$

$$\hat{\mathbf{n}}_g^O = \begin{bmatrix} -1 & 0 & 0 \\ 0 & \cos\phi & -\sin\phi \\ 0 & -\sin\phi & -\cos\phi \end{bmatrix} \begin{bmatrix} \hat{\mathbf{n}}_g^B\cdot\hat{\mathbf{a}}^B \\ \hat{\mathbf{n}}_g^B\cdot\hat{\mathbf{b}}^B \\ \hat{\mathbf{n}}_g^B\cdot\hat{\mathbf{c}}^B \end{bmatrix} \quad (\text{nadir pointing}), \tag{4.44}$$

$$\kappa_g(\nu) = \begin{cases} \max\left(0, \hat{\mathbf{n}}_g^B\cdot\hat{\mathbf{a}}^B\right), & \text{Sun pointing}, \\ \max\left(0, \hat{\mathbf{n}}_g^O\cdot\hat{\mathbf{s}}^O(\nu)\right), & \text{nadir pointing}, \end{cases} \tag{4.45}$$

$$\overline{\kappa}_g = \frac{1}{T_s}\int_{\mathrm{sunlight}} \kappa_g(\nu(t))\,dt \in [0,1], \tag{4.46}$$

$$= \frac{1}{nT_s}\int_{\mathcal{E}(\mathrm{sunlight})} \kappa_g(\nu(\mathcal{E}))(1-e\cos\mathcal{E})\,d\mathcal{E}.$$

For FR-14, Vasile and Alicino include inherent degradation, electrical assembly losses and incidence angle in their area calculation [74, Sec. II.1]. With the selected operating efficiency, the sunlight-mean power density and active area bound will be:

$$\overline{p}_s = G\eta_{\mathrm{op}} I_d \eta_a \overline{\kappa}, \qquad \overline{p}_s > 0, \tag{4.47}$$

$$A_{\mathrm{active,min}} = \frac{P_{\mathrm{sp}_{\mathrm{min}}}}{\overline{p}_s} = \frac{P_{\mathrm{sp}_{\mathrm{min}}}}{G\eta_{\mathrm{op}} I_d \eta_a \overline{\kappa}}, \tag{4.48}$$

$$\kappa(\nu) \equiv \cos\theta > 0 \quad \Longrightarrow \quad \overline{\kappa} = \cos\theta.$$

The last relation recovers the fixed-incidence expression, where θ is the angle between the array normal and the Sun direction. The active area will be checked for $A_{\mathrm{active}} \geq A_{\mathrm{active,min}}$ under these common conditions. The factors I_d and η_a represent inherent degradation and electrical assembly efficiency. Each loss will enter the assessment once. The path efficiencies in Equation (4.16) will describe losses from the assessed array output to the loads. Packing will remain a separate geometric constraint.

For FR-14's preliminary tumbling approximation, Nogrady considers a CubeSat with cells on all six faces and assumes half the faces are illuminated at an average incidence angle of 50°. With uniform packing, the required total panel area becomes [73, Sec. 4.3.4, Eq. (27)]:

$$A_{\text{tumbling,min}} \approx \frac{2A_{\text{effective,min}}}{\cos(50^\circ) f_{\text{packing}}}. \tag{4.49}$$

For this geometry, the check will be $\sum_g A_{\text{panel},g} \geq A_{\text{tumbling,min}}$. For FR-14 and FR-19, each section's available source power $P_{g,\text{avail}}(t)$ will use its selected operating point and the calculated sunlight-mean projection [70, Sec. II]. An external attitude and illumination analysis can instead supply an applicable mean power. With assessed irradiance G_g and $T_s > 0$, generated and required power will be compared at the same point in the circuit:

$$\overline{p}_{g,s} = G_g \eta_{\text{op},g} I_{d,g} \eta_{a,g} \overline{\kappa}_g, \tag{4.50}$$

$$\overline{P}_{g,s} = A_{\text{active},g} \overline{p}_{g,s}, \tag{4.51}$$

$$\tau_{\text{proj},g} = T_s \overline{\kappa}_g, \tag{4.52}$$

$$E_{\text{source},g} = T_s \overline{P}_{g,s}, \tag{4.53}$$

$$\sum_g \overline{P}_{g,s} = \frac{1}{T_s} \int_{\text{sunlight}} \sum_g P_{g,\text{avail}}(t)\, dt = \frac{\sum_g E_{\text{source},g}}{T_s} \geq P_{\text{sp}_{\text{min}}}. \tag{4.54}$$

The assessment will use the selected configuration's path efficiencies. Solar cell count and installed area will therefore be accompanied by an illumination-dependent generation verdict. These scalar means will assume fixed irradiance, curve conditions and path settings during each assessed case. The sunlight-time projection will account for eclipse without another eclipse duty factor.

4.4.8 Battery capacity and configuration (FR-14 to FR-16)

These equations will report battery capacity for FR-14 and provide common technology and series/parallel checks for FR-15 and FR-16. Let N_s^{bat} and N_p^{bat} be the entered positive integer series and parallel counts. For identical cells with balanced current sharing, the initial cell-based pack estimates will be:

$$\begin{aligned} N_{\text{bat}} &= N_s^{\text{bat}} N_p^{\text{bat}}, \\ E_{\text{pack,BOL}} &= N_s^{\text{bat}} N_p^{\text{bat}} E_{\text{cell,BOL}}, \\ Q_{\text{pack,BOL}} &= N_p^{\text{bat}} Q_{\text{cell,BOL}}. \end{aligned} \tag{4.55}$$

$$\begin{aligned} \left[V_{\text{pack}}^{\text{lo}}, V_{\text{pack}}^{\text{hi}}\right] &= \left[N_s^{\text{bat}} V_{\text{cell}}^{\text{lo}}, N_s^{\text{bat}} V_{\text{cell}}^{\text{hi}}\right], \\ I_{\text{pack,allow}} &= N_p^{\text{bat}} I_{\text{cell,allow}}. \end{aligned} \tag{4.56}$$

Energy E will be expressed in J and charge capacity Q in Ah, with 1 Wh = 3600 J. Interconnects, balancing and protection can reduce the cell-based ratings. Finished packs will use their supplier's pack ratings.

The configured pack will be checked against Equation (4.21). For the entered series count and $E_{\text{cell,BOL}} > 0$, the energy check can also be expressed as a lower bound on parallel strings:

$$E_{\text{pack,BOL}} \geq E_{\text{Battery}}, \qquad N_p^{\text{bat}} \geq \left\lceil \frac{E_{\text{Battery}}}{N_s^{\text{bat}} E_{\text{cell,BOL}}} \right\rceil. \tag{4.57}$$

The framework will check the pack voltage range against its connected charge and discharge paths. For FR-15, the engineer will supply product-specific conditions for charger chemistry, series count, charging voltage, current and termination compatibility [38, Secs. 2.2–2.3].

For supplied retention fractions f_E and f_Q, EOL capacity will be:

$$E_{\text{pack,EOL}} = f_E E_{\text{pack,BOL}}, \qquad Q_{\text{pack,EOL}} = f_Q Q_{\text{pack,BOL}}. \tag{4.58}$$

For FR-14, the margin M will enter preliminary sizing once through Equation (4.21). Retained energy and charge will use their separate supplied factors. These scalar capacity and rating checks will apply to the assessed configuration and conditions.

4.4.9 Bus voltage and regulation choices (FR-17)

For FR-17, bus voltage alternatives and buck/boost choices will use the explicitly selected converter's input interval, output setting, output current rating and characterized efficiency. The source interval, including the battery bounds from Equation (4.56), will be checked against the connected converter's operating input interval:

$$[V_{\text{src}}^{\text{lo}}, V_{\text{src}}^{\text{hi}}] \subseteq [V_{\text{in,allow}}^{\text{lo}}, V_{\text{in,allow}}^{\text{hi}}]. \tag{4.59}$$

For the assessed load rail and state, the nominal voltage and current checks will be:

$$|V_{\text{out}} - V_{\text{required}}| \leq \epsilon_V, \qquad 0 \leq I_{\text{required}} \leq I_{\text{out,allow}}. \tag{4.60}$$

The accepted voltage tolerance $\epsilon_V \geq 0$ will be explicit and will default to zero for an exact nominal match. Currents will be added only on the same rail at the same operating point. Battery and converter input currents will be calculated at their respective terminals, including applicable conversion losses. The engineer will link limits to each connected path and assess each selected mode. Supported output settings, dropout and converter-specific operating limits will require explicit conditions from the selected product. These checks will establish operating-point feasibility. They will infer no converter topology or switching behavior.

4.4.10 Protection and electrical extrema (FR-20)

For FR-20, adding a protection component will create a requirement that its declared detectors remain inactive for the supplied normal operating cases. The engineer will supply voltage samples at the detector's sensed node and nonnegative current magnitudes in the protected direction. For case index q, the extrema will be:

$$(V_{\min}, V_{\max}) = (\min_q V_q, \max_q V_q), \qquad (I_{\min}, I_{\max}) = (\min_q I_q, \max_q I_q). \tag{4.61}$$

A power budget can supply current bounds when power and voltage refer to the same point in the protected circuit. The power values will include applicable conversion losses. For nonnegative powers and $0 < V_{\min} \le V_{\max}$, the conservative independent power/voltage bounds will be:

$$I_{\min} = \frac{P_{\min}}{V_{\max}}, \qquad I_{\max} = \frac{P_{\max}}{V_{\min}}. \tag{4.62}$$

These bounds will assume no pairing or chronology between the power and voltage cases. An orbit-average power budget will establish energy demand. Peak-current checks will require peak power or independently established current cases.

For FR-20, the COTS comparison will use the highest applicable undervoltage trip threshold $V^{\mathrm{hi}}_{\mathrm{UV,trip}}$, the lowest applicable overvoltage trip threshold $V^{\mathrm{lo}}_{\mathrm{OV,trip}}$ and the lowest applicable overcurrent trip threshold $I^{\mathrm{lo}}_{\mathrm{OC,trip}}$:

$$\begin{aligned} V_{\min} &> V^{\mathrm{hi}}_{\mathrm{UV,trip}}, \\ V_{\max} &< V^{\mathrm{lo}}_{\mathrm{OV,trip}}, \\ 0 \le I_{\min} \le I_{\max} &< I^{\mathrm{lo}}_{\mathrm{OC,trip}}. \end{aligned} \tag{4.63}$$

Only the detectors declared for the selected product will be checked. Thresholds will be nonnegative and selected for the applicable resistor settings, temperature and configuration. Cell-level detectors will use individual cell voltages. Sense-voltage current thresholds will require an explicit conversion to amperes. Equality will fail this conservative check. Missing samples or thresholds will remain unresolved. Empty or nonfinite sample sets will produce an error.

A passing result will establish that the supplied normal operating cases remain within the declared no-trip bounds for an enabled, recovered device. A failed comparison will identify a possible trip. Trip delay, hysteresis, startup, fault interruption and recovery will require separate evidence. This will define the scope of the FR-20 analysis.

4.5 L04: Configuration Management

For FR-09, Git retains the Python model configuration and analysis functions as text, allowing changes to individual property expressions to be compared (Figure 4.12). Each commit identifies a retained baseline with its author and rationale. A GitHub repository can synchronize these baselines across devices. Verification results are recalculated from the retained inputs and analysis definitions.

(a) Git comparison with the retained baseline

```
$ git diff -- battery-study.py
diff --git a/battery-study.py b/battery-study.py
index ad30670..19d3b45 100644
--- a/battery-study.py
+++ b/battery-study.py
@@ -25,3 +25,3 @@ eps.SolarArray.parallel_count = 4
 battery = eps.Battery
-battery.rated_energy = 20.35
+battery.rated_energy = 3.0
 requirement = battery.add_requirement(
```

(b) Baseline evaluation

```
>>> battery.bol_energy.value
73260.0
>>> requirement.verify().verified
True
```

(c) Edited evaluation

```
>>> battery.bol_energy.value
10800.0
>>> requirement.verify().verified
False
```

Figure 4.12: A Git diff records the same battery rating change used in Figure 4.9. Evaluating the retained 20.35 Wh configuration gives 73260 J and a passing requirement. Evaluating the edited 3.0 Wh configuration gives 10800 J and a failing requirement against the unchanged 14400 J target. The verdicts are calculated results, separate from the stored input change.

Chapter 5

Results and Analysis

This section reports the verification evidence for the framework requirements. The results of the two case studies represent the deliverable of the fifth objective of this project.

5.1 User trials

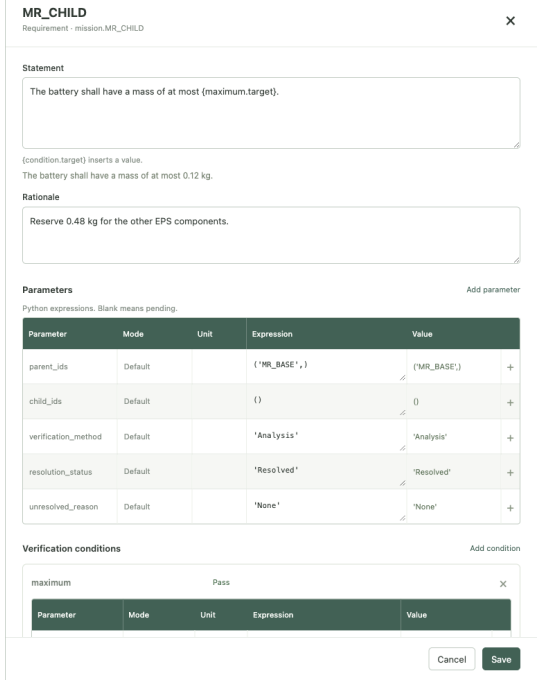
The user trials assess the time required to maintain requirements, retrieve supporting evidence, assess changes and evaluate supplied alternatives through the framework. The protocol in Table 3.2 assigns eleven tasks to each participant, with a limit of 60/11 min per task and a total allocation of 60 min. Each task uses the same prepared initial state for both participants. The mission conditions and numerical allocations are purely illustrative. They provide controlled starting points for measuring the effort required to perform only the assigned engineering activities, not an entire mission definition.

The trials use the GUI and a prepared mission environment. For FR-09, participants are also given access to a terminal. Before that task, they are instructed to use `git status --short` to view repository status and `git diff` to inspect changes, and practice both commands on the prepared baseline. Preparation and this briefing occur before timing begins. Task time runs from presentation of the task to submission. Submission is when the participant declares verbally that they have completed the task, after which a screenshot is taken. Only the screenshots from the student club member are presented in this document, since the deterministic analyses of the framework produced the same outputs under equal inputs for both participants.

Each MOP compares its two elapsed times with the limit separately, without averaging or any metric about the correctness of the results. Table 5.1 shows the first task, and Appendix A gives the prepared inputs and the complete table of user trial results.

Both times for each of the eleven user trial MOPs are below the limit. The sums of the eleven times are 33 min 57.04 s for the PhD role and 35 min 34.56 s for the club role. Individual times range from 79.60 to 318.16 s. The longest time, for MOP-06 in the club role, is 9.11 s below the per-task limit.

Table 5.1: First user trial task. Screenshots show the shared initial state and the student club member's submission. Times are in seconds.

MOP/Task	Shared initial state	Final state of student club member	Time taken by PhD student	Time taken by student club member	Verifies FR?
MOP-01 FR-01 Create and retain a requirement			226.88	243.76	Yes

The 60 min allocation reflects the duration of the stakeholders' weekly meeting. Verification of all eleven user trial MOPs by both stakeholders would suggest that they can assess the impact of requirement changes and evaluate supplied alternatives within that meeting. This is considerably faster than what was observed during stakeholder analysis, when stakeholders would have to wait until the next weekly meeting to see the results of tasks comparable to those considered in the user trials.

Furthermore, the fact that a single engineering tool was used across requirements, analyses and component comparisons mitigates the core problem of evidence fragmentation. The framework retains the relationships between requirements, design inputs and supporting results in one model, which reduces the effort needed to reconstruct those relationships during a review and supports evaluating alternatives on a common basis. The Git commands run in the terminal provide the configuration control required for FR-09, including inspection of changes and retrieval of an authorized baseline.

It must be noted that the user trials were attempted multiple times with the same participants, and the presented results represent the last iteration. Previous iterations were considered failures since the participants found the user interface so unclear that the entirety of the allocated time was spent in explanations of usage. Failed trials provided qualitative feedback with which to improve the GUI. However, a discussion of the evolution of the GUI and the user feedback is out of the scope of this document.

5.2 Catalog inspection

Table 5.2) provides the counts of COTS components added to the L02 database, per category. Component counts satisfy MOP-10, and thus verify FR-10. Battery and solar array categories contain 13 and 12 products, respectively. The DC/DC converter and regulator/converter categories together contain 20 products. These catalog categories describe product functions.

Table 5.2: COTS product counts by catalog category.

Category	Products	Category	Products
Battery	13	Solar array	12
DC/DC converter	17	Regulator/converter	3
Integrated EPS	3	Battery charger	5
Battery protector	6	Ideal diode	7
Load switch	17	Power circuit switch	11
Current sensor	13	Temperature sensor	11
Sensor	3	Heater	6
Processor	8	Nonvolatile memory	5
Real-time clock	5	Hardware component	2
Total distinct products			147

5.3 EPS case studies

The EPS case studies verify the calculations required by FR-12 and FR-14 to FR-20 using BIRDS-4 and OreSat data. Mission publications supply component ratings and selected calculation results [46, 51, 53, 75]. Explicit assumptions supply the remaining inputs.

5.3.1 Verification setup

The BIRDS electrical case uses a two-cell 3G30A solar section and a 3S2P NiMH battery. The OreSat case uses a two-cell XTE-SF section and two independent 2S1P lithium-ion packs. Both baseline sections face the Sun. The comparisons also cover changes in illumination, cell technology, series and parallel arrangements, bus voltage and power-transfer method. Appendix D groups the inputs and selected results by the requirement they verify.

Independent reference values were calculated directly from the input data, without calling the framework's numerical functions. The reference summed component masses and volumes, calculated duty-weighted state powers, and obtained phase energies from mean power and duration. Loss-adjusted phase energies determined source-power demand. Battery sizing used eclipse demand, the assumed cycle-life/DoD relation and the energy margin. Power and energy ratios were rounded upward to obtain integer cell counts. These sums, products and ratios used decimal arithmetic, with 40 significant digits for products and ratios. A separate orbital reference solved Kepler's equation at uniformly spaced times and tested Earth shadow in Cartesian coordinates. This provided an independent check of the framework's shadow-boundary and panel-projection calculations.

Each nonzero numerical output had to agree with its reference within 1%. Zero values, integer counts and discrete outcomes required exact agreement. The appendix reports maximum deviations and representative comparisons, with repeated outputs and deliberately failing cases omitted from the detailed tables.

5.3.2 Results and discussion

The numerical comparisons met the 1% criterion for every assessed requirement in both mission cases. Integer counts and discrete outcomes also matched their references. No unexpected mismatch was found. Appendix D separates the evidence for mass accounting, sizing, component choices, electrical configurations, conversion, photovoltaic characteristics, transfer paths and protection.

The largest deviation was 0.331% in the BIRDS photovoltaic power comparison for FR-18. The framework calculated $5.123 \times 0.050 = 0.25615$ W, compared with the published 0.257 W label. The next largest deviation was 0.315% for FR-14, where the OreSat cell ratings gave 2.34738 W against a published two-cell value of 2.34 W. Both comparisons use printed values with limited precision. They check consistency with the publications at the stated tolerance, while the separately calculated references provide a more precise check of the arithmetic.

The largest deviation from an independent calculation was 0.000864% in the BIRDS battery-energy requirement. The framework returned 4243.934 J against 4243.970 J from the reference. This difference follows from the finite time grid used to estimate eclipse duration in the orbital reference. Doubling that grid changed eclipse duration by at most 0.00205%. The minimum integer cell counts remained unchanged. The small energy difference therefore had no effect on the sizing decisions checked here.

These results support using the framework for EPS accounting, preliminary sizing and comparisons of entered component options. The evidence includes independent calculations of both the component quantities and the orbital geometry that links illumination to the power budget. For FR-15 and FR-16, the technology and arrangement comparisons reproduced the expected changes in power, voltage, energy and capacity. For FR-17, FR-19 and FR-20, terminal currents, transfer-path powers and protection comparisons agreed with the separately evaluated electrical relations. This connects the numerical agreement to the decisions the framework is intended to support.

The remaining uncertainty concerns specific input models. The complete verification curves use three photovoltaic points, and the battery life fits are assumed. The published BIRDS 1.575 W measured peak remains unreconstructed. OreSat's partial solar sweep supplies power-product checks but lacks the endpoints needed for a complete characteristic. The case studies consequently provide stronger evidence for correct calculation and comparison than for reproducing either mission's measured energy performance.

5.4 Economic Analysis

The analysis follows Ulrich and Eppinger's sequence of a base case, sensitivity analysis, development tradeoffs and qualitative assessment [76, Chap. 17]. The comparison is between using the framework and continuing the as-is workflow with two funded researchers. ASCEND does not sell services, so the monetary benefit considered here is the value assigned to personnel time released for other research. Cash expenditure is assessed separately. The modeled period comprises four months of development followed immediately by three mission years, following the CUBICS mission lifespan. All amounts are held at constant 2026 Canadian dollar values.

5.4.1 Base case and included costs

ASCEND is assumed to fund the author's (to be understood as author of the framework) scholarship of CA$1500 per month for four months devoted entirely to this project. The resulting scholarship expense is CA$6000. The author also paid CA$27.68 per month for an OpenAI subscription during those four months to assist with extracting COTS parameters from datasheets. This initial subscription expense totals CA$110.72. The combined direct development expenditure is:

$$C_{\text{direct}} = 4(1500) + 4(27.68) = 6110.72. \tag{5.1}$$

Of this CA$6110.72, CA$6000 is assigned to ASCEND and CA$110.72 was paid by the author.

Initial supervision and user trial participation are covered by existing funded positions. Their effort is excluded from the personnel allocation used here. The two researchers and the personnel responsible for subsequent maintenance are each valued at CA$1500 per month for a 35 h working week, comprising seven hours from Monday to Friday. Using 52 weeks and 12 months per year gives the hourly funding proxy:

$$c_h = \frac{12(1500)}{52(35)} \approx 9.89 \text{ CA\$/h}. \tag{5.2}$$

This proxy values research capacity at its stipend funding cost. The base case assumes that all released time is used for other productive research. Stipend payments remain unchanged in both workflows.

5.4.2 Review effort and maintenance

The effort baseline uses a comparable update delivered two days later by two participants who reported working only on that activity. The base case assigns two full working days of seven hours to each person and treats that instance as typical. This gives 28 combined hours per review cycle. Section 5.1 gives complete trial times of 33 min 57.04 s and 35 min 34.56 s. For this estimate, each participant's complete eleven-task trial is rounded up to one hour to include instructions and preparation. A comparable cycle with the framework therefore takes two combined hours and releases 26 hours of personnel time.

Comparable work occurred weekly during the four-month observation period. The forecast assumes the same frequency for the first 18 mission months, covering Phases 0 and A. One cycle per month is assumed for the next six months of Phase B. No further review cycles are credited in the third mission year. Weekly activity is distributed across months using 52/12 cycles per month, with no separate reduction for holidays or breaks. These assumptions give 78 weekly cycles and six monthly cycles, releasing 2184 hours before maintenance.

Maintenance and database expansion consume ten combined personnel hours per month during the first nine mission months. This effort assumes AI assistance for COTS parameter extraction. One subscription at CA$27.68 per month is therefore included during those nine months. This initial period covers the first year's maintenance. One further month of ten combined hours and one subscription payment is assigned to each of mission years two and three. These annual activities are placed at the end of their respective years. Total support effort is 110 hours, and the eleven additional subscription payments total CA$304.48. The resulting net released time is 2074 hours.

5.4.3 Net present value

A 7% annual real discount rate is adopted from the Treasury Board of Canada Secretariat's guidance for Canadian regulatory cost-benefit analysis [77, Sec. 6.1]. It serves as a Canadian

reference rate for this assessment. The equivalent monthly rate is $i = (1.07)^{1/12} - 1$. The valuation date is the start of the internship, and all expenditure and capacity benefits are placed at month end. Development comprises four payments of CA$1527.68. For mission month m, let H_m denote the released hours before maintenance, M_m the maintenance hours and A_m the additional subscription expense. The economic net present value is:

$$V_{\text{econ}} = -\sum_{t=1}^{4} \frac{1527.68}{(1+i)^t} + \sum_{m=1}^{36} \frac{c_h(H_m - M_m) - A_m}{(1+i)^{m+4}}. \tag{5.3}$$

Table 5.3 gives the period totals using the unrounded hourly proxy and discount rate. The base case gives an economic net present value of approximately CA$12614.

Table 5.3: Economic value by period in constant 2026 Canadian dollars. Mission years follow the four-month internship. Present values use monthly discounting, and totals use unrounded values.

Period	Released time (h)	Maintenance time (h)	Net economic value (CA$)	Present value (CA$)
Development	0	0	−6110.72	−6025.31
Mission year 1	1352	90	12232.20	11522.21
Mission year 2	832	10	8101.99	7218.11
Mission year 3	0	10	−126.58	−101.02
Total	2184	110	14096.89	12613.98

The cash assessment credits no income or payroll reduction from the released time. The continuing stipends are common to both workflows and cancel from the incremental cash comparison. Development and subsequent subscription payments total CA$6415.20, with a cash net present value of −CA$6307.86. The positive economic result represents the assigned value of additional research capacity.

5.4.4 Sensitivity and interpretation

The baseline effort is recalled from one instance, and the trial times follow earlier unsuccessful trials with prepared inputs. There is no controlled comparison of work effort under the two workflows. The estimate therefore depends on the assumed transfer of that effort difference to repeated mission reviews. Holding other assumptions fixed, the economic net present value reaches zero at approximately 5.77 baseline hours per person and cycle, compared with the assumed 14 hours. A separate sensitivity retains the 14-hour baseline and varies the fraction of released time credited at the hourly proxy. The economic result reaches zero when approximately 36.7% of that capacity receives its full assigned value, while all support costs remain included. Using the alternative 3% social rate discussed in the Treasury Board guide gives approximately CA$13431 [77, Sec. 6.1].

The zero yearly cost for C3 in Table 3.5 is interpreted as zero core software license expenditure. The AI subscription and personnel effort are separate support costs. No avoided

subscription payments for C1, C2 or C4 are credited, since the comparison uses the as-is workflow. This valuation includes development expenditure, which is a sunk cost for subsequent support decisions [76, Chap. 17, App. A].

The EPS case studies in Section 5.3 demonstrate detection of infeasible configurations before hardware fabrication. No avoided hardware purchase, fabrication or paid service is monetized, and no residual value is credited at the end of the assessment. The additional value of reusable models, traceable evidence and student experience remains qualitative. CUBICS 2026 Stream 2 offers a maximum grant of CA$350000 over up to three years [8, Sec. 7.1]. The modeled cash expenditure is approximately 1.83% of that ceiling. The positive economic value supports an internal capability investment under the stated effort, utilization and support assumptions.

Chapter 6

Conclusions

The review of normative documents and state-of-the-art research on MBSE in Chapter 2 characterized the needs, requirements, interfaces and technical criteria of nanosatellite EPS, fulfilling the first objective of this project. The second objective, to define the operational flow of modeling, analysis, and evidence management of the MBSE framework, was delivered as the Python code that constitutes the framework's models and analyses (presented in full in Appendix E), which is plain text, and can be version-controlled.

The third objective's deliverable can also be found in Appendix E, corresponding to the CSRM model classes. For the fourth objective, the characterization early EPS verification and validation within the EPS framework was achieved by the automatic assignment of parametric requirements to generic EPS components, for which the list of requirements is given in Appendix B.

Finally, the two case studies of the BIRDS-4 and OreSat CubeSat missions discussed in Chapter 5 delivered the fifth objective. The results suggested that the outputs of the analyses bundled with the framework were accurate with respect to the original BIRDS-4 and OreSat values calculated using the same inputs. This implies that the parametric requirements modeled by the framework don't just link quantities but represent real EPS engineering knowledge. User trials then motivated the conclusion that, under controlled conditions and represent individual tasks and not a complete mission definition, stakeholders can now manage parametric requirements, assess the impact of a requirement change, and reproduce analyses much faster than before by leveraging the framework.

Continued adoption and improvement of the framework therefore appears to be a feasible way to mitigate the core problem of evidence fragmentation that motivated this project, and in turn produces positive economic value. This in turn strenghtens the systems engineering capabilities at ASCEND as they look forward to applying to the next cohort of the CUBICS initiative.

Future work should expand the COTS database with as many components and parameters as possible and useful, covering subsystems other than EPS. In particular, time-dependent properties would enable analyses much more detailed than those currently implemented using

only time-independent parameters.

The MBSE models should be expanded to automatically add representative requirements for all subsystems and components defined by CSRM, not just those related to the EPS.

The fact that the models are implemented in plain-text Python makes them legible for artificial intelligence agents. The potential of such agents to further speed up design processes in MBSE while respecting their modeled constraints is an interesting subject of study.

Bibliography

[1] E. Kulu, "Nanosats database." [Online]. Available: https://www.nanosats.eu/database

[2] R. B. Buruso, S. Himmiche, T. Tejumola, Y. Alemayehu, and B. Rose, "Model-based systems engineering in space applications: A comprehensive literature review," *Systems Engineering*, vol. 29, no. 5, pp. 975–1017, 2026. [Online]. Available: https://incose.onlinelibrary.wiley.com/doi/abs/10.1002/sys.70051

[3] [Online]. Available: https://github.com/anthonyarguedas/nanosatellite-mbse

[4] National Aeronautics and Space Administration, "NASA systems engineering handbook," National Aeronautics and Space Administration, Washington, DC, NASA Special Publication NASA/SP-2016-6105 Rev2, 2016. [Online]. Available: https://www.nasa.gov/wp-content/uploads/2018/09/nasa_systems_engineering_handbook_0.pdf

[5] T. Zhu, H. Wang, Z. Cao, S. See, and Y. Wen, "Towards intelligent systems for battery management: A five-tier digital twin architecture," arXiv preprint arXiv:2509.02366, 2025, version 2, 12 January 2026. [Online]. Available: https://arxiv.org/abs/2509.02366

[6] S. Friedenthal, A. Moore, and R. Steiner, *A Practical Guide to SysML: The Systems Modeling Language*, 3rd ed. Boston, MA: Morgan Kaufmann, 2014.

[7] Canadian Space Agency, "CubeSats initiative in canada for STEM (CUBICS) 2022," Announcement of Opportunity, Sep. 2022. [Online]. Available: https://www.asc-csa.gc.ca/eng/funding-programs/funding-opportunities/ao/2022-cubics-cubesats-initiative-in-canada-for-stem.asp

[8] ——, "CubeSat initiative in canada for STEM (CUBICS) 2026," Announcement of Opportunity, Aug. 2026, updated 24 September 2026. [Online]. Available: https://www.asc-csa.gc.ca/eng/funding-programs/funding-opportunities/ao/2026-cubics-cubesats-initiative-in-canada-for-stem.asp

[9] ——, "Systems engineering mission tailoring guidelines," Canadian Space Agency, Guidelines CSA-SE-GDL-0001, Jul. 2020, initial release. [Online]. Available: https://sqreports.s3.ca-central-1.amazonaws.com/2021/CSA-SE-GDL-0001-IR-Systems-Engineering-Mission-Tailoring-Guidelines.pdf

[10] California Polytechnic State University, San Luis Obispo, "CubeSat design specification (1u–12u)," California Polytechnic State University, San Luis Obispo, Specification CP-CDS-R14.1, Feb. 2022, revision 14.1. [Online]. Available: https://www.cubesat.org/s/CDS-REV14_1-2022-02-09.pdf

[11] United Nations Office for Outer Space Affairs, "Space debris mitigation guidelines of the committee on the peaceful uses of outer space," United Nations, Vienna, United Nations publication ST/SPACE/49, 2010, guidelines adopted in 2007. [Online]. Available: https://www.unoosa.org/pdf/publications/st_space_49E.pdf

[12] Federal Communications Commission, "Satellite communications," 47 CFR, Part 25, 2026, electronic Code of Federal Regulations, current through 13 October 2026. Accessed 15 October 2026. [Online]. Available: https://www.ecfr.gov/current/title-47/part-25

[13] Innovation, Science and Economic Development Canada, "Decision on changes to licensing requirements and conditions of licence for space debris mitigation," Innovation, Science and Economic Development Canada, Tech. Rep. SMSE-009-26, Apr. 2026. [Online]. Available: https://ised-isde.canada.ca/site/spectrum-management-telecommunications/en/learn-more/key-documents/consultations/decision-changes-licensing-requirements-and-conditions-licence-space-debris-mitigation

[14] Exolaunch GmbH, *EXOpod Nova User Manual*, Exolaunch GmbH, Mar. 2024, revision 1.1. [Online]. Available: https://exolaunch.com/documents/EXOpod_Nova_User_Manual_March_2024.pdf

[15] Y. El Hachimi, A. Daghouri, H. Mahmoudi, A. Ouhammam, and S. El Hani, "A framework for sizing, designing, and testing of electrical power system for nanosatellites: A case study of a 3U university CubeSat," *e-Prime - Advances in Electrical Engineering, Electronics and Energy*, vol. 12, p. 101018, 2025. [Online]. Available: https://doi.org/10.1016/j.prime.2025.101018

[16] P. Boan, "Concurrent engineering process for electrical architecture of nanosatellites preliminary phases," in *Proceedings of SPIE the International Society for Optical Engineering, 13546, 135463U*, 2025.

[17] Object Management Group, "CubeSat system reference model profile (CSRM), version 1.1," Object Management Group, OMG Specification formal/24-08-03, Mar. 2025. [Online]. Available: https://www.omg.org/spec/CSRM/1.1

[18] S. C. Spangelo, D. Kaslow, C. Delp, B. Cole, L. Anderson, E. Fosse, B. S. Gilbert, L. Hartman, T. Kahn, and J. Cutler, "Applying model based systems engineering (MBSE) to a standard CubeSat," in *Proceedings of the 2012 IEEE Aerospace Conference*, 2012, pp. 1–20. [Online]. Available: https://doi.org/10.1109/AERO.2012.6187339

[19] S. C. Spangelo, J. Cutler, L. Anderson, E. Fosse, L. Cheng, R. Yntema, M. Bajaj, C. Delp, B. Cole, G. Soremekun, and D. Kaslow, "Model based systems engineering (MBSE) applied to radio aurora explorer (RAX) CubeSat mission operational scenarios," in *Proceedings of the 2013 IEEE Aerospace Conference*, 2013, pp. 1–18. [Online]. Available: https://doi.org/10.1109/AERO.2013.6496894

[20] L. Anderson, B. Cole, R. Yntema, M. Bajaj, S. Spangelo, D. Kaslow, C. Lowe, E. Sudano, M. Boghosian, R. Reil, S. Asundi, and S. Friedenthal, "Enterprise modeling for CubeSats," in *Proceedings of the 2014 IEEE Aerospace Conference*, 2014, pp. 1–16.

[21] D. Kaslow, G. Soremekun, H. Kim, and S. Spangelo, "Integrated model-based systems engineering (mbse) applied to the simulation of a cubesat mission," in *IEEE Aerospace Conference Proceedings, 6836317*, 2014.

[22] D. Kaslow, L. Anderson, S. Asundi, B. Ayres, C. Iwata, B. Shiotani, and R. Thompson, "Developing a CubeSat model-based system engineering (MBSE) reference model: Interim status," in *Proceedings of the 2015 IEEE Aerospace Conference*, 2015, pp. 1–16. [Online]. Available: https://doi.org/10.1109/AERO.2015.7118965

[23] D. Kaslow, B. J. Ayres, C. Iwata, and B. Shiotani, "CubeSat model based system engineering (MBSE) reference model: Application in the concept lifecycle phase," in *Proceedings of AIAA SPACE 2015*, 2015.

[24] D. Kaslow, L. Hart, B. Ayres, C. Massa, M. J. Chonoles, R. Yntema, S. D. Gasster, and B. Shiotani, "Developing a CubeSat model-based system engineering (MBSE) reference model: Interim status #2," in *Proceedings of the 2016 IEEE Aerospace Conference*, 2016, pp. 1–16. [Online]. Available: https://doi.org/10.1109/AERO.2016.7500592

[25] D. Kaslow, B. J. Ayres, P. T. Cahill, M. J. Chonoles, L. Hart, C. Iwata, A. G. Levi, and R. Yntema, "CubeSat model-based systems engineering (MBSE) reference model: Development and distribution, interim status," in *Proceedings of AIAA SPACE 2016*, 2016.

[26] D. Kaslow, B. Ayres, P. T. Cahill, L. Hart, and R. Yntema, "Developing a CubeSat model-based system engineering (MBSE) reference model: Interim status #3," in *Proceedings of the 2017 IEEE Aerospace Conference*, 2017, pp. 1–15.

[27] ——, "A model-based systems engineering (MBSE) approach for defining the behaviors of CubeSats," in *Proceedings of the 2017 IEEE Aerospace Conference*, 2017, pp. 1–14.

[28] D. Kaslow and A. M. Madni, "Validation and verification of MBSE-compliant CubeSat reference model," in *Disciplinary Convergence in Systems Engineering Research*. Cham: Springer International Publishing, 2018, pp. 381–393, presented at the 2017 Conference on Systems Engineering Research.

[29] D. Kaslow, B. Ayres, P. T. Cahill, and L. Hart, "A model-based systems engineering approach for technical measurement with application to a CubeSat," in *Proceedings of the 2018 IEEE Aerospace Conference*, 2018, pp. 1–10.

[30] D. Kaslow, B. J. Ayres, P. T. Cahill, C. Croney, L. Hart, and A. G. Levi, "Developing a CubeSat model-based system engineering (MBSE) reference model: Interim status #4," in *Proceedings of the 2018 AIAA SPACE and Astronautics Forum and Exposition*, 2018.

[31] D. Kaslow, P. T. Cahill, and R. Frank, "Developing a CubeSat system MBSE reference model: Interim status no. 5," in *Proceedings of the AIAA/USU Conference on Small Satellites*, 2019, pp. 1–15, paper SSC19-P2-16. [Online]. Available: https://digitalcommons.usu.edu/smallsat/2019/all2019/48/

[32] D. Kaslow, P. T. Cahill, and B. Ayres, "Development and application of the CubeSat system reference model," in *Proceedings of the 2020 IEEE Aerospace Conference*, 2020, pp. 1–15. [Online]. Available: https://doi.org/10.1109/AERO47225.2020.9172714

[33] D. Kaslow, A. Levi, P. T. Cahill, B. Ayres, D. Hurst, and C. Croney, "Mission engineering and the CubeSat system reference model," in *Proceedings of the 2021 IEEE Aerospace Conference*, 2021, pp. 1–8. [Online]. Available: https://doi.org/10.1109/AERO50100.2021.9438168

[34] D. Kaslow, A. Levi, B. Ayres, D. Hurst, C. Croney, P. T. Cahill, and S. A. MacLaird, "Mission engineering and the CubeSat system reference model: Status #2," in *Proceedings of the 36th Annual AIAA/USU Conference on Small Satellites*, 2022, pp. 1–12.

[35] S. Rudder, D. Kaslow, and J. Adams, "Evaluating metamodel quality of the CubeSat system reference model (CSRM) to improve domain-specific languages (DSL)," in *Proceedings of the 2026 IEEE Aerospace Conference*, 2026.

[36] G. Luccisano, S. Salas Cordero, T. Gateau, and N. Viola, "Open-source data formalization through model-based systems engineering for concurrent preliminary design of CubeSats," *Aerospace*, vol. 11, no. 9, p. 702, 2024. [Online]. Available: https://doi.org/10.3390/aerospace11090702

[37] J.-P. Ceglarek and R. Bertrand, "AI4CE: Building the infrastructure for AI-supported CubeSat generation," in *Proceedings of the 76th International Astronautical Congress*, 2025, paper IAC-25-D1,3,8,x95465.

[38] P. Casado, C. Torres, J. M. Blanes, A. Garrigós, and D. Marroquí, "Implementation of a 6U CubeSat electrical power system digital twin," *Aerospace*, vol. 11, no. 8, p. 688, 2024. [Online]. Available: https://doi.org/10.3390/aerospace11080688

[39] M. Wijaya, S. Lazreg, M. Cordy, A. Hein, and J. Bussemaker, "Multidisciplinary co-design optimization and reinforcement learning for CubeSat architecting," in

AIAA SCITECH 2026 Forum, 2026. [Online]. Available: https://doi.org/10.2514/6.2026-1609

[40] C. Strobbe, S. Salas Cordero, and R. Vingerhoeds, "Open-source CubeSat MBSE approach to address traceability: Power subsystem," in *Proceedings of the 17th Annual IEEE International Systems Conference (SYSCON)*, 2023. [Online]. Available: https://doi.org/10.1109/SYSCON53073.2023.10131054

[41] A. Jbara, "Applying model-based systems engineering on CubeSat budget calculating," in *2025 IEEE Aerospace Conference*, 2025. [Online]. Available: https://doi.org/10.1109/AERO63441.2025.11068482

[42] M. Wijaya, S. Lazreg, M. Cordy, A. Hein, and J. Bussemaker, "Automated variability evaluation using a model-based approach with an implementation in cubesat," *The Journal of Systems and Software, 241, 112965*, 2026.

[43] M. Halvorson and L. D. Thomas, "Architecture framework standardization for satellite software generation using MBSE and F Prime," in *Proceedings of the 2022 IEEE Aerospace Conference (AERO)*, 2022, pp. 1–20. [Online]. Available: https://doi.org/10.1109/AERO53065.2022.9843358

[44] M. Eshaq, M. S. Zitouni, S. Atalla, S. Al-Mansoori, and M. Macdonald, "CubeSat flight software: Insights and a case study," *Journal of Spacecraft and Rockets*, vol. 62, no. 4, pp. 1328–1345, 2025. [Online]. Available: https://doi.org/10.2514/1.A35882

[45] K. Djebko, F. Puppe, and H. Kayal, "Model-based fault detection and diagnosis for spacecraft with an application for the SONATE triple cube nano-satellite," *Aerospace*, vol. 6, no. 10, p. 105, 2019. [Online]. Available: https://doi.org/10.3390/aerospace6100105

[46] H. R. Shrestha, "Power budget for 1U satellite: BIRDS, EPS," LaSEINE, Kyushu Institute of Technology, presentation, 2022, BIRDS-4 component, generation and consumption budgets, pp. 5, 8–9, 13, 15. [Online]. Available: https://birds-project.com/open-source/pdf/Power-Budget-Analysis-for-1U-satellit20220514.pdf

[47] OreSat Project, "OreSat power budget," Mission power budget, Sep. 2019, 30 September 2019. Generation, storage and operating mode budgets, pp. 1–2. [Online]. Available: https://github.com/oresat/oresat.github.io/blob/0d879a503be4a6d48e849f3088524c2748503640/pub/OreSat_Power_Budget_190930.pdf

[48] H. R. Shrestha, M. A. C. Purio, I. Z. C. Bautista, A. J. Jara Cespedes, T. Yamauchi, S. Kim, and M. Cho, "Verification of the 1U standard CubeSat electrical power system (EPS) for the international space station (ISS) safety requirements," in *Proceedings of the 72nd International Astronautical Congress*, Oct. 2021, paper IAC-21-C3.3.1. Maya-2 solar curves, Figs. 13–14. BCR overvoltage test, Sec. 3.6. [Online]. Available: https://iafastro.directory/iac/paper/id/62691/summary/

[49] Portland State Aerospace Society, "OreSat: Oregon's first nanosatellite," Portland State University, Tech. Rep., Nov. 2016, NASA CubeSat Launch Initiative application, 22 November 2016. Structure and mass budget, Sec. D.2, pp. 21–22. [Online]. Available: https://github.com/oresat/oresat.github.io/blob/0d879a503be4a6d48e849f3088524c2748503640/pub/OreSat_CSLI_Application-2016.pdf

[50] H. R. Shrestha, J. R. Cordova-Alarcon, N. C. Örger, V. H. Schulz, F. Moumni, T. Yamauchi, and M. Cho, "Improving the reliability of a CubeSat power system by combining small-sized monocrystalline cells with triple-junction solar cells," *IEEE Access*, vol. 14, pp. 25 038–25 054, Feb. 2026. [Online]. Available: https://doi.org/10.1109/ACCESS.2026.3664750

[51] OreSat Project, "OreSat solar module design notes," Public mission hardware documentation, Feb. 2025, stated update: 2 February 2025, with later dated additions. GaAs and silicon module calculations, converter losses and ground testing. [Online]. Available: https://docs.google.com/document/d/1mogZjYhJievwLsnVkg4Q0sEF9xPMA5q7KPciky3YWKE/edit

[52] ——, "Battery pack and cell requirements and research," Public mission design study, n.d., battery chemistry, bus voltage, packaging and capacity trades. Linked from the OreSat Battery Card Design Notes. [Online]. Available: https://docs.google.com/document/d/1tGRrhCfbk0fJJVCt0csk0iamH-1ZTGMi4msohmOQFPg/edit

[53] A. Greenberg, D. Lay, G. Lebrasseur, E. Levy, R. Medick, H. Reinhold, R. Rushford, and M. Simpson, "OreSat: A student team-based approach to an inexpensive, open, and modular (1–3U) CubeSat bus," in *Proceedings of the 35th Annual Small Satellite Conference*, Aug. 2021, paper SSC21-WKI-09. Battery configuration and capacity, p. 6. [Online]. Available: https://digitalcommons.usu.edu/smallsat/2021/all2021/236/

[54] OreSat Project, "OreSat solar modules," Public solar hardware documentation, revision v5.3, n.d., design revisions 1–4. SPV1040 voltage incompatibility after the change from a 1S to a 2S battery pack. [Online]. Available: https://github.com/oresat/oresat-solar-hardware/blob/47d610c1964c31f049a8f9844f7881931b5d0f06/README.md

[55] ——, "OreSat solar module ground test data," Public spreadsheet linked from the OreSat Solar Module Design Notes, n.d., worksheet minicom.cap, rows 2–206, columns F–H. Calibrated current, voltage and calculated power. [Online]. Available: https://docs.google.com/spreadsheets/d/1vNrHLC0C6YMA_wZTWuHt7_7BPwSQPb8aQuyxLQmDAQ8/edit

[56] Kyushu Institute of Technology, "BIRDS platform interface control document," Laboratory of Lean Satellite Enterprises and In-Orbit Experiments, Tech. Rep., Mar. 2021, revision F, 3 March 2021. Regulated lines and protection thresholds, Sec. 6.3. [Online]. Available: https://github.com/BIRDSOpenSource/

BIRDS-GeneralDocumentation/blob/76cf923be9dc7e923e7397b5426c5b1e3d3f41c6/BIRDS_Platform_ICD_ENGLISH_RevF_0303.pdf

[57] OreSat Project, "OreSat battery card design notes," Public mission hardware documentation, Mar. 2025, stated update: 4 March 2025. Cell and protection IC selection, current calculations and protection state logic. [Online]. Available: https://docs.google.com/document/d/1yaPGtJSM5RQG5k7sVpsDxBkVu4CHtm4LMNnisgOaNRg/edit

[58] Altium, "Requirements Portal: Engineering assistant," updated 26 October 2026. Accessed 12 November 2026. [Online]. Available: https://www.altium.com/documentation/altium-365/requirements-portal/agentic-requirements-engineering/engineering-assistant

[59] ——, "Requirements Portal: Using and managing AI Skills," updated 3 November 2026. Accessed 12 November 2026. [Online]. Available: https://www.altium.com/documentation/altium-365/requirements-portal/agentic-requirements-engineering/ai-skills

[60] ——, "Requirements Portal: REST and Python API documentation," accessed 9 October 2026. [Online]. Available: https://www.altium.com/documentation/altium-365/requirements-portal/rest-python-api-documentation

[61] Dassault Systèmes, "CATIA STIMULUS," accessed 9 October 2026. [Online]. Available: https://www.3ds.com/products/catia/stimulus

[62] ——, "CATIA STIMULUS for education," accessed 9 October 2026. [Online]. Available: https://www.3ds.com/edu/education/academics/solutions/catia-stimulus

[63] T. Gateau, S. Salas Cordero, and R. Vingerhoeds, "Hierarchical planning applied to the preliminary design of CubeSats: Nanospace study," *Systems Engineering*, vol. 28, no. 3, pp. 438–452, 2025.

[64] M. Deutsch, J. Hoffmann, and R. Bertrand, "Development of a concurrent engineering tutorial as part of the ESA_Lab@ initiative," in *Proceedings of the 73rd International Astronautical Congress*, 2022, paper IAC-22-E1.3.8.x72945.

[65] Altium, "Requirements Portal documentation," accessed 9 October 2026. [Online]. Available: https://www.altium.com/documentation/altium-365/requirements-portal

[66] Dassault Systèmes, "FMI (Functional Mock-up Interface): Capability table," accessed 9 October 2026. [Online]. Available: https://3dswym.3dexperience.3ds.com/wiki/catia-mbse-cyber-systems/fmi-functional-mock-up-interface_sFWjG3x4R9qmSTT0o3erTQ

[67] K. Y.-T. Chiu, "SeaLion CubeSat mission architecture using model based systems engineering with a docs as code approach," Master's thesis, Old Dominion University, 2023. [Online]. Available: https://digitalcommons.odu.edu/mae_etds/132/

[68] INCOSE Space Systems Working Group and OMG Space Domain Task Force, "CubeSat system reference model," SysML reference model, 2025, model snapshot dated 18 November 2025. Copyright 2022 INCOSE Space Systems Working Group and OMG Space Domain Task Force. CC BY 4.0. [Online]. Available: https://github.com/ObjectManagementGroup/CSRM/tree/69c7f021fd1b2b0e9135dc9e6f926c12e514de7e

[69] Object Management Group, "OMG Systems Modeling Language (SysML), version 2.0, part 1: Language specification," Object Management Group, Tech. Rep. formal/2026-03-02, Mar. 2026. [Online]. Available: https://www.omg.org/spec/SysML/2.0/

[70] A. Freimann, J. Scharnagl, T. Petermann, H. Döbler, and K. Schilling, "CubeSat energy modelling for improved mission planning and operations," in *Proceedings of the 70th International Astronautical Congress*, 2019, paper IAC-19-B6.IP.7.

[71] M. Amato, "Pre-phase A design through MBSE of CUBE mission," Master's thesis, Politecnico di Milano, 2023.

[72] I. Qureshi, "ESSENCE electrical power system modelling & simulation," Ryerson University, Department of Aerospace Engineering, AER 870 Aerospace Engineering Thesis, Apr. 2020, final report dated 11 April 2020. [Online]. Available: https://hdl.handle.net/10.32920/ryerson.14663187.v1

[73] S. K. Nogrady, "Development of a CubeSat conceptual design tool and implementation of the EPS design module," Master's thesis, California Polytechnic State University, San Luis Obispo, Jun. 2021.

[74] M. Vasile and S. Alicino, "An approach for the robust design of the power systems of small satellites," in *Proceedings of the 65th International Astronautical Congress*, Toronto, Canada, 2014, paper IAC-14.C3.4.2. [Online]. Available: https://dl.iafastro.directory/event/IAC-2014/paper/25543/

[75] BIRDS-4 Project, "Battery (cells) screening procedures and its verification report," Kyushu Institute of Technology, Tech. Rep., 2021, combined report containing September 2020 BIRDS-4 tests and screening instructions updated in February 2021. Cell specification, p. 21. Pack measurements and converter tests, pp. 41–42. [Online]. Available: https://github.com/BIRDSOpenSource/BIRDS4-ProceduresAndReports/blob/d7eadd98f5327e1ddea82b640293d49bfdb09555/Battery%28Cells%29%20Screening%20Procedures%20and%20its%20verification%20Report_V1.0%20%282%29.pdf

[76] K. T. Ulrich and S. D. Eppinger, *Diseño y desarrollo de productos*, 5th ed. Mexico City, Mexico: McGraw-Hill/Interamericana Editores, 2013.

[77] Treasury Board of Canada Secretariat, "Canada's cost-benefit analysis guide for regulatory proposals," Treasury Board of Canada Secretariat, Tech. Rep., Jan. 2019, updated January 2025. Accessed 13 November 2026. [Online]. Available: https://wiki.gccollab.ca/images/8/8e/CBA_Guide-EN.doc

[78] Molicel, "Product data sheet of INR-18650-M30A," Manufacturer datasheet, version 2. Physical characteristics, p. 1. Accessed 30 October 2026. [Online]. Available: https://www.molicel.com/wp-content/uploads/INR18650M30A-V2-80088.pdf

[79] *INR-18650-M35A Datasheet*, Manufacturer datasheet, Molicel, revision 1.4. Accessed 30 October 2026. [Online]. Available: https://www.molicel.com/wp-content/uploads/INR18650M35A_1.4_Product-Data-Sheet-of-INR-18650-M35A-80096.pdf

[80] *INR-18650-P26A Datasheet*, Manufacturer datasheet, Molicel, revision 1.4. Accessed 30 October 2026. [Online]. Available: https://www.molicel.com/wp-content/uploads/INR18650P26A_1.4_Product-Data-Sheet-of-INR-18650-P26A-80087.pdf

[81] *INR-18650-P28A Datasheet*, Manufacturer datasheet, Molicel, revision 1.3. Accessed 30 October 2026. [Online]. Available: https://www.molicel.com/wp-content/uploads/INR18650P28A_1.3_Product-Data-Sheet-of-INR-18650-P28A-80093.pdf

[82] *INR-18650-P28B Datasheet*, Manufacturer datasheet, Molicel, revision V1. Accessed 30 October 2026. [Online]. Available: https://www.molicel.com/wp-content/uploads/INR18650P28B-V1-80104.pdf

[83] *INR-18650-P30B Datasheet*, Manufacturer datasheet, Molicel, revision 1.0. Accessed 30 October 2026. [Online]. Available: https://www.molicel.com/wp-content/uploads/Product-Data-Sheet-of-INR-18650-P30B-80111-1.pdf

[84] *INR-21700-M65A Datasheet*, Manufacturer datasheet, Molicel, revision 0.1. Accessed 30 October 2026. [Online]. Available: https://www.molicel.com/wp-content/uploads/Product-Data-Sheet-of-INR-21700-M65A-80153-0.1_250805.pdf

[85] *INR-21700-P42A Datasheet*, Manufacturer datasheet, Molicel, revision 1.7. Accessed 30 October 2026. [Online]. Available: https://www.molicel.com/wp-content/uploads/INR21700P42A_1.7_Product-Data-Sheet-of-INR-21700-P42A-80092.pdf

[86] *INR-21700-P45B Datasheet*, Manufacturer datasheet, Molicel, revision 1.4. Accessed 30 October 2026. [Online]. Available: https://www.molicel.com/wp-content/uploads/INR21700P45B_1.4_Product-Data-Sheet-of-INR-21700-P45B-80109.pdf

[87] *INR-21700-P50B Datasheet*, Manufacturer datasheet, Molicel, revision 1.1. Accessed 30 October 2026. [Online]. Available: https://www.molicel.com/wp-content/uploads/4.TR%E7%B0%A1%E6%98%93%E8%A6%8F%E6%A0%BC_INR21700P50B_1.1_Product-Data-Sheet-of-INR-21700-P50B-80122.pdf

[88] *BK-3MCC (eneloop) Datasheet*, Manufacturer datasheet, Panasonic, accessed 11 November 2026. [Online]. Available: https://www.optimusdigital.ro/ro/index.php?controller=attachment&id_attachment=3561

[89] *30005-0 (ICR18650-2600, LR1865SK) Datasheet*, Manufacturer datasheet, Tenergy, accessed 11 November 2026. [Online]. Available: https://files.batteryjunction.com/frontend/files/tenergy/datasheet/TENERGY-30005-18650-FLAT-Datasheet.pdf

[90] *NanoPower BPX 100 Wh (8S1P) Datasheet*, Manufacturer datasheet, GomSpace, revision 1.1.0. Accessed 30 October 2026. [Online]. Available: https://gomspace.com/wp-content/uploads/2025/09/gs-ds-nanopower-BPX-100-Wh-1076870-1.1.0.pdf

[91] *81441 Datasheet*, Manufacturer datasheet, AZUR SPACE Solar Power GmbH, revision DB 00010891-01. Accessed 30 October 2026. [Online]. Available: https://www.azurspace.com/media/uploads/file_links/file/bdb_00010891-01-00_tj3g30-advanced_4x8.pdf

[92] *82921 Datasheet*, Manufacturer datasheet, AZUR SPACE Solar Power GmbH, revision DB 00010895-02. Accessed 30 October 2026. [Online]. Available: https://www.azurspace.com/media/uploads/file_links/file/bdb_00010895-02-00_4g32-advanced_4x8.pdf

[93] *CTJ30 Datasheet*, Manufacturer datasheet, CESI S.p.A., revision v2025.1. Accessed 30 October 2026. [Online]. Available: https://www.cesi.it/app/uploads/2025/04/CESI-CTJ30-v2025.1.pdf

[94] *CTJLC Datasheet*, Manufacturer datasheet, CESI S.p.A., revision v2025.1. Accessed 30 October 2026. [Online]. Available: https://www.cesi.it/app/uploads/2025/04/CESI-CTJ-LC-v2025.1.pdf

[95] *IMM3J Solar Cells Datasheet*, Manufacturer datasheet, Sharp Energy Solutions Corporation, revision 10.11.2023. Accessed 30 October 2026. [Online]. Available: https://global.sharp/solar/en/space-qualified/pdf/datasheet_IMM3J.pdf

[96] *XTE+ GEO Datasheet*, Manufacturer datasheet, Spectrolab Inc., accessed 30 October 2026. [Online]. Available: https://www.spectrolab.com/photovoltaics/XTE%2B%20GEO%20Data%20Sheet.pdf

[97] *XTE+ LEO Datasheet*, Manufacturer datasheet, Spectrolab Inc., accessed 30 October 2026. [Online]. Available: https://www.spectrolab.com/photovoltaics/XTE%2B%20LEO%20Data%20Sheet.pdf

[98] *XTE-HF Datasheet*, Manufacturer datasheet, Spectrolab Inc., accessed 30 October 2026. [Online]. Available: https://www.spectrolab.com/photovoltaics/XTE-HF%20Data%20Sheet%20_12.23.19.pdf

[99] *XTE-LILT Datasheet*, Manufacturer datasheet, Spectrolab Inc., accessed 30 October 2026. [Online]. Available: https://www.spectrolab.com/photovoltaics/XTE-LILT%20Data%20Sheet%20_12.23.19.pdf

[100] *XTE-SF Datasheet*, Manufacturer datasheet, Spectrolab Inc., revision 2022-07-27. Accessed 30 October 2026. [Online]. Available: https://www.spectrolab.com/photovoltaics/XTE-SF%20Data%20Sheet%20_07.27.22.pdf

[101] *XTJ Prime Datasheet*, Manufacturer datasheet, Spectrolab Inc., revision 2022-07-27. Accessed 30 October 2026. [Online]. Available: https://www.spectrolab.com/photovoltaics/XTJ-Prime_Data_Sheet%20_07.27.22.pdf

[102] *3G30A (TJ Solar Cell Assembly) Datasheet*, Manufacturer datasheet, AZUR SPACE Solar Power GmbH, revision DB 0003401-01-01. Accessed 11 November 2026. [Online]. Available: https://d35rpq4gusjz9h.cloudfront.net/files/73192/download/634236/73192_19_20230906054714917796.pdf

[103] *LT1618EMS#TRPBF Datasheet*, Manufacturer datasheet, Analog Devices, revision 1618fa. Accessed 11 November 2026. [Online]. Available: https://www.analog.com/media/en/technical-documentation/data-sheets/1618fas.pdf

[104] *LTC3119EUFD#PBF Datasheet*, Manufacturer datasheet, Analog Devices, revision B. Accessed 11 November 2026. [Online]. Available: https://www.analog.com/media/en/technical-documentation/data-sheets/3119fb.pdf

[105] *LTC3530EDD#PBF Datasheet*, Manufacturer datasheet, Analog Devices, revision 3530fb. Accessed 11 November 2026. [Online]. Available: https://www.analog.com/media/en/technical-documentation/data-sheets/3530fb.pdf

[106] *LMR33630ADDAR Datasheet*, Manufacturer datasheet, Texas Instruments, revision SNVSAN3F. Accessed 30 October 2026. [Online]. Available: https://www.ti.com/lit/ds/symlink/lmr33630.pdf

[107] *TPS61022RWUR Datasheet*, Manufacturer datasheet, Texas Instruments, revision SLVSDX7D. Accessed 30 October 2026. [Online]. Available: https://www.ti.com/lit/ds/symlink/tps61022.pdf

[108] *TPS61023DRLR Datasheet*, Manufacturer datasheet, Texas Instruments, revision SLVSF14B. Accessed 30 October 2026. [Online]. Available: https://www.ti.com/lit/ds/symlink/tps61023.pdf

[109] *TPS61235PRWLT Datasheet*, Manufacturer datasheet, Texas Instruments, revision SLVSCK4A. Accessed 11 November 2026. [Online]. Available: https://www.ti.com/lit/ds/symlink/tps61235p.pdf

[110] *TPS62088YFPR Datasheet*, Manufacturer datasheet, Texas Instruments, revision SLVSD94F. Accessed 30 October 2026. [Online]. Available: https://www.ti.com/lit/ds/symlink/tps62088.pdf

[111] *TPS62111RSAT, TPS62112RSAT Datasheet*, Manufacturer datasheet, Texas Instruments, revision SLVS585E. Accessed 11 November 2026. [Online]. Available: https://www.ti.com/lit/ds/symlink/tps62110.pdf

[112] *TPS62130ARGTR Datasheet*, Manufacturer datasheet, Texas Instruments, revision SLVSAG7F. Accessed 30 October 2026. [Online]. Available: https://www.ti.com/lit/ds/symlink/tps62130a.pdf

[113] *TPS62160DSGR Datasheet*, Manufacturer datasheet, Texas Instruments, revision SLVSAM2E. Accessed 30 October 2026. [Online]. Available: https://www.ti.com/lit/ds/symlink/tps62160.pdf

[114] *TPS62933DRLR Datasheet*, Manufacturer datasheet, Texas Instruments, revision SLUSEA4D. Accessed 30 October 2026. [Online]. Available: https://www.ti.com/lit/ds/symlink/tps62933.pdf

[115] *TPS63020DSJR Datasheet*, Manufacturer datasheet, Texas Instruments, revision SLVS916I. Accessed 30 October 2026. [Online]. Available: https://www.ti.com/lit/ds/symlink/tps63020.pdf

[116] *TPS63020QDSJTQ1 Datasheet*, Manufacturer datasheet, Texas Instruments, revision SLVSD52A. Accessed 11 November 2026. [Online]. Available: https://www.ti.com/lit/ds/symlink/tps63020-q1.pdf

[117] *TPS63070RNMR Datasheet*, Manufacturer datasheet, Texas Instruments, revision SLVSC58B. Accessed 30 October 2026. [Online]. Available: https://www.ti.com/lit/ds/symlink/tps63070.pdf

[118] *TPS63802DLAR Datasheet*, Manufacturer datasheet, Texas Instruments, revision SLVSEU9D. Accessed 10 November 2026. [Online]. Available: https://www.ti.com/lit/ds/symlink/tps63802.pdf

[119] *LP2985AIM5-1.8/NOPB Datasheet*, Manufacturer datasheet, Texas Instruments, revision SNOS510Q. Accessed 11 November 2026. [Online]. Available: https://www.ti.com/lit/ds/symlink/lp2985lv-n.pdf

[120] *LP2985AIM5-2.5/NOPB, LP2985AIM5-3.3/NOPB Datasheet*, Manufacturer datasheet, Texas Instruments, revision SNVS018AB. Accessed 11 November 2026. [Online]. Available: https://www.ti.com/lit/ds/symlink/lp2985-n.pdf

[121] *ICEPS2 Type A Datasheet*, Manufacturer datasheet, ISISPACE, revision 1.2. Accessed 30 October 2026. [Online]. Available: https://isispace.nl/wp-content/uploads/2025/01/ISIS-ICEPS2-DSH-0001-ICEPS2_Datasheet-01_02.pdf

[122] *IMEPS2 (1 PCU, 1 PBU, 1 PDU, 1 PBP-4S1P) Datasheet*, Manufacturer datasheet, ISISPACE, revision 2.0. Accessed 10 November 2026. [Online]. Available: https://www.isispace.nl/wp-content/uploads/2025/01/ISIS-IMEPS2-DSH-0001-Datasheet-02_00.pdf

[123] *EPSM1 Rev C Datasheet*, Manufacturer datasheet, Pumpkin, revision I. Accessed 30 October 2026. [Online]. Available: https://www.pumpkininc.com/space/module/epsm/710-01952C_DS_EPSM1.pdf

[124] *BQ24074RGTR Datasheet*, Manufacturer datasheet, Texas Instruments, revision SLUS810N. Accessed 30 October 2026. [Online]. Available: https://www.ti.com/lit/ds/symlink/bq24074.pdf

[125] *BQ24610RGER Datasheet*, Manufacturer datasheet, Texas Instruments, revision SLUS892D. Accessed 30 October 2026. [Online]. Available: https://www.ti.com/lit/ds/symlink/bq24610.pdf

[126] *BQ24650RVAR Datasheet*, Manufacturer datasheet, Texas Instruments, revision SLUSA75B. Accessed 10 November 2026. [Online]. Available: https://www.ti.com/lit/ds/symlink/bq24650.pdf

[127] *BQ25606RGER Datasheet*, Manufacturer datasheet, Texas Instruments, revision SLUSCK6C. Accessed 30 October 2026. [Online]. Available: https://www.ti.com/lit/ds/symlink/bq25606.pdf

[128] *BQ25798RQMR Datasheet*, Manufacturer datasheet, Texas Instruments, revision SLUSDV2C. Accessed 10 November 2026. [Online]. Available: https://www.ti.com/lit/ds/symlink/bq25798.pdf

[129] *S-8209AAA-T8T1U Datasheet*, Manufacturer datasheet, ABLIC, revision 3.8 00. Accessed 11 November 2026. [Online]. Available: https://raw.githubusercontent.com/oresat/oresat-batteries/78c092388955d8bcce095ed5f090b1c0dfa9226c/docs/Datasheets/ABLIC-S8209A_E-BattProtection.pdf

[130] *S-8252AAG-M6T1U Datasheet*, Manufacturer datasheet, ABLIC, revision 4.0 00. Accessed 30 October 2026. [Online]. Available: https://www.ablic.com/en/doc/datasheet/battery_protection/S8252_E.pdf

[131] *BQ29700DSER Datasheet*, Manufacturer datasheet, Texas Instruments, revision SLUSBU9I. Accessed 30 October 2026. [Online]. Available: https://www.ti.com/lit/ds/symlink/bq2970.pdf

[132] *BQ298000RUGR Datasheet*, Manufacturer datasheet, Texas Instruments, revision SLUSCS3L. Accessed 30 October 2026. [Online]. Available: https://www.ti.com/lit/ds/symlink/bq2980.pdf

[133] *BQ7790500PWR Datasheet*, Manufacturer datasheet, Texas Instruments, revision SLUSCM3K. Accessed 30 October 2026. [Online]. Available: https://www.ti.com/lit/ds/symlink/bq77904.pdf

[134] *BQ7791500PWR Datasheet*, Manufacturer datasheet, Texas Instruments, revision SLUSCU0L. Accessed 30 October 2026. [Online]. Available: https://www.ti.com/lit/ds/symlink/bq77915.pdf

[135] *LTC4359IMS8#PBF Datasheet*, Manufacturer datasheet, Analog Devices, revision F. Accessed 30 October 2026. [Online]. Available: https://www.analog.com/media/en/technical-documentation/data-sheets/ltc4359.pdf

[136] *LTC4412ES6#TRPBF, LTC4412IS6#TRPBF Datasheet*, Manufacturer datasheet, Analog Devices, revision C. Accessed 30 October 2026. [Online]. Available: https://www.analog.com/media/en/technical-documentation/data-sheets/ltc4412.pdf

[137] *LM5050MK-1/NOPB Datasheet*, Manufacturer datasheet, Texas Instruments, revision SNVS629F. Accessed 30 October 2026. [Online]. Available: https://www.ti.com/lit/ds/symlink/lm5050-1.pdf

[138] *LM66100DCKR Datasheet*, Manufacturer datasheet, Texas Instruments, revision SLVSEZ8A. Accessed 30 October 2026. [Online]. Available: https://www.ti.com/lit/ds/symlink/lm66100.pdf

[139] *LM66200DRLR Datasheet*, Manufacturer datasheet, Texas Instruments, revision SLVSG04. Accessed 30 October 2026. [Online]. Available: https://www.ti.com/lit/ds/symlink/lm66200.pdf

[140] *LM74700QDBVRQ1 Datasheet*, Manufacturer datasheet, Texas Instruments, revision SNOSD17G. Accessed 30 October 2026. [Online]. Available: https://www.ti.com/lit/ds/symlink/lm74700-q1.pdf

[141] *G6KU-2F-Y DC3 Datasheet*, Manufacturer datasheet, Omron, revision K106-E1-11. Accessed 11 November 2026. [Online]. Available: https://omronfs.omron.com/en_US/ecb/products/pdf/en-g6k.pdf

[142] *LTC4361IDC-2#TRMPBF Datasheet*, Manufacturer datasheet, Analog Devices, revision C. Accessed 11 November 2026. [Online]. Available: https://www.analog.com/media/en/technical-documentation/data-sheets/ltc4361-1-4361-2.pdf

[143] *LTC4368IMS-2#PBF Datasheet*, Manufacturer datasheet, Analog Devices, revision C. Accessed 30 October 2026. [Online]. Available: https://www.analog.com/media/en/technical-documentation/data-sheets/ltc4368.pdf

[144] *MAX892LEUA+T Datasheet*, Manufacturer datasheet, Analog Devices, document 19-1223, revision 0. Accessed 11 November 2026. [Online]. Available: https://www.analog.com/media/en/technical-documentation/data-sheets/MAX891L-MAX892L.pdf

[145] *TPS22919DCKR Datasheet*, Manufacturer datasheet, Texas Instruments, revision SLVSEN5B. Accessed 30 October 2026. [Online]. Available: https://www.ti.com/lit/ds/symlink/tps22919.pdf

[146] *TPS22948DCKR.A Datasheet*, Manufacturer datasheet, Texas Instruments, revision SLVSEZ7A. Accessed 30 October 2026. [Online]. Available: https://www.ti.com/lit/ds/symlink/tps22948.pdf

[147] *TPS25200DRVR Datasheet*, Manufacturer datasheet, Texas Instruments, revision SLVSCJ0F. Accessed 30 October 2026. [Online]. Available: https://www.ti.com/lit/ds/symlink/tps25200.pdf

[148] *TPS2553DBVR Datasheet*, Manufacturer datasheet, Texas Instruments, revision SLVS841G. Accessed 30 October 2026. [Online]. Available: https://www.ti.com/lit/ds/symlink/tps2553.pdf

[149] *TPS2556DRBR Datasheet*, Manufacturer datasheet, Texas Instruments, revision SLVS931B. Accessed 30 October 2026. [Online]. Available: https://www.ti.com/lit/ds/symlink/tps2556.pdf

[150] *TPS25921ADR Datasheet*, Manufacturer datasheet, Texas Instruments, revision SLVSCE1C. Accessed 30 October 2026. [Online]. Available: https://www.ti.com/lit/ds/symlink/tps25921a.pdf

[151] *TPS25940ARVCR Datasheet*, Manufacturer datasheet, Texas Instruments, revision SLVSCF3A. Accessed 30 October 2026. [Online]. Available: https://www.ti.com/lit/ds/symlink/tps25940.pdf

[152] *TPS259531DSGR Datasheet*, Manufacturer datasheet, Texas Instruments, revision SLVSE57C. Accessed 30 October 2026. [Online]. Available: https://www.ti.com/lit/ds/symlink/tps2595.pdf

[153] *TPS259620DDAT, TPS259621DDAT Datasheet*, Manufacturer datasheet, Texas Instruments, revision SLVSET8A. Accessed 11 November 2026. [Online]. Available: https://www.ti.com/lit/ds/symlink/tps2596.pdf

[154] *TPS259824ONRGER Datasheet*, Manufacturer datasheet, Texas Instruments, revision SLVSEI3D. Accessed 30 October 2026. [Online]. Available: https://www.ti.com/lit/ds/symlink/tps25982.pdf

[155] *TPS26600PWPR Datasheet*, Manufacturer datasheet, Texas Instruments, revision SLVSDG2G. Accessed 30 October 2026. [Online]. Available: https://www.ti.com/lit/ds/symlink/tps2660.pdf

[156] *TPS26630RGER Datasheet*, Manufacturer datasheet, Texas Instruments, revision SLVSE94G. Accessed 30 October 2026. [Online]. Available: https://www.ti.com/lit/ds/symlink/tps2663.pdf

[157] *DMP2240UDM-7 Datasheet*, Manufacturer datasheet, Diodes Incorporated, revision DS31197 Rev. 5-2. Accessed 11 November 2026. [Online]. Available: https://www.diodes.com/datasheet/download/DMP2240UDM.pdf

[158] *DMP3050LVT-7 Datasheet*, Manufacturer datasheet, Diodes Incorporated, revision DS35748 Rev. 4 - 2. Accessed 11 November 2026. [Online]. Available: https://www.diodes.com/assets/Datasheets/DMP3050LVT.pdf

[159] *1206L150/12SLYR, 1206L500/12SLPR Datasheet*, Manufacturer datasheet, Littelfuse, revision GD.03/06/25. Accessed 11 November 2026. [Online]. Available: https://www.littelfuse.com/assetdocs/resettable-ptcs-low-rho-smd-ptc-datasheet

[160] *PMV45EN2R Datasheet*, Manufacturer datasheet, Nexperia, revision PMV45EN2 v.2. Accessed 11 November 2026. [Online]. Available: https://assets.nexperia.com/documents/data-sheet/PMV45EN2.pdf

[161] *NTR4101PT1G Datasheet*, Manufacturer datasheet, onsemi, revision NTR4101P/D. Accessed 11 November 2026. [Online]. Available: https://www.onsemi.com/pdf/datasheet/ntr4101p-d.pdf

[162] *SSM6K411TU Datasheet*, Manufacturer datasheet, Toshiba, revision 2014-03-01. Accessed 11 November 2026. [Online]. Available: https://www.mouser.com/datasheet/2/408/SSM6K411TU_datasheet_en_20140301-708726.pdf

[163] *SiA447DJ-T1-GE3 Datasheet*, Manufacturer datasheet, Vishay, revision 63774. Accessed 11 November 2026. [Online]. Available: https://www.vishay.com/docs/63774/sia447dj.pdf

[164] *Si7106DN Datasheet*, Manufacturer datasheet, Vishay Siliconix, revision 73142. Accessed 11 November 2026. [Online]. Available: https://www.vishay.com/docs/73142/si7106dn.pdf

[165] *Si7232DN-T1-GE3 Datasheet*, Manufacturer datasheet, Vishay Siliconix, revision S15-0931-Rev.C. Accessed 11 November 2026. [Online]. Available: https://www.vishay.com/docs/68986/si7232dn.pdf

[166] *Si7625DN-T1-GE3 Datasheet*, Manufacturer datasheet, Vishay Siliconix, revision 65737. Accessed 11 November 2026. [Online]. Available: https://www.vishay.com/docs/65737/si7625dn.pdf

[167] *ACS722LLCTR-05AB-T, ACS722LLCTR-10AU Datasheet*, Manufacturer datasheet, Allegro MicroSystems, revision ACS722-DS Rev. 8, March 12 2025, MCO-0000537. Accessed 11 November 2026. [Online]. Available: https://www.allegromicro.com/-/media/files/datasheets/acs722-datasheet.pdf

[168] *LT6106HS5#TRMPBF Datasheet*, Manufacturer datasheet, Analog Devices, revision D. Accessed 11 November 2026. [Online]. Available: https://www.analog.com/media/en/technical-documentation/data-sheets/lt6106.pdf

[169] *MAX4211FETE Datasheet*, Manufacturer datasheet, Analog Devices, document 19-3285, revision 1. Accessed 11 November 2026. [Online]. Available: https://www.analog.com/media/en/technical-documentation/data-sheets/max4210-max4211.pdf

[170] *INA180A1IDBVR Datasheet*, Manufacturer datasheet, Texas Instruments, revision SBOS741H. Accessed 30 October 2026. [Online]. Available: https://www.ti.com/lit/ds/symlink/ina180.pdf

[171] *INA190A1IDDFR Datasheet*, Manufacturer datasheet, Texas Instruments, revision SBOS863D. Accessed 30 October 2026. [Online]. Available: https://www.ti.com/lit/ds/symlink/ina190.pdf

[172] *INA219BIDCNR Datasheet*, Manufacturer datasheet, Texas Instruments, revision SBOS448G. Accessed 30 October 2026. [Online]. Available: https://www.ti.com/lit/ds/symlink/ina219.pdf

[173] *INA226AIDGSR Datasheet*, Manufacturer datasheet, Texas Instruments, revision SBOS547C. Accessed 30 October 2026. [Online]. Available: https://www.ti.com/lit/ds/symlink/ina226.pdf

[174] *INA228AIDGSR Datasheet*, Manufacturer datasheet, Texas Instruments, revision SLYS021A. Accessed 30 October 2026. [Online]. Available: https://www.ti.com/lit/ds/symlink/ina228.pdf

[175] *INA238AIDGSR Datasheet*, Manufacturer datasheet, Texas Instruments, revision SLYS025B. Accessed 30 October 2026. [Online]. Available: https://www.ti.com/lit/ds/symlink/ina238.pdf

[176] *INA240A1PWR Datasheet*, Manufacturer datasheet, Texas Instruments, revision SBOS662C. Accessed 30 October 2026. [Online]. Available: https://www.ti.com/lit/ds/symlink/ina240.pdf

[177] *INA260AIPWR Datasheet*, Manufacturer datasheet, Texas Instruments, revision SBOS656C. Accessed 30 October 2026. [Online]. Available: https://www.ti.com/lit/ds/symlink/ina260.pdf

[178] *LMP8640MK-F/NOPB Datasheet*, Manufacturer datasheet, Texas Instruments, revision SNOSB28H. Accessed 11 November 2026. [Online]. Available: https://www.ti.com/lit/ds/symlink/lmp8640.pdf

[179] *LM35CAZ/NOPB Datasheet*, Manufacturer datasheet, Texas Instruments, revision SNIS159H. Accessed 30 October 2026. [Online]. Available: https://www.ti.com/lit/ds/symlink/lm35.pdf

[180] *LMT84DCKT Datasheet*, Manufacturer datasheet, Texas Instruments, revision SNIS167E. Accessed 11 November 2026. [Online]. Available: https://www.ti.com/lit/ds/symlink/lmt84.pdf

[181] *LMT85LPG Datasheet*, Manufacturer datasheet, Texas Instruments, revision SNIS168E. Accessed 30 October 2026. [Online]. Available: https://www.ti.com/lit/ds/symlink/lmt85.pdf

[182] *TMP101NA/3K Datasheet*, Manufacturer datasheet, Texas Instruments, revision SBOS231I. Accessed 11 November 2026. [Online]. Available: https://www.ti.com/lit/ds/symlink/tmp101.pdf

[183] *TMP102AIDRLR Datasheet*, Manufacturer datasheet, Texas Instruments, revision SBOS397I. Accessed 30 October 2026. [Online]. Available: https://www.ti.com/lit/ds/symlink/tmp102.pdf

[184] *TMP1075DGKR Datasheet*, Manufacturer datasheet, Texas Instruments, revision SBOS854F. Accessed 30 October 2026. [Online]. Available: https://www.ti.com/lit/ds/symlink/tmp1075.pdf

[185] *TMP112AIDRLR Datasheet*, Manufacturer datasheet, Texas Instruments, revision SBOS473L. Accessed 30 October 2026. [Online]. Available: https://www.ti.com/lit/ds/symlink/tmp112.pdf

[186] *TMP116AIDRVR Datasheet*, Manufacturer datasheet, Texas Instruments, revision SBOS740A. Accessed 30 October 2026. [Online]. Available: https://www.ti.com/lit/ds/symlink/tmp116.pdf

[187] *TMP117AIDRVR Datasheet*, Manufacturer datasheet, Texas Instruments, revision SNOSD82D. Accessed 30 October 2026. [Online]. Available: https://www.ti.com/lit/ds/symlink/tmp117.pdf

[188] *TMP235A2DCKR Datasheet*, Manufacturer datasheet, Texas Instruments, revision SBOS857E. Accessed 30 October 2026. [Online]. Available: https://www.ti.com/lit/ds/symlink/tmp235.pdf

[189] *Temperature Sensors Line Guide*, Manufacturer product guide, Honeywell, document 009033-4-EN IL50. Accessed 11 November 2026. [Online]. Available: https://prod-edam.honeywell.com/content/dam/honeywell-edam/sps/siot/en-us/products/sensors/temperature-sensors/common/documents/sps-siot-temperature-sensors-line-guide-009033-4-en-ciid-45085.pdf

[190] *AD7490 Datasheet*, Manufacturer datasheet, Analog Devices, revision A. Accessed 11 November 2026. [Online]. Available: https://www.datasheets.pl/elementy_czynne/IC/AD/AD7490.pdf

[191] *MAX17205G+T Datasheet*, Manufacturer datasheet, Analog Devices, document 19-8424, revision 2. Accessed 11 November 2026. [Online]. Available: https://raw.githubusercontent.com/oresat/oresat-batteries/78c092388955d8bcce095ed5f090b1c0dfa9226c/docs/Datasheets/Max-MAX17205-Fuel_Gauge.pdf

[192] *MIC842LYC5-TR Datasheet*, Manufacturer datasheet, Microchip Technology, revision DS20005758A. Accessed 11 November 2026. [Online]. Available: https://ww1.microchip.com/downloads/en/DeviceDoc/20005758A.pdf

[193] *HK6903 Heater Drawing*, Manufacturer product drawing, Minco, revision B. Accessed 11 November 2026. [Online]. Available: https://dpk3n3gg92jwt.cloudfront.net/domains/minco/pdf/HK6903.pdf

[194] *KHLVA-0502/5, KHLVA-0504/2, KHLVA-101/5, KHLVA-102/2, KHLVA-103/2 Datasheet*, Manufacturer datasheet, Omega Engineering, accessed 30 October 2026. [Online]. Available: https://assets.omega.com/spec/KHRA-KHLVA-KHA-SERIES.pdf

[195] *ATSAMD21G18A-AU Datasheet*, Manufacturer datasheet, Microchip Technology, revision DS40001882L 2026. Accessed 30 October 2026. [Online]. Available: https://ww1.microchip.com/downloads/aemDocuments/documents/MCU32/ProductDocuments/DataSheets/SAM-D21-DA1-Family-Data-Sheet-DS40001882.pdf

[196] *PIC16F1789 Datasheet*, Manufacturer datasheet, Microchip Technology, revision DS40001675C. Accessed 11 November 2026. [Online]. Available: https://ww1.microchip.com/downloads/en/DeviceDoc/40001675C.pdf

[197] *PIC16LF18326-I/SL Datasheet*, Manufacturer datasheet, Microchip Technology, revision DS40001839B 2017 preliminary. Accessed 30 October 2026. [Online]. Available: https://ww1.microchip.com/downloads/en/DeviceDoc/40001839B.pdf

[198] *STM32F091CBT6, STM32F091CCT7 Datasheet*, Manufacturer datasheet, STMicroelectronics, document DocID026284, revision 4. Accessed 11 November 2026. [Online]. Available: https://www.st.com/resource/en/datasheet/stm32f091cb.pdf

[199] *MSP430FR2433IRGER Datasheet*, Manufacturer datasheet, Texas Instruments, revision SLASE59F 2019-12. Accessed 30 October 2026. [Online]. Available: https://www.ti.com/lit/ds/symlink/msp430fr2433.pdf

[200] *MSP430G2553IN20 Datasheet*, Manufacturer datasheet, Texas Instruments, revision SLAS735J 2013-05. Accessed 30 October 2026. [Online]. Available: https://www.ti.com/lit/ds/symlink/msp430g2553.pdf

[201] *MSPM0L1306TRGER Datasheet*, Manufacturer datasheet, Texas Instruments, revision SLASEX0D 2024-01. Accessed 30 October 2026. [Online]. Available: https://www.ti.com/lit/ds/symlink/mspm0l1306.pdf

[202] *MR25H40CDFR Datasheet*, Manufacturer datasheet, Everspin Technologies, revision 12.4 2018-03. Accessed 30 October 2026. [Online]. Available: https://www.everspin.com/sites/default/files/MR2xH40_Datasheet.pdf

[203] *24LC256-I/SN Datasheet*, Manufacturer datasheet, Microchip Technology, revision DS20001203W 2019. Accessed 30 October 2026. [Online]. Available: https://ww1.microchip.com/downloads/en/DeviceDoc/24AA256-24LC256-24FC256-Data-Sheet-20001203W.pdf

[204] *SST25VF016B-50-4I-S2AF Datasheet*, Manufacturer datasheet, Microchip Technology, revision DS20005044C 2015. Accessed 30 October 2026. [Online]. Available: https://ww1.microchip.com/downloads/en/DeviceDoc/20005044C.pdf

[205] *MB85RC256VPNF-G-JNERE1 Datasheet*, Manufacturer datasheet, RAMXEED, revision DS501-00017-11v3-E. Accessed 30 October 2026. [Online]. Available: https://www.ramxeed.com/assets/images/products/datasheet/FeRAM/i3/MB85RC256V-DS11v3-E.pdf

[206] *MB85RS256BPNF-G-JNERE1 Datasheet*, Manufacturer datasheet, RAMXEED, revision DS501-00021-5v2-E 2025-10. Accessed 30 October 2026. [Online]. Available: https://www.ramxeed.com/assets/images/products/datasheet/FeRAM/s4/MB85RS256B-DS5v2-E.pdf

[207] *AB1805-T3 Datasheet*, Manufacturer datasheet, Abracon, revision C 2014-10-16. Accessed 30 October 2026. [Online]. Available: https://abracon.com/Precisiontiming/AB18X5-RTC.pdf

[208] *PCF8523T/1,118 Datasheet*, Manufacturer datasheet, NXP Semiconductors, revision 7 2015-04-28. Accessed 30 October 2026. [Online]. Available: https://www.nxp.com/docs/en/data-sheet/PCF8523.pdf

[209] *BQ32000D Datasheet*, Manufacturer datasheet, Texas Instruments, revision SLUS900F 2026-08. Accessed 30 October 2026. [Online]. Available: https://www.ti.com/lit/ds/symlink/bq32000.pdf

[210] *RV-3028-C7 TA QC Datasheet*, Manufacturer datasheet, Micro Crystal, revision 1.4 2021-11. Accessed 30 October 2026. [Online]. Available: https://www.microcrystal.com/fileadmin/Media/Products/RTC/App.Manual/RV-3028-C7_App-Manual.pdf

[211] *RV-3032-C7 TA QC Datasheet*, Manufacturer datasheet, Micro Crystal, revision 1.3 2023-05. Accessed 30 October 2026. [Online]. Available: https://www.microcrystal.com/fileadmin/Media/Products/RTC/App.Manual/RV-3032-C7_App-Manual.pdf

[212] *STWD100NYWY3F Datasheet*, Manufacturer datasheet, STMicroelectronics, document DocID14134, revision 11. Accessed 11 November 2026. [Online]. Available: https://www.st.com/resource/en/datasheet/stwd100.pdf

[213] *DFLS2100Q-7 Datasheet*, Manufacturer datasheet, Diodes Incorporated, revision DS38837 Rev. 3 - 2. Accessed 11 November 2026. [Online]. Available: https://www.diodes.com/datasheet/download/DFLS2100Q.pdf

[214] OreSat Project, "OreSat power domain (OPD) design notes," Public mission hardware documentation, Aug. 2021, stated update: 22 August 2021. OPD supply, current sensing and TPS62111/TPS62112 converter selection. [Online]. Available: https://docs.google.com/document/d/1NOkO6gGEJUFRpOrkwClRlZlzWuTLteq6fP9LPf_Ez0M/edit

Appendix A

User Trial Evidence

The prepared mission conditions and numerical allocations are purely illustrative, as their only purpose is to provide common starting points for the timed tasks. The inputs are utilized as values for the corresponding properties of the MBSE models.

Table A.1: Illustrative inputs shared by the prepared sizing tasks. Changes for each task are given in Table A.2. Groups are illustrative.

Group	Prepared values
Mission and orbit	1U bus, lifetime 2 years, 365.25 days/year, altitude 600 km, eccentricity 0.05, beta angle 0°, Sun projection angle 30°, Earth radius 6378 km and gravitational parameter 398601 km^3/s^2.
Attitude	Ideal Sun pointing with the $+Z$ face toward the Sun and the array mounted on $+Z$.
EPS loads and accounting	Sunlight and eclipse mean loads 5 and 3 W. Delivered energy allocations 25000 and 8000 J. Each of the four default EPS components has mass 0.1 kg and volume 0.0001 m^3.
Array electrical inputs	6S4P, EOL cell voltage 2.3 V, EOL cell current 0.45 A, open-circuit cell voltage 2.6 V and short-circuit cell current 0.5 A. Sunlight and eclipse path efficiencies 0.8 and 0.65. Ideal MPPT through the regulator and converter.
Array area and illumination	Cell active area 0.00345 m^2, packing fraction 0.9, usable area 0.1 m^2, BOL efficiency 0.32, lifetime retention 0.9375, temperature factor 1, irradiance 1000 W/m^2, inherent retention 1 and assembly efficiency 1.
Battery	1S2P. Cell ratings 3.7 V, 2.75 Ah and 10.175 Wh. Cell voltage interval 3.0–4.2 V. Cell charge and discharge limits 1 and 3 A. Cycle law coefficient 2000, exponent -1 and characterized DoD interval 0.1–0.8. Degradation margin 0.2. Energy and capacity retention 0.8.
Receiving limits	Converter input interval 6–18 V, absolute maximum input 22 V and maximum input current 2.5 A. Output 5 V and maximum output current 3 A. Battery receiving interval 2.8–4.3 V. Required battery charge and discharge currents 1 and 2 A.

Group	Prepared values
Load rail	Instrument mass 0.01 kg and volume 0.00001 m^3. Active on state 5 V and 1 A, off state 0 V and 0 A. The bus voltage and current checks link this on state to the converter output ratings.

A.1 Task presentations

Table A.2: Task inputs and requested submissions. Each role receives a separate copy of the same initial mission for each task.

MOP	Prepared state and task
01	Given MR_BASE, which limits EPS mass to 0.6 kg, create MR_CHILD at mission level. Its parent is MR_BASE and its child list is empty. Limit battery mass to 0.12 kg through a condition named maximum, with measure linked to battery mass and predicate measure $\leq$ target. Retain the rationale "Reserve 0.48 kg for the other EPS components," verification method Analysis, resolution status Resolved and unresolved field reason None. The identifier is the requirement name. Parent and child identifiers, method and resolution information use explicit requirement parameters. Save and reopen the requirement, then submit its fields and condition result.
02	Retrieve MR_PARENT's forward links to MR_ARRAY and MR_BATTERY, their allocations to SolarArray and Battery, and each child's reverse link to MR_PARENT. The prepared links use parent, child and component identifier parameters. Submit the associated identifiers without changing them.
03	Inspect analysis_complete, with 2 W multiplied by 5 s, and analysis_failed, which deliberately divides 2 W by zero seconds. Both retain a link to MR_ENERGY, a method description, a synthetic nominal scenario and their input bindings. Submit the available result views and execution states. Report any limitation in retrieving retained execution records.
04	Reload the retained sizing mission through Reopen and rerun sunlight_energy and solar_power through their stored bindings. Submit the rerun results. The prepared environment supplies the framework dependency.
05	Fill seven prepared allocation parameters. Assign generation to SolarArray, storage to Battery, conversion and regulation to RegulatorAndConverter, and protection, distribution and control to PowerCircuit. Save each assignment and submit the mission requirement table. The retained component identifier appears in each requirement statement.
06	Apply four separate baseline variations. Change lifetime from 2 to 1 year, eclipse mean load from 3 to 4 W, battery parallel count from 2 to 3, and Earth radius from 6378 to 6371 km. Restore each preceding edit before the next variation. Capture intermediate EPS result views for each variation and submit the final variation's result view.

MOP	Prepared state and task
07	The array's EOL cell current is omitted. Identify the missing quantity through the generated solar_cell_count requirement, supply 0.45 A, save and submit the updated requirement.
08	Inspect the generated solar_cell_count requirement. Submit its required count calculation, required power and cell power mappings, and minimum verification condition. Preserve the expressions and inputs.
09	Use the GUI and terminal in the prepared mission repository to save an authored baseline with the decision rationale "Approved by trial coordinator for this synthetic demonstration, retain the 600 km baseline." Use author Trial Operator. Change altitude from 600 to 650 km in the GUI, inspect Git status and the diff using the briefed commands, retrieve the baseline through Git and reopen it. Submit the retained requirement, mission, design and model revision identifiers R0, M0, D0 and A0 and the decision history. These are illustrative revision labels.
11	Query the battery alternatives in Table A.3. The generic battery's direct requirements demand minimum rated energy 11 Wh, maximum mass 0.075 kg and nominal voltage 3.6 V. Find all matches and sort by rated energy, highest first. Submit the part numbers and leave the live battery unassigned.
13	Start from a blank mission input sheet with the bus, instrument, modes and allocation rows already created. Enter lifetime 2 years, 365.25 days/year, altitude 600 km, eccentricity 0, beta angle 0°, Sun pointing on $+Z$, and irradiance 1000 W/m^2. Set nominal orbital fraction and the instrument's nominal mode share to 1, and on/off state shares to 0.2/0.8. Set the on state to 5 V and 1 A and the off state to 0 V and 0 A. Reopen and submit retained inputs. A prepared requirement statement displays their bindings without inferring chronology or averaging.

Table A.3: Catalogue values supplied to MOP-11. The supplied energy values are minimum ratings and the masses are maximum ratings. Every candidate has nominal voltage 3.6 V.

Molicel part number	Energy (Wh)	Mass (kg)
INR-18650-M30A	10.4	0.048
INR-18650-M35A	11.9	0.048
INR-21700-P42A	14.7	0.070
INR-21700-P45B	15.5	0.070
INR-18650-P26A	Unknown	0.050

A.2 Submitted states and elapsed times

Table A.4 gives the shared initial state, the student club member's submitted state and both participants' times for each task. Each reported time is four times the recorded duration.

Table A.4: User trial submissions. Screenshots show the shared initial states and the student club member's submissions. Times are in seconds.

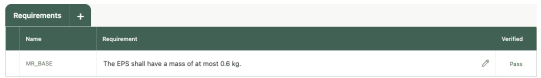
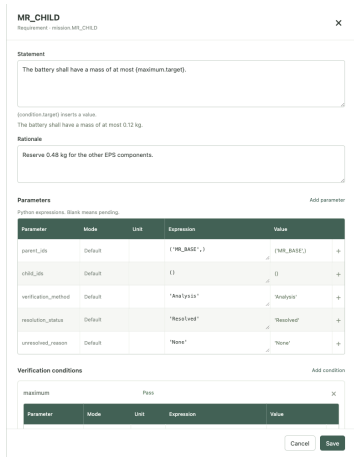
MOP/Task	Shared initial state	Final state of student club member	Time taken by PhD student	Time taken by student club member	Verifies FR?
MOP-01 FR-01 Create and retain a requirement			226.88	243.76	Yes
MOP-02 FR-02 Retrieve forward and reverse links			136.28	115.80	Yes

Table A.4: User trial submissions (continued). Times are in seconds.

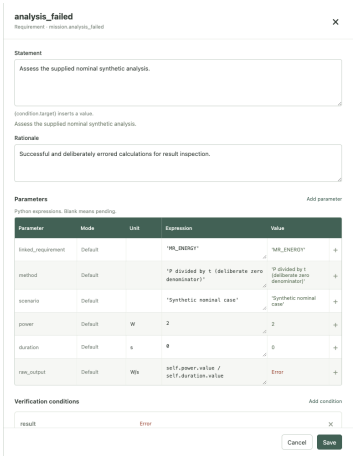
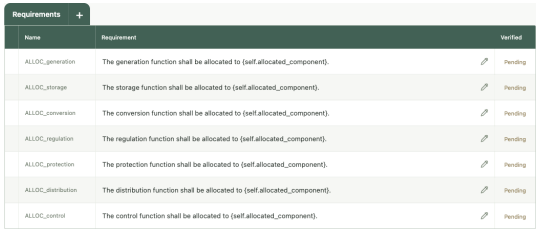
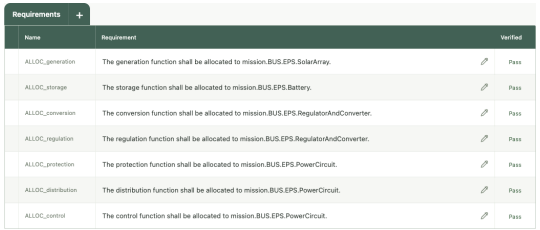
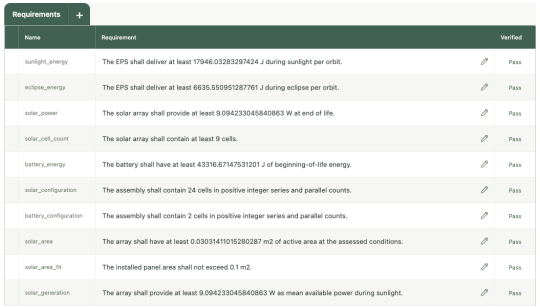
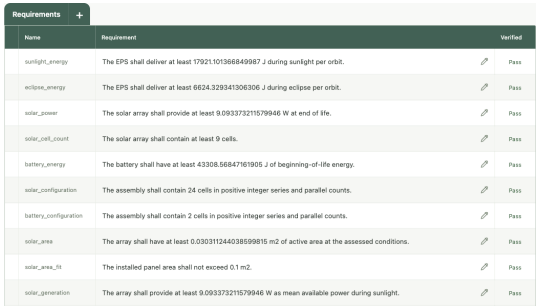
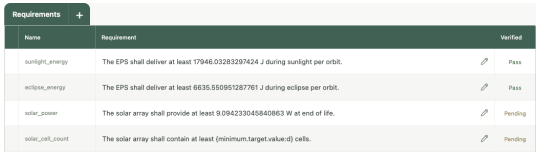
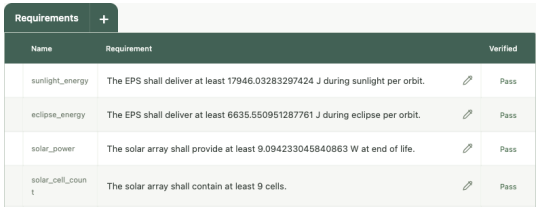
MOP/Task	Shared initial state	Final state of student club member	Time taken by PhD student	Time taken by student club member	Verifies FR?
MOP-03 FR-03 Inspect analysis results and errors			112.28	88.20	Yes
MOP-04 FR-04 Reconstruct and rerun analyses			127.32	160.76	Yes

Table A.4: User trial submissions (continued). Times are in seconds.

MOP/Task	Shared initial state	Final state of student club member	Time taken by PhD student	Time taken by student club member	Verifies FR?
MOP-05 FR-05 Allocate seven EPS functions			317.20	264.48	Yes
MOP-06 FR-06 Assess four separate changes			310.92	318.16	Yes
MOP-07 FR-07 Resolve an omitted input			140.40	140.12	Yes

MOP-06 shows the final Earth radius variation. Its intermediate submissions appear in Figure A.2.

Table A.4: User trial submissions (continued). Times are in seconds.

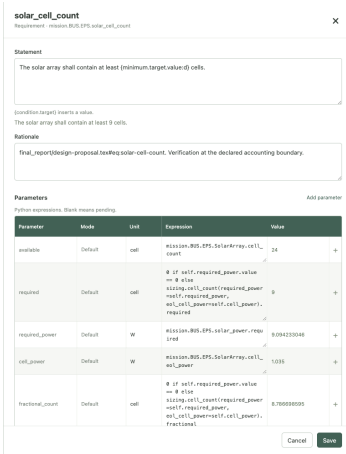
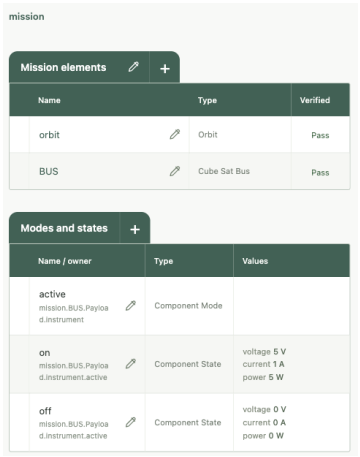
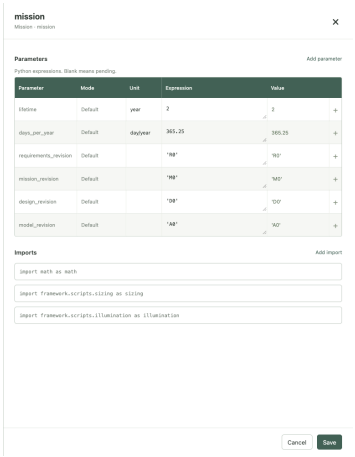
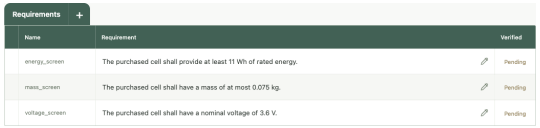
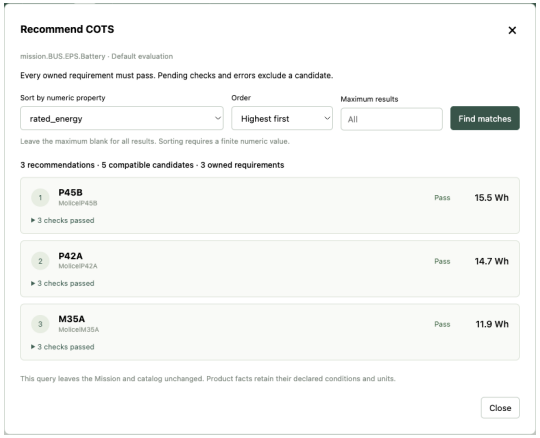
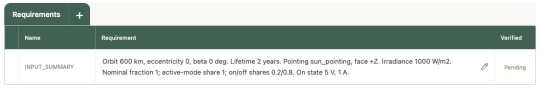
MOP/Task	Shared initial state	Final state of student club member	Time taken by PhD student	Time taken by student club member	Verifies FR?
MOP-08 FR-08 Inspect assigned methods and mappings			136.48	151.28	Yes

Table A.4: User trial submissions (continued). Times are in seconds.

MOP/Task	Shared initial state	Final state of student club member	Time taken by PhD student	Time taken by student club member	Verifies FR?
MOP-09 FR-09 Save and retrieve a baseline			198.96	290.52	Yes
MOP-11 FR-11 Query compatible COTS			79.60	79.72	Yes
MOP-13 FR-13 Enter and reopen mission inputs			250.72	281.76	Yes

MOP-09 uses both the GUI and terminal. MOP-11 shows component query results, and MOP-13 shows retained mission inputs.

A.3 Evidence detail

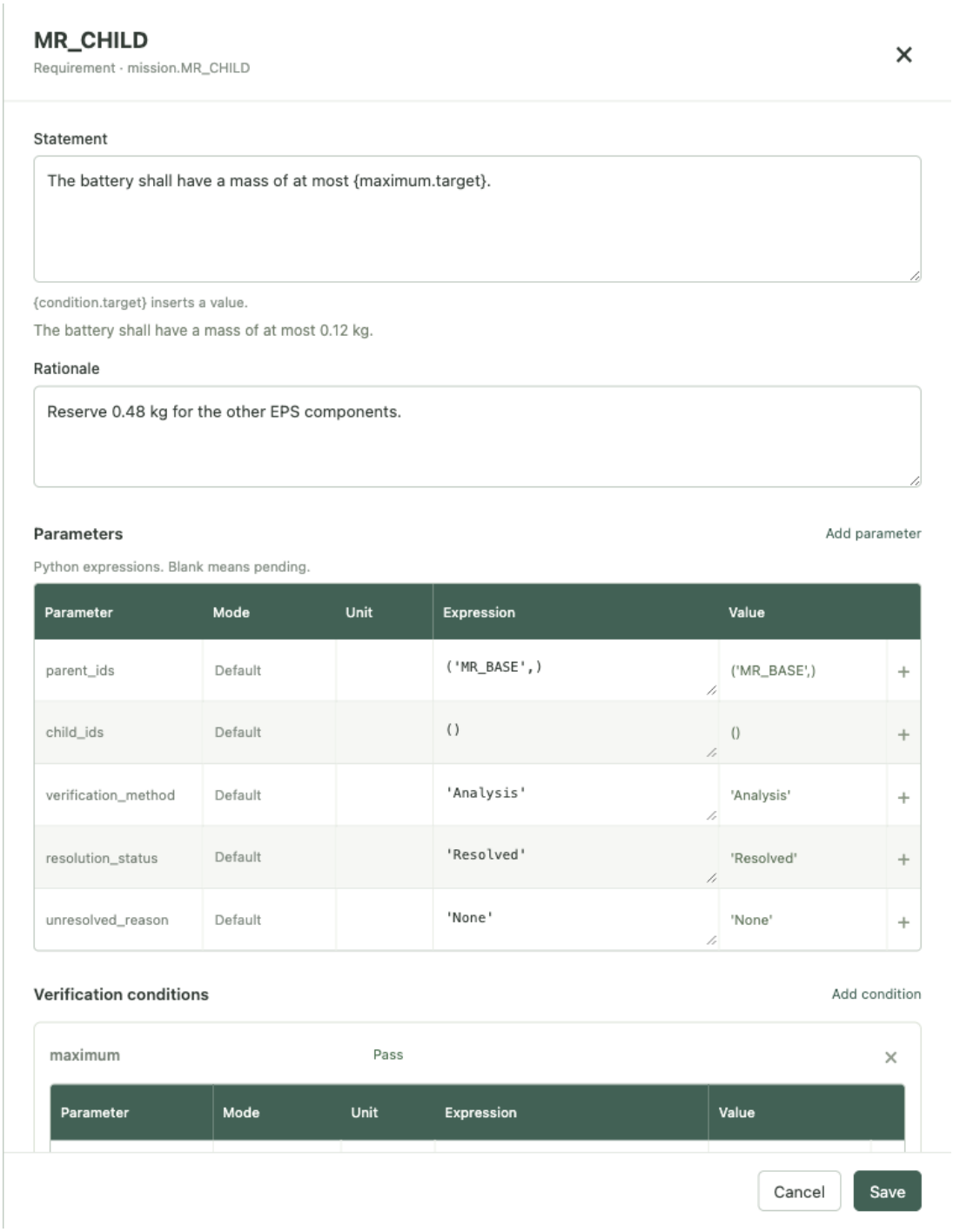


Figure A.1: Enlarged MOP-01 submission by the student club member, showing retained requirement fields and the verification condition status.

Mission change

Requirements +

Name	Requirement	Verified
sunlight_energy	The EPS shall deliver at least 17946.03283297424 J during sunlight per orbit.	Pass
eclipse_energy	The EPS shall deliver at least 6635.550951287761 J during eclipse per orbit.	Pass
solar_power	The solar array shall provide at least 9.094233045840863 W at end of life.	Pass
solar_cell_count	The solar array shall contain at least 9 cells.	Pass
battery_energy	The battery shall have at least 21658.335737656005 J of beginning-of-life energy.	Pass

Requirement change

Requirements +

Name	Requirement	Verified
sunlight_energy	The EPS shall deliver at least 17946.03283297424 J during sunlight per orbit.	Pass
eclipse_energy	The EPS shall deliver at least 8847.40126838368 J during eclipse per orbit.	Fail
solar_power	The solar array shall provide at least 10.042310727787816 W at end of life.	Pass
solar_cell_count	The solar array shall contain at least 10 cells.	Pass
battery_energy	The battery shall have at least 57755.561967082685 J of beginning-of-life energy.	Pass

Component change

Requirements +

Name	Requirement	Verified
sunlight_energy	The EPS shall deliver at least 17946.03283297424 J during sunlight per orbit.	Pass
eclipse_energy	The EPS shall deliver at least 6635.550951287761 J during eclipse per orbit.	Pass
solar_power	The solar array shall provide at least 9.094233045840863 W at end of life.	Pass
solar_cell_count	The solar array shall contain at least 9 cells.	Pass
battery_energy	The battery shall have at least 43316.67147531201 J of beginning-of-life energy.	Pass
solar_configuration	The assembly shall contain 24 cells in positive integer series and parallel counts.	Pass
battery_configuration	The assembly shall contain 3 cells in positive integer series and parallel counts.	Pass

Model change

Requirements +

Name	Requirement	Verified
sunlight_energy	The EPS shall deliver at least 17921.101366849987 J during sunlight per orbit.	Pass
eclipse_energy	The EPS shall deliver at least 6624.329341306306 J during eclipse per orbit.	Pass
solar_power	The solar array shall provide at least 9.093373211579946 W at end of life.	Pass
solar_cell_count	The solar array shall contain at least 9 cells.	Pass
battery_energy	The battery shall have at least 43308.56847161905 J of beginning-of-life energy.	Pass

Figure A.2: Student club member's intermediate screenshots for the four separate MOP-06 variations. Each preceding edit was restored before the next variation. The task clock continued through these captures and stopped at the final submission screenshot.

Appendix B

Automatic EPS Requirements

The requirements automatically assigned to EPS elements form part of the fourth objective's deliverable, which characterizes early EPS verification and validation. Table B.1 associates each requirement statement with its EPS element. Calculated targets are shown symbolically because their numerical values depend on the mission inputs, following the equations in Chapter 4. The EPS owns the first 18 requirements, and each added protection component owns its normal operation requirement.

Table B.1: Requirements automatically assigned to EPS elements.

EPS element	**Requirement statement**
EPS	The EPS shall deliver at least E_s of energy to the loads during sunlight per orbit, where $E_s = \overline{P}_{L,s}T_s$ in J (Equation (4.14)).
EPS	The EPS shall deliver at least E_e of energy to the loads during eclipse per orbit, where $E_e = \overline{P}_{L,e}T_e$ in J (Equation (4.15)).
Solar array	The solar array shall provide at least $P_{\mathrm{sp}_{\mathrm{min}}}$ in W at end of life (Equation (4.16)).
Solar array	The solar array shall contain at least $\lceil N_{\mathrm{sc}} \rceil$ cells (Equation (4.18)).
Battery	The battery shall have at least E_{Battery} in J of beginning-of-life energy (Equation (4.21)).
Solar array	The solar array shall contain $N_s^{\mathrm{pv}} N_p^{\mathrm{pv}}$ cells arranged with positive integer series and parallel counts (Equation (4.22), for one section).
Battery	The battery shall contain $N_s^{\mathrm{bat}} N_p^{\mathrm{bat}}$ cells arranged with positive integer series and parallel counts (Equation (4.55)).
Solar array	The array shall have at least $A_{\mathrm{active,min}}$ in m^2 of active area at the assessed conditions (Equation (4.48)).
Solar array	The installed panel area shall not exceed the entered usable area A_{usable} in m^2 (Equation (4.39)).
Solar array	The array shall provide at least $P_{\mathrm{sp}_{\mathrm{min}}}$ in W as mean available source power during sunlight (Equation (4.54)).
Solar array	The array operating voltage shall remain between the explicitly assigned minimum and maximum receiving voltages in V.

EPS element	Requirement statement
Battery	The battery pack voltage shall remain between the explicitly assigned minimum and maximum receiving voltages in V.
Solar array	The array open-circuit voltage shall not exceed the assigned receiver's absolute voltage limit $V_{\mathrm{abs},r}$ in V (Equation (4.25)).
Solar array	The array short-circuit current shall not exceed the assigned receiver's permissible source current $I_{\mathrm{source,allow},r}$ in A (Equation (4.25)).
Bus power path	The selected bus path shall supply at least the entered required load current I_{required} in A (Equation (4.60)).
Battery	The battery shall accept at least the entered required charging current in A.
Battery	The battery shall supply at least the entered required discharge current in A.
Bus power path	The selected bus path shall provide the entered required voltage V_{required} within the entered tolerance $\epsilon_V \geq 0$ in V (Equation (4.60)).
Protection	The declared protection detectors shall remain inactive for all supplied normal operating samples under the applicable bounds in Equation (4.63).

Appendix C

COTS Database

The COTS database forms part of the third objective's deliverable by linking the generic EPS architecture to physical component alternatives. Table C.1 identifies each product's generic EPS component type, manufacturer and part identifier, with a citation to its datasheet or other manufacturer documentation. Mass is shown as one representative property, with its rating type retained. Unknown denotes a mass not established in the database.

Table C.1: COTS products and source documents.

EPS component	Manufacturer	Part identifier	Mass (kg)	Datasheet
Battery	Molicel	INR-18650-M30A	0.048 (maximum)	[78, p. 1]
Battery	Molicel	INR-18650-M35A	0.048 (maximum)	[79, p. 1]
Battery	Molicel	INR-18650-P26A	0.05 (maximum)	[80, p. 1]
Battery	Molicel	INR-18650-P28A	0.046 (typical)	[81, p. 1]
Battery	Molicel	INR-18650-P28B	0.048 (maximum)	[82, p. 1]
Battery	Molicel	INR-18650-P30B	0.047 (maximum)	[83, p. 1]
Battery	Molicel	INR-21700-M65A	0.0745 (maximum)	[84, p. 1]
Battery	Molicel	INR-21700-P42A	0.070 (maximum)	[85, p. 1]
Battery	Molicel	INR-21700-P45B	0.070 (maximum)	[86, p. 1]
Battery	Molicel	INR-21700-P50B	0.071 (maximum)	[87, p. 1]
Battery	Panasonic	BK-3MCC (eneloop)	0.027 (typical)	[88, p. 1]
Battery	Tenergy	30005-0 (ICR18650-2600, LR1865SK)	0.045 (typical)	[89, p. 3]
Battery	GomSpace	NanoPower BPX 100 Wh (8S1P)	0.5 (nominal)	[90, p. 17]
Solar array	AZUR SPACE Solar Power GmbH	81441	0.0026 (nominal)	[91, p. 2]
Solar array	AZUR SPACE Solar Power GmbH	82921	0.0033 (nominal)	[92, p. 2]

EPS component	Manufacturer	Part identifier	Mass (kg)	Datasheet
Solar array	CESI S.p.A.	CTJ30	Unknown	[93]
Solar array	CESI S.p.A.	CTJLC	Unknown	[94]
Solar array	Sharp Energy Solutions Corporation	IMM3J Solar Cells	0.00033976 (derived)	[95, pp. 1–2]
Solar array	Spectrolab Inc.	XTE+ GEO	Unknown	[96]
Solar array	Spectrolab Inc.	XTE+ LEO	Unknown	[97]
Solar array	Spectrolab Inc.	XTE-HF	Unknown	[98]
Solar array	Spectrolab Inc.	XTE-LILT	Unknown	[99]
Solar array	Spectrolab Inc.	XTE-SF	Unknown	[100]
Solar array	Spectrolab Inc.	XTJ Prime	Unknown	[101]
Solar array	AZUR SPACE Solar Power GmbH	3G30A (TJ Solar Cell Assembly)	Unknown	[102]
DC/DC converter	Analog Devices	LT1618EMS#TRPBF	Unknown	[103]
DC/DC converter	Analog Devices	LTC3119EUFD#PBF	Unknown	[104]
DC/DC converter	Analog Devices	LTC3530EDD#PBF	Unknown	[105]
DC/DC converter	Texas Instruments	LMR33630ADDAR	Unknown	[106]
DC/DC converter	Texas Instruments	TPS61022RWUR	Unknown	[107]
DC/DC converter	Texas Instruments	TPS61023DRLR	Unknown	[108]
DC/DC converter	Texas Instruments	TPS61235PRWLT	Unknown	[109]
DC/DC converter	Texas Instruments	TPS62088YFPR	Unknown	[110]
DC/DC converter	Texas Instruments	TPS62111RSAT	Unknown	[111]
DC/DC converter	Texas Instruments	TPS62112RSAT	Unknown	[111]
DC/DC converter	Texas Instruments	TPS62130ARGTR	Unknown	[112]
DC/DC converter	Texas Instruments	TPS62160DSGR	Unknown	[113]
DC/DC converter	Texas Instruments	TPS62933DRLR	Unknown	[114]
DC/DC converter	Texas Instruments	TPS63020DSJR	Unknown	[115]

EPS component	**Manufacturer**	**Part identifier**	**Mass (kg)**	**Datasheet**
DC/DC converter	Texas Instruments	TPS63020QDSJTQ1	Unknown	[116]
DC/DC converter	Texas Instruments	TPS63070RNMR	Unknown	[117]
DC/DC converter	Texas Instruments	TPS63802DLAR	Unknown	[118]
Regulator/ converter	Texas Instruments	LP2985AIM5-1.8/NOPB	Unknown	[119]
Regulator/ converter	Texas Instruments	LP2985AIM5-2.5/NOPB	Unknown	[120]
Regulator/ converter	Texas Instruments	LP2985AIM5-3.3/NOPB	Unknown	[120]
Integrated EPS	ISISPACE	ICEPS2 Type A	0.184 (typical)	[121, p. 9]
Integrated EPS	ISISPACE	IMEPS2 (1 PCU, 1 PBU, 1 PDU, 1 PBP-4S1P)	Unknown	[122]
Integrated EPS	Pumpkin	EPSM1 Rev C	0.230 (typical)	[123, p. 7]
Battery charger	Texas Instruments	BQ24074RGTR	Unknown	[124]
Battery charger	Texas Instruments	BQ24610RGER	Unknown	[125]
Battery charger	Texas Instruments	BQ24650RVAR	Unknown	[126]
Battery charger	Texas Instruments	BQ25606RGER	Unknown	[127]
Battery charger	Texas Instruments	BQ25798RQMR	Unknown	[128]
Battery protector	ABLIC	S-8209AAA-T8T1U	Unknown	[129]
Battery protector	ABLIC	S-8252AAG-M6T1U	Unknown	[130]
Battery protector	Texas Instruments	BQ29700DSER	Unknown	[131]
Battery protector	Texas Instruments	BQ298000RUGR	Unknown	[132]
Battery protector	Texas Instruments	BQ7790500PWR	Unknown	[133]
Battery protector	Texas Instruments	BQ7791500PWR	Unknown	[134]
Ideal diode	Analog Devices	LTC4359IMS8#PBF	Unknown	[135]
Ideal diode	Analog Devices	LTC4412ES6#TRPBF	Unknown	[136]
Ideal diode	Analog Devices	LTC4412IS6#TRPBF	Unknown	[136]

EPS component	Manufacturer	Part identifier	Mass (kg)	Datasheet
Ideal diode	Texas Instruments	LM5050MK-1/NOPB	Unknown	[137]
Ideal diode	Texas Instruments	LM66100DCKR	Unknown	[138]
Ideal diode	Texas Instruments	LM66200DRLR	Unknown	[139]
Ideal diode	Texas Instruments	LM74700QDBVRQ1	Unknown	[140]
Load switch	Omron	G6KU-2F-Y DC3	0.0007 (nominal)	[141, p. 3]
Load switch	Analog Devices	LTC4361IDC-2#TRMPBF	Unknown	[142]
Load switch	Analog Devices	LTC4368IMS-2#PBF	Unknown	[143]
Load switch	Analog Devices	MAX892LEUA+T	Unknown	[144]
Load switch	Texas Instruments	TPS22919DCKR	Unknown	[145]
Load switch	Texas Instruments	TPS22948DCKR.A	Unknown	[146]
Load switch	Texas Instruments	TPS25200DRVR	Unknown	[147]
Load switch	Texas Instruments	TPS2553DBVR	Unknown	[148]
Load switch	Texas Instruments	TPS2556DRBR	Unknown	[149]
Load switch	Texas Instruments	TPS25921ADR	Unknown	[150]
Load switch	Texas Instruments	TPS25940ARVCR	Unknown	[151]
Load switch	Texas Instruments	TPS259531DSGR	Unknown	[152]
Load switch	Texas Instruments	TPS259620DDAT	Unknown	[153]
Load switch	Texas Instruments	TPS259621DDAT	Unknown	[153]
Load switch	Texas Instruments	TPS259824ONRGER	Unknown	[154]
Load switch	Texas Instruments	TPS26600PWPR	Unknown	[155]
Load switch	Texas Instruments	TPS26630RGER	Unknown	[156]
Power circuit switch	Diodes Incorporated	DMP2240UDM-7	0.000015 (nominal)	[157, p. 1]

EPS component	Manufacturer	Part identifier	Mass (kg)	Datasheet
Power circuit switch	Diodes Incorporated	DMP3050LVT-7	0.000013 (nominal)	[158, p. 1]
Power circuit switch	Littelfuse	1206L150/12SLYR	Unknown	[159]
Power circuit switch	Littelfuse	1206L500/12SLPR	Unknown	[159]
Power circuit switch	Nexperia	PMV45EN2R	Unknown	[160]
Power circuit switch	onsemi	NTR4101PT1G	Unknown	[161]
Power circuit switch	Toshiba	SSM6K411TU	0.000007 (typical)	[162, p. 1]
Power circuit switch	Vishay	SiA447DJ-T1-GE3	Unknown	[163]
Power circuit switch	Vishay Siliconix	Si7106DN	Unknown	[164]
Power circuit switch	Vishay Siliconix	Si7232DN-T1-GE3	Unknown	[165]
Power circuit switch	Vishay Siliconix	Si7625DN-T1-GE3	Unknown	[166]
Current sensor	Allegro MicroSystems	ACS722LLCTR-05AB-T	Unknown	[167]
Current sensor	Allegro MicroSystems	ACS722LLCTR-10AU	Unknown	[167]
Current sensor	Analog Devices	LT6106HS5#TRMPBF	Unknown	[168]
Current sensor	Analog Devices	MAX4211FETE	Unknown	[169]
Current sensor	Texas Instruments	INA180A1IDBVR	Unknown	[170]
Current sensor	Texas Instruments	INA190A1IDDFR	Unknown	[171]
Current sensor	Texas Instruments	INA219BIDCNR	Unknown	[172]
Current sensor	Texas Instruments	INA226AIDGSR	Unknown	[173]
Current sensor	Texas Instruments	INA228AIDGSR	Unknown	[174]
Current sensor	Texas Instruments	INA238AIDGSR	Unknown	[175]
Current sensor	Texas Instruments	INA240A1PWR	Unknown	[176]
Current sensor	Texas Instruments	INA260AIPWR	Unknown	[177]

EPS component	Manufacturer	Part identifier	Mass (kg)	Datasheet
Current sensor	Texas Instruments	LMP8640MK-F/NOPB	Unknown	[178]
Temperature sensor	Texas Instruments	LM35CAZ/NOPB	Unknown	[179]
Temperature sensor	Texas Instruments	LMT84DCKT	Unknown	[180]
Temperature sensor	Texas Instruments	LMT85LPG	Unknown	[181]
Temperature sensor	Texas Instruments	TMP101NA/3K	Unknown	[182]
Temperature sensor	Texas Instruments	TMP102AIDRLR	Unknown	[183]
Temperature sensor	Texas Instruments	TMP1075DGKR	Unknown	[184]
Temperature sensor	Texas Instruments	TMP112AIDRLR	Unknown	[185]
Temperature sensor	Texas Instruments	TMP116AIDRVR	Unknown	[186]
Temperature sensor	Texas Instruments	TMP117AIDRVR	Unknown	[187]
Temperature sensor	Texas Instruments	TMP235A2DCKR	Unknown	[188]
Temperature sensor	Honeywell	197-103LAG-A01	Unknown	[189]
Sensor	Analog Devices	AD7490	Unknown	[190]
Sensor	Analog Devices	MAX17205G+T	Unknown	[191]
Sensor	Microchip Technology	MIC842LYC5-TR	Unknown	[192]
Heater	Minco	HK6903	Unknown	[193]
Heater	Omega Engineering	KHLVA-0502/5	Unknown	[194]
Heater	Omega Engineering	KHLVA-0504/2	Unknown	[194]
Heater	Omega Engineering	KHLVA-101/5	Unknown	[194]
Heater	Omega Engineering	KHLVA-102/2	Unknown	[194]
Heater	Omega Engineering	KHLVA-103/2	Unknown	[194]
Processor	Microchip Technology	ATSAMD21G18A-AU	Unknown	[195]

EPS component	**Manufacturer**	**Part identifier**	**Mass (kg)**	**Datasheet**
Processor	Microchip Technology	PIC16F1789	Unknown	[196]
Processor	Microchip Technology	PIC16LF18326-I/SL	Unknown	[197]
Processor	STMicro-electronics	STM32F091CBT6	Unknown	[198]
Processor	STMicro-electronics	STM32F091CCT7	Unknown	[198]
Processor	Texas Instruments	MSP430FR2433IRGER	Unknown	[199]
Processor	Texas Instruments	MSP430G2553IN20	Unknown	[200]
Processor	Texas Instruments	MSPM0L1306TRGER	Unknown	[201]
Nonvolatile memory	Everspin Technologies	MR25H40CDFR	Unknown	[202]
Nonvolatile memory	Microchip Technology	24LC256-I/SN	Unknown	[203]
Nonvolatile memory	Microchip Technology	SST25VF016B-50-4I-S2AF	Unknown	[204]
Nonvolatile memory	RAMXEED	MB85RC256VPNF-G-JNERE1	Unknown	[205]
Nonvolatile memory	RAMXEED	MB85RS256BPNF-G-JNERE1	Unknown	[206]
Real-time clock	Abracon	AB1805-T3	Unknown	[207]
Real-time clock	NXP Semiconductors	PCF8523T/1,118	Unknown	[208]
Real-time clock	Texas Instruments	BQ32000D	Unknown	[209]
Real-time clock	Micro Crystal	RV-3028-C7 TA QC	Unknown	[210]
Real-time clock	Micro Crystal	RV-3032-C7 TA QC	Unknown	[211]
Hardware component	STMicro-electronics	STWD100NYWY3F	Unknown	[212]
Hardware component	Diodes Incorporated	DFLS2100Q-7	0.00001 (nominal)	[213, p. 1]

Appendix D

EPS Case Study Evidence

The results are organized by FR-12 and FR-14 to FR-20. The overview gives the greatest numerical deviation for each requirement and mission across all executed numerical comparisons. The detailed tables select representative quantities and identifiable published benchmarks. Repeated outputs, deliberately failing configurations and missing/invalid-input records are omitted. No unexpected mismatch was observed.

D.1 Verification basis

Each nonzero numerical output uses a 1% relative comparison limit. Zero values, integer counts, records and discrete outcomes require exact agreement. I identifies an independent calculation from the stated inputs. P identifies a published reference value. Displayed values use seven significant figures. Deviations use unrounded values.

The electrical baselines are circular and Sun pointing. Both use $R_E = 6378$ km, $\mu = 398601$ km^3/s^2 and 365.25 days per year. Solar irradiance is 1367 W/m^2. Sunlight and eclipse load-path efficiencies are 0.85 and 0.75. The additional age-independent source derating is 0.95 and electrical assembly efficiency is 0.97. Packing fraction is 0.8 and usable section area is 0.008 m^2. These efficiencies, allocations and mission durations are assumptions.

Cell ratings come from the 3G30A datasheet, BIRDS battery documentation and OreSat component records [51, 53, 75, 89, 102]. BIRDS EOL ratings use the datasheet's 10^{15} 1 MeV electrons/cm^2 condition. OreSat EOL current retention of 0.9 is assumed at unchanged voltage. Nominal cell energies use the selected nominal voltage times charge capacity.

Electrical input	BIRDS	OreSat
Altitude (km)	400	550
Beta angle (°)	30	0
Lifetime (year)	1	2
Solar arrangement	2S1P 3G30A	2S1P XTE-SF
BOL cell MPP (V, A)	[2.409, 0.5029]	[2.43, 0.483]
EOL cell MPP (V, A)	[2.244, 0.4851]	[2.43, 0.4347]

Electrical input	BIRDS	OreSat
EOL cell Voc, Isc (V, A)	[2.512, 0.5013]	[2.75, 0.4518]
Cell active area (m^2)	0.003018	0.0027
Battery arrangement	3S2P NiMH	Two independent 2S1P Li-ion packs
Cell nominal voltage (V)	1.2	3.6
Cell capacity (Ah)	1.9	2.6
Cell nominal energy (Wh)	2.28	9.36

The BIRDS power levels approximate the measured OBC/EPS/FAB, UHF receiver and CW transmitter entries [46, p. 9]. The voltage/current representation and the separate sunlight/eclipse duty fractions are assumed. OreSat's 2019 power budget provides component loads and operating-mode energy budgets [47]. The simplified OreSat profile below uses assumed voltage/current pairs and phase-specific duty fractions.

Case	State	Voltage (V), current (A)	Sunlight, eclipse duty
BIRDS	OBC/EPS/FAB	[3.3, 0.13]	[1, 1]
BIRDS	UHF receiver	[3.3, 0.044]	[1, 1]
BIRDS	CW transmitter	[5, 0.056]	[0.25, 0.1]
OreSat	main	[3.3, 0.06]	[1, 1]
OreSat	receiver	[3.3, 0.04]	[1, 1]
OreSat	payload	[5, 0.2]	[0.2, 0.1]

Independent scalar references use decimal sums, products, ratios and linear interpolation. Products and ratios use 40 significant digits. The orbital reference solves Kepler's equation on 262144 uniform time samples and tests cylindrical Earth shadow in Cartesian coordinates. A 131072-sample calculation checks convergence. Published comparisons retain each source's component grouping and conditions. Ambiguous entries and inconsistent published budget arithmetic are excluded.

D.2 Results by requirement

All numerical maxima below meet the 1% criterion. Integer counts, source/unit records and discrete outcomes also agree with their references. A matching comparison establishes correct calculation for the entered case, including cases with unfavorable design verdicts.

Requirement	Calculation checked	BIRDS max. (%)	OreSat max. (%)
FR-12	Mass, volume and allocation margins	$< 10^{-10}$	$< 10^{-10}$
FR-14	Energy budgets and BOL/EOL sizing	0.0259	0.315
FR-15	Solar technology and battery chemistry	0.000864	0.000731
FR-16	Series and parallel arrangements	0.000864	0.107
FR-17	Bus rails and converter compatibility	0	$< 10^{-10}$
FR-18	PV power products and sampled MPP	0.331	0

Requirement	Calculation checked	BIRDS max. (%)	OreSat max. (%)
FR-19	Matched DET and ideal MPPT paths	$< 10^{-10}$	0.175
FR-20	Normal extrema and trip comparisons	0	0.0799

D.2.1 FR-12: mass, volume and allocation margins

Published checks use six BIRDS cells at 27 g each and the historical OreSat battery and solar modules [49, 75]. The complete mass estimates add assumed EPS contributions. The values below list battery, solar array, conversion and distribution contributions, in that order. The OreSat mass case uses 2016 hardware, separate from its later electrical baseline.

Case	Component masses (g)	Occupied volumes (cm^3)	Mass (g), volume (cm^3) allocations
BIRDS	[240, 18, 25, 17]	[80, 6, 20, 15]	[350, 150]
OreSat	[416, 252, 35, 25]	[160, 240, 30, 20]	[800, 500]

BIRDS battery mass includes 78 g of assumed packaging and EPS overhead above the 162 g cell sum. OreSat solar volume uses eight assumed $100 \times 100 \times 3$ mm complete module envelopes. References sum the contributions and subtract them from their allocations.

Case	Quantity and reference basis	Reference	Framework
BIRDS	Published cells (g) (P)	162	162
OreSat	Published battery modules (g) (P)	416	416
BIRDS	EPS mass (g) (I)	300	300
OreSat	EPS mass (g) (I)	728	728
BIRDS	EPS volume (cm^3) (I)	121	121
OreSat	EPS volume (cm^3) (I)	450	450
BIRDS	Mass margin (g) (I)	50	50
OreSat	Mass margin (g) (I)	72	72

D.2.2 FR-14: energy budgets and BOL/EOL sizing

MOP-14a covers power and orbital energy accounting. MOP-14b covers sizing and generation. References use $P = VI$, duty-weighted phase power, $E = \overline{P}T$ and $P_{req} = (E_s/0.85 + E_e/0.75)/T_s$. The assumed battery law is $N = 2000\,DoD^{-1}$ over $0.1 \leq DoD \leq 0.8$. One cycle per orbit is assumed, so cycle demand is mission duration divided by orbital period. This demand determines DoD, and required BOL energy is $E_e(1 + 0.2)/DoD$. Retained battery energy and Ah use separate assumed 0.8 factors. Source generation uses the selected age-specific curve once. Cell efficiency is $V_{mp}I_{mp}/(G_{ref}A_{cell})$. Required active area divides source-power demand by irradiance, efficiency, the two source factors and mean sunlight projection. Additional orbital checks include eccentric nadir cases and an eclipse-free case.

The eccentricities are 0.02 for BIRDS and 0.04 for OreSat, with $\beta = 25°$ and Sun projection angle 47°. The eclipse-free case uses $\beta = 90°$.

Case	Quantity and reference basis	Reference	Framework
BIRDS	Required source power (W) (I)	1.233913	1.233906
OreSat	Required source power (W) (I)	0.9635935	0.9635975
BIRDS	BOL mean source power (W) (I)	2.232769	2.232769
OreSat	BOL mean source power (W) (I)	2.163111	2.163111
BIRDS	EOL mean source power (W) (I)	2.006224	2.006224
OreSat	EOL mean source power (W) (I)	1.9468	1.9468
BIRDS	Power-based minimum cell count (I)	2	2
OreSat	Power-based minimum cell count (I)	1	1
BIRDS	Required battery energy (J) (I)	4243.97	4243.934
OreSat	Required battery energy (J) (I)	6062.541	6062.585
BIRDS	Required active area (m^2) (I)	0.003712395	0.003712375
OreSat	Required active area (m^2) (I)	0.002672799	0.00267281
BIRDS	Published regulator energy (mWh) (P)	1546	1546.4
OreSat	Published BOL module power (W) (P)	2.34	2.34738

The BIRDS published regulator row evaluates $(2173 - 240) \times 0.8$ mWh using supplied generation and stated losses [46, p. 8]. The OreSat published BOL module comparison uses its stated cell ratings [51]. Doubling the reference grid changes eclipse duration by at most 0.00205% and leaves integer sizing bounds unchanged.

D.2.3 FR-15: solar technology and battery chemistry

The silicon alternative is a 4S10P IXOLAR section with 4.46 V and 5.4 mA per cell at the selected EOL MPP [50]. Its EOL current retention of 0.9 is assumed. The BIRDS battery alternative is 1S2P Tenergy Li-ion, using 3.6 V and 2.6 Ah per cell. The OreSat alternative is 4S1P LiFePO4 per independent pack, using 3.3 V and 1.1 Ah per cell [52, 89]. References recompute $P_{array} = n_s n_p V_{mp} I_{mp}$ and $E_{pack} = n_s n_p V_{nom} C_{cell}$. Charger comparisons retain the selected chemistry, cell count, endpoint and termination method.

Case	Quantity and reference basis	Reference	Framework
BIRDS	Silicon array MPP power (W) (I)	0.96336	0.96336
OreSat	Silicon array MPP power (W) (I)	0.96336	0.96336
BIRDS	Alternative pack energy (Wh) (I)	18.72	18.72
OreSat	Alternative pack energy (Wh) (I)	14.52	14.52

D.2.4 FR-16: series and parallel arrangements

The array variation uses 4S1P in both cases. BIRDS storage changes to 2S1P, while each OreSat branch changes to 3S1P. References scale series voltage and energy, and parallel charge capacity and current ratings. The four-cell array uses a 2 by 2 grid on an assumed

100 × 170 mm plate. Package envelopes are 40.15 × 80.15 mm for BIRDS and 39.7 × 69.1 mm for OreSat [100, 102]. The grid adds 2 mm gaps, 3 mm edge clearances and 0.1 mm per-dimension allowances. The OreSat package envelope is assumed from the nominal cell outline.

Case	Quantity and reference basis	Reference	Framework
BIRDS	Four-cell MPP voltage (V) (I)	8.976	8.976
OreSat	Four-cell MPP voltage (V) (I)	9.72	9.72
BIRDS	Changed primary-pack energy (Wh) (I)	4.56	4.56
OreSat	Changed primary-pack energy (Wh) (I)	28.08	28.08
BIRDS	Changed primary-pack capacity (Ah) (I)	1.9	1.9
OreSat	Changed primary-pack capacity (Ah) (I)	2.6	2.6
BIRDS	Deployed footprint (cm^2) (I)	149.1225	149.1225
OreSat	Deployed footprint (cm^2) (I)	128.2464	128.2464
OreSat	Published baseline card energy (Wh) (P)	37.4	37.44

The last row concerns the baseline card's two independent 2S1P packs [53, p. 6]. Stored energies add. Current limits remain per branch without an assumed sharing law.

D.2.5 FR-17: bus rails and converter compatibility

The selected rails are 3.3 V at 0.5 A and 5 V at 0.4 A. Terminal battery current is $I_{bat} = P_{rail}/(0.9V_{bat})$ at nominal pack voltage. BIRDS uses TPS63020-Q1 for 3.3 V and LTC3119 for 5 V, with assumed complete-circuit efficiency 0.9 and 1 A output limit. Their entered input intervals are 1.8–5.5 V and 2.5–18 V. OreSat uses TPS62111/12 with a 3.1–17 V input interval, 1.5 A output limit and assumed efficiency 0.9 [214]. Voltage tolerance is an assumed 0.05 V. Input containment, output tolerance and current-limit comparisons matched their references.

Case	Quantity and reference basis	Reference	Framework
BIRDS	Battery current, 3.3 V rail (A) (I)	0.5092593	0.5092593
OreSat	Battery current, 3.3 V rail (A) (I)	0.2546296	0.2546296
BIRDS	Battery current, 5 V rail (A) (I)	0.617284	0.617284
OreSat	Battery current, 5 V rail (A) (I)	0.308642	0.308642

D.2.6 FR-18: PV characteristics and sampled MPP

The verification curves have short-circuit, MPP and open-circuit anchors from the baseline input table. The reference calculates each VI product and selects the largest sampled value.

Published checks use the BIRDS 5.123 V, 50 mA point [48, Figs. 13–14] and OreSat's 4.435 V, 0.37304 A row [55].

Case	Quantity and reference basis	Reference	Framework
BIRDS	Sampled cell MPP power (W) (I)	1.088564	1.088564
OreSat	Sampled cell MPP power (W) (I)	1.056321	1.056321
BIRDS	Published 50 mA power label (W) (P)	0.257	0.25615
OreSat	Published row 197 product (W) (P)	1.654432	1.654432

The BIRDS 1.575 W measured peak remains unreconstructed. The OreSat sweep lacks short/open-circuit endpoints and contains repeated voltages, so it supports power-product checks rather than a complete measured-curve benchmark.

D.2.7 FR-19: matched DET and ideal MPPT paths

The BIRDS pair uses the baseline 2S1P section and 3.6 V battery bus. The OreSat pair uses an explicitly unflown 4S1P section and the same 7.2 V battery bus. Four series cells provide a DET operating point at that bus voltage. Within each pair, both paths share the curve, illumination, source losses and 0.5 W bus demand. DET imposes $V_{op} = V_{bat} + 0.2$ V and obtains array current by linear interpolation. Its bus power is mean source power times V_{bat}/V_{op}. Ideal MPPT selects the sampled MPP and applies assumed conversion efficiency 0.9. The separate OreSat published net-power calculation subtracts 0.387 W of listed losses from its stated 2.10 W module power [51].

Case	Quantity and reference basis	Reference	Framework
BIRDS	DET mean bus power (W) (I)	1.617509	1.617509
BIRDS	MPPT mean bus power (W) (I)	1.805602	1.805602
OreSat	DET mean bus power (W) (I)	2.911227	2.911227
OreSat	MPPT mean bus power (W) (I)	3.504239	3.504239
OreSat	Separate published net power (W) (P)	1.71	1.713

D.2.8 FR-20: normal extrema and protection comparisons

The BIRDS voltage samples are 3.9, 4.0 and 4.2 V at pack terminals, with assumed UV/OV thresholds of 3.5/4.8 V. OreSat uses 3.0, 3.6 and 4.0 V cell samples against nominal 2.50/4.10 V thresholds [57]. Separate current bounds use $I_{min} = P_{min}/V_{max}$ and $I_{max} = P_{max}/V_{min}$. BIRDS uses 1–7.5 W at a 4.75–5.25 V regulated rail with a 2 A threshold [56, Sec. 6.3]. OreSat uses 6–30 W at the 6–8.4 V spacecraft battery bus with an assumed 6 A threshold. These are separate from the OreSat heater branch's 0.375 A detector.

Case	Quantity and reference basis	Reference	Framework
BIRDS	Normal voltage extrema (V) (I)	[3.9, 4.2]	[3.9, 4.2]

Case	Quantity and reference basis	Reference	Framework
OreSat	Normal voltage extrema (V) (I)	[3, 4]	[3, 4]
BIRDS	Power/voltage current bounds (A) (I)	[0.1904762, 1.578947]	[0.1904762, 1.578947]
OreSat	Power/voltage current bounds (A) (I)	[0.7142857, 5]	[0.7142857, 5]
OreSat	Published 30 W / 7.2 V current (A) (P)	4.17	4.166667

Normal-case no-trip decisions and the strict comparisons at each declared threshold matched their references. The separate assumed OreSat charger endpoint remains 4.20 V per cell against the nominal 4.10 V trip threshold.

Appendix E

Framework Models and Analysis Scripts

```
framework/
|-- models/
|   |-- __init__.py
|   |-- elements.py
|   |-- eps.py
|   |-- functions.py
|   |-- geometry.py
|   |-- mission.py
|   |-- modes.py
|   |-- objects.py
|   |-- properties.py
|   |-- recommendations.py
|   |-- requirements.py
|   |-- spacecraft.py
|   `-- stakeholders.py
`-- scripts/
    |-- illumination.py
    `-- sizing.py
```

Figure E.1: Model and analysis folder structure.

```python
"""Small, trusted Python model API. Start with Mission and Property."""

from .properties import EvaluationResult, EvaluationStatus, PendingValue, Property, Unit
from .objects import ModelObject
from .geometry import Face, Pointing
from .functions import Function, ModuleImport
from .stakeholders import Stakeholder, RegulatoryAgency, MissionStakeholder, LaunchProvider, Partner, ThirdParty
from .elements import DomainElement, SpaceSegment, GroundSegment, Subsystem, Component, Orbit
from .modes import SpacecraftMode, ComponentMode, ComponentState, TimeAllocation, OrbitAllocation, ModeAllocation, StateAllocation
from .eps import EPS, EPSComponent, SolarPowerPath, TransferMethod, Battery, SolarArray, RegulatorAndConverter, PowerCircuit, Protection, COTS
from .eps import CotsBattery, CotsSolarArray, CotsRegulatorAndConverter, CotsPowerCircuit, CotsProtection
from .spacecraft import CubeSatBus, GroundStation, CommandAndDataHandling, Communication
from .spacecraft import AttitudeDeterminationAndControl, GuidanceNavigationAndControl, StructuresAndMechanisms, Thermal, Propulsion, Payload
from .requirements import Requirement, VerificationCondition, ConditionResult, VerificationReport, InstanceVerification, VerificationFailure
from .mission import Mission, Session
from .recommendations import CotsRecommendation, recommend_cots

__all__ = [
    "Mission", "Session", "ModelObject", "Property", "Unit", "Function", "ModuleImport", "Face", "Pointing",
    "EvaluationResult", "EvaluationStatus", "PendingValue",
    "Stakeholder", "RegulatoryAgency", "MissionStakeholder", "LaunchProvider", "Partner", "ThirdParty",
    "DomainElement", "SpaceSegment", "GroundSegment", "Subsystem", "Component", "Orbit",
    "SpacecraftMode", "ComponentMode", "ComponentState", "TimeAllocation", "OrbitAllocation", "ModeAllocation", "StateAllocation",
    "EPS", "EPSComponent", "SolarPowerPath", "TransferMethod", "Battery", "SolarArray", "RegulatorAndConverter", "PowerCircuit", "Protection", "COTS",
    "CotsBattery", "CotsSolarArray", "CotsRegulatorAndConverter", "CotsPowerCircuit", "CotsProtection",
    "CubeSatBus", "GroundStation", "CommandAndDataHandling", "Communication",
    "AttitudeDeterminationAndControl", "GuidanceNavigationAndControl", "StructuresAndMechanisms", "Thermal", "Propulsion", "Payload",
    "Requirement", "VerificationCondition", "ConditionResult", "VerificationReport", "InstanceVerification", "VerificationFailure",
    "CotsRecommendation", "recommend_cots",
]
```

Figure E.2: `models/__init__.py`.

```
"""Domain containment and class-defined cumulative property patterns."""

from .modes import ComponentMode, SpacecraftMode
from .objects import ModelObject
from .properties import Property
from .requirements import Requirement


class DomainElement(ModelObject):
    @property
    def subsystems(self):
        return tuple(child for child in self.children if isinstance(child, Subsystem))

    def accepts(self, child):
        return isinstance(child, (Subsystem, Requirement))


class SpaceSegment(DomainElement):
    default_subsystems = ()
    cost = Property("sum(child.cost.value for child in self.subsystems)", "currency")
    mass = Property("sum(child.mass.value for child in self.subsystems)", "kg")
    volume = Property("sum(child.volume.value for child in self.subsystems)", "m3")
    power = Property("sum(child.power.value for child in self.subsystems)", "W")

    @property
    def modes(self):
        return tuple(child for child in self.children if isinstance(child, SpacecraftMode))

    def accepts(self, child):
        return super().accepts(child) or isinstance(child, SpacecraftMode)

    def create_defaults(self):
        for cls in self.default_subsystems:
            if not any(child.name == cls.__name__ for child in self.children):
                self.add(cls(cls.__name__))


class GroundSegment(DomainElement):
    pass


class Subsystem(ModelObject):
    default_components = ()
    cost = Property("sum(child.cost.value for child in self.components)", "currency")
    mass = Property("sum(child.mass.value for child in self.components)", "kg")
    volume = Property("sum(child.volume.value for child in self.components)", "m3")
    power = Property("sum(child.power.value for child in self.components)", "W")

```

Figure E.3: `models/elements.py`.

```
    @property
    def components(self):
        return tuple(child for child in self.children if isinstance(child, Component))

    @property
    def modes(self):
        return getattr(self.parent, "modes", ())

    def accepts(self, child):
        return isinstance(child, Requirement) or isinstance(child, Component) and (
            type(child) is Component
            or child.subsystem_type is not None and isinstance(self, child.subsystem_type)
        )

    def create_defaults(self):
        for cls in self.default_components:
            if not any(child.name == cls.__name__ for child in self.children):
                self.add(cls(cls.__name__))


class Component(ModelObject):
    subsystem_type = None
    cost = Property(unit="currency")
    mass = Property(unit="kg")
    volume = Property(unit="m3")
    power = Property(unit="W")

    @property
    def modes(self):
        return tuple(child for child in self.children if isinstance(child, ComponentMode))

    @property
    def spacecraft(self):
        parent = self.parent
        while parent is not None and not isinstance(parent, SpaceSegment):
            parent = parent.parent
        return parent

    @property
    def spacecraft_modes(self):
        return self.spacecraft.modes if self.spacecraft is not None else ()

    @property
    def cots(self):
        return tuple(child for child in self.children if isinstance(child, Component) and
    child.fixed_properties)

    def recommend_cots(self, candidates, *, sort_by=None, reverse=False, limit=None, mode=None):
```

Figure E.3: `models/elements.py` (continued).

```
        """Query compatible products against this component's owned requirements."""
        from .recommendations import recommend_cots

        return recommend_cots(self, candidates, sort_by=sort_by, reverse=reverse, limit=limit,
    mode=mode)

    def accepts(self, child):
        if isinstance(child, (ComponentMode, Requirement)):
            return True
        return isinstance(child, Component) and child.fixed_properties and isinstance(child,
    type(self))


class Orbit(ModelObject):
    altitude = Property(unit="km")
    beta_angle = Property(unit="deg")
    eccentricity = Property(unit="1")
    earth_radius = Property("6378", "km")
    gravitational_parameter = Property("398601", "km3/s2")
    semimajor_axis = Property("self.earth_radius.value + self.altitude.value", "km")
    sun_projection_angle = Property(unit="deg")
    effective_sun_projection_angle = Property(
        "0 if self.eccentricity.value == 0 or abs(self.beta_angle.value) == 90 else
    self.sun_projection_angle", "deg")
    perigee_altitude = Property("self.semimajor_axis.value * (1 - self.eccentricity.value) -
    self.earth_radius.value", "km")
    period = Property(
        "sizing.orbit_period(semimajor_axis=self.semimajor_axis, eccentricity=self.eccentricity, "
        "earth_radius=self.earth_radius,
    gravitational_parameter=self.gravitational_parameter).seconds", "s",
    )
    mean_motion = Property("2 * math.pi / self.period.value", "rad/s")
    orbits_per_day = Property("86400 / self.period.value", "orbit/day")
    eclipse_duration = Property(
        "sizing.circular_illumination(semimajor_axis=self.semimajor_axis,
    earth_radius=self.earth_radius, "
        "beta_angle=self.beta_angle, period=self.period).eclipse "
        "if self.eccentricity.value == 0 or abs(self.beta_angle.value) == 90 else "
        "sizing.eccentric_illumination(semimajor_axis=self.semimajor_axis,
    eccentricity=self.eccentricity, "
        "earth_radius=self.earth_radius, beta_angle=self.beta_angle, "
        "sun_projection_angle=self.sun_projection_angle, period=self.period).eclipse", "s",
    )
    sunlight_duration = Property("self.period.value - self.eclipse_duration.value", "s")

    def accepts(self, child):
        return isinstance(child, Requirement)
```

Figure E.3: `models/elements.py` (continued).

```python
"""EPS component types and fixed COTS specialisations."""

from enum import Enum

from .elements import Component, Subsystem
from .properties import Property
from .requirements import VerificationCondition


class TransferMethod(str, Enum):
    DET = "DET"
    MPPT = "MPPT"
    FIXED_VOLTAGE = "FIXED_VOLTAGE"


class EPS(Subsystem):
    power = Property("sum(child.power.value for child in self.power_consumers)", "W")
    available_sunlight_energy = Property(unit="J")
    available_eclipse_energy = Property(unit="J")

    def _sizing_requirement(self, name, available, required, unit, basis, statement, *, integer=False, maximum=False, **inputs):
        measure = "self.parent.available" if integer or maximum else "0 if self.parent.required.value == 0 else self.parent.available"
        comparison = "0 <= measure <= target" if maximum else "0 <= target <= measure"
        condition = f"math.isfinite(measure) and math.isfinite(target) and {comparison}"
        if integer:
            condition += " and type(measure) is int"
        return self.add_requirement(
            name,
            statement=statement,
            rationale=f"final_report/design-proposal.tex#eq:{basis}. Verification at the declared accounting boundary.",
            available=available, required=required if isinstance(required, Property) else Property(required, unit),
            conditions=(VerificationCondition("maximum" if maximum else "minimum", Property(measure, unit), Property("self.parent.required", unit), condition),),
            **inputs,
        )

    def _voltage_requirement(self, name, minimum, maximum, basis):
        return self.add_requirement(
            name,
            statement="The source voltage shall remain between {self.minimum_allowed} and {self.maximum_allowed}.",
            rationale=f"final_report/design-proposal.tex#eq:{basis}. Receiving limits are explicitly assigned to this path.",
            minimum_voltage=minimum, maximum_voltage=maximum,
            minimum_allowed=Property(unit="V"), maximum_allowed=Property(unit="V"),
            conditions=(VerificationCondition(
                "range", Property("(self.parent.minimum_voltage.value, self.parent.maximum_voltage.value)", "V"),
```

Figure E.4: `models/eps.py`.

```
45                Property("(self.parent.minimum_allowed.value, self.parent.maximum_allowed.value)",
    "V"),
46                "all(math.isfinite(v) for v in (*measure, *target)) and 0 <= target[0] <= measure[0]
    <= measure[1] <= target[1]",
47            ),),
48        )
49
50      def create_requirements(self):
51          energies = {}
52          for phase in ("sunlight", "eclipse"):
53              energies[phase] = self._sizing_requirement(
54                  f"{phase}_energy", getattr(self, f"available_{phase}_energy"),
55                  "0 if self.duration.value == 0 else
    sizing.load_energy(average_power=self.average_power, duration=self.duration).joules",
56                  "J", f"{phase}-load-energy",
57                  f"The EPS shall deliver at least {{minimum.target}} during {phase} per orbit.",
58                  average_power=Property(unit="W"),
59                  duration=Property(f"self.mission.orbit.{phase}_duration", "s"),
60              )
61          solar_power = self._sizing_requirement(
62              "solar_power", self.SolarArray.eol_power,
63              "sizing.minimum_panel_power(sunlight_energy=self.sunlight_energy, "
64              "eclipse_energy=self.eclipse_energy,
    sunlight_efficiency=self.effective_sunlight_efficiency, "
65              "eclipse_efficiency=self.effective_eclipse_efficiency,
    sunlight_duration=self.mission.orbit.sunlight_duration).watts",
66              "W", "minimum-panel-power",
67              "The solar array shall provide at least {minimum.target} at end of life.",
68              sunlight_energy=energies["sunlight"].required,
69              eclipse_energy=energies["eclipse"].required,
70              effective_sunlight_efficiency=Property(
71                  "1 if self.sunlight_energy.value == 0 else
    self.context.SolarArray.sunlight_path_efficiency", "1"),
72              effective_eclipse_efficiency=Property(
73                  "1 if self.eclipse_energy.value == 0 else
    self.context.SolarArray.eclipse_path_efficiency", "1"),
74          )
75          self._sizing_requirement(
76              "solar_cell_count", self.SolarArray.cell_count,
77              "0 if self.required_power.value == 0 else
    sizing.cell_count(required_power=self.required_power, eol_cell_power=self.cell_power).required",
78              "cell", "solar-cell-count",
79              "The solar array shall contain at least {minimum.target.value:d} cells.", integer=True,
80              required_power=solar_power.required,
81              cell_power=self.SolarArray.cell_eol_power,
82              fractional_count=Property(
83                  "0 if self.required_power.value == 0 else
    sizing.cell_count(required_power=self.required_power, eol_cell_power=self.cell_power).fractional",
    "cell"),
84              series_count=self.SolarArray.series_count,
85              minimum_parallel_count=Property(
86                  "0 if self.required_power.value == 0 else
    sizing.parallel_strings(required_total=self.required_power, "
```

Figure E.4: `models/eps.py` (continued).

```
                "per_cell=self.cell_power, series_count=self.series_count).required", "string"),
        )
        self._sizing_requirement(
            "battery_energy", self.Battery.bol_energy,
            "0 if self.eclipse_energy.value == 0 else "
            "sizing.battery_energy(eclipse_energy=self.eclipse_energy, "
            "depth_of_discharge=self.maximum_dod, degradation_margin=self.degradation_margin).joules",
            "J", "battery-energy",
            "The battery shall have at least {minimum.target} of beginning-of-life energy.",
            eclipse_energy=energies["eclipse"].required,
            required_cycles=Property(
                "sizing.mission_cycles(lifetime=self.mission.lifetime,
    days_per_year=self.mission.days_per_year, "
                "orbits_per_day=self.mission.orbit.orbits_per_day).cycles", "cycle"),
            maximum_dod=Property(
                "sizing.depth_of_discharge(required_cycles=self.required_cycles,
    cycle_coefficient=self.cycle_coefficient, "
                "cycle_exponent=self.cycle_exponent, fit_dod_min=self.fit_dod_min,
    fit_dod_max=self.fit_dod_max).fraction", "1"),
            cycle_coefficient=self.Battery.cycle_coefficient,
            cycle_exponent=self.Battery.cycle_exponent,
            fit_dod_min=self.Battery.fit_dod_min,
            fit_dod_max=self.Battery.fit_dod_max,
            degradation_margin=self.Battery.degradation_margin,
            cell_energy=self.Battery.cell_bol_energy,
            series_count=self.Battery.series_count,
            minimum_parallel_count=Property(
                "0 if self.required.value == 0 else
    sizing.parallel_strings(required_total=self.required, "
                "per_cell=self.cell_energy, series_count=self.series_count).required", "string"),
        )
        for name, component, basis in (
            ("solar_configuration", self.SolarArray, "pv-integer-configuration"),
            ("battery_configuration", self.Battery, "battery-assembly-capacity"),
        ):
            self.add_requirement(
                name,
                statement="The assembly shall contain {total.target.value:d} cells in positive
    integer series and parallel counts.",
                rationale=f"final_report/design-proposal.tex#eq:{basis}. Counts are supplied by the
    engineer.",
                series_count=component.series_count, parallel_count=component.parallel_count,
                conditions=(
                    VerificationCondition("series", component.series_count, Property(1, "cell"),
    "type(measure) is int and measure >= target"),
                    VerificationCondition("parallel", component.parallel_count, Property(1,
    "string"), "type(measure) is int and measure >= target"),
                    VerificationCondition("total", component.cell_count,
                                          Property("self.parent.series_count.value *
    self.parent.parallel_count.value", "cell"),
                                          "type(measure) is int and type(target) is int and measure
    == target"),
                ),
```

Figure E.4: `models/eps.py` (continued).

```
130             )
131
132         self._sizing_requirement(
133             "solar_area", self.SolarArray.active_area,
134             "0 if self.required_power.value == 0 else sizing.minimum_array_area("
135             "required_power=self.required_power,
    power_density=self.context.SolarArray.eol_power_density).square_metres",
136             "m2", "array-generation-screen",
137             "The array shall have at least {minimum.target} of active area at the assessed
    conditions.",
138             required_power=solar_power.required,
139             effective_area=Property(
140                 "0 if self.required_power.value == 0 else
    sizing.minimum_array_area(required_power=self.required_power, "
141                 "power_density=self.context.SolarArray.normal_power_density).square_metres", "m2"),
142             tumbling_panel_area=Property(
143                 "sizing.tumbling_array_area(effective_area=self.effective_area, "
144                 "packing_fraction=self.context.SolarArray.packing_fraction).square_metres", "m2"),
145         )
146         self._sizing_requirement(
147             "solar_area_fit", self.SolarArray.panel_area, self.SolarArray.usable_area,
148             "m2", "array-area-fit", "The installed panel area shall not exceed {maximum.target}.",
    maximum=True,
149         )
150         self._sizing_requirement(
151             "solar_generation", self.SolarArray.available_power, solar_power.required,
152             "W", "array-generation-screen",
153             "The array shall provide at least {minimum.target} as mean available power during
    sunlight.",
154         )
155         self._voltage_requirement("solar_voltage", self.SolarArray.operating_voltage,
    self.SolarArray.operating_voltage, "mppt-voltage-compatibility")
156         self._voltage_requirement("battery_voltage", self.Battery.minimum_voltage,
    self.Battery.maximum_voltage, "battery-assembly-ratings")
157         for name, available, unit, basis, statement, maximum in (
158             ("solar_open_circuit_voltage", self.SolarArray.open_circuit_voltage, "V",
    "pv-input-ratings",
159              "The array open-circuit voltage shall not exceed {maximum.target}.", True),
160             ("solar_short_circuit_current", self.SolarArray.short_circuit_current, "A",
    "pv-input-ratings",
161              "The array short-circuit current shall not exceed {maximum.target}.", True),
162             ("bus_current", Property(unit="A"), "A", "battery-assembly-ratings",
163              "The selected bus path shall supply at least {minimum.target}.", False),
164             ("battery_charge_current", self.Battery.maximum_charge_current, "A",
    "battery-assembly-ratings",
165              "The battery shall accept at least {minimum.target} of charging current.", False),
166             ("battery_discharge_current", self.Battery.maximum_discharge_current, "A",
    "battery-assembly-ratings",
167              "The battery shall supply at least {minimum.target} of discharge current.", False),
168         ):
169             self._sizing_requirement(name, available, None, unit, basis, statement, maximum=maximum)
170         self.add_requirement(
171             "bus_voltage",
```

Figure E.4: `models/eps.py` (continued).

```
            statement="The selected bus path shall provide {match.target} within {self.tolerance}.",
            rationale="The engineer assigns the source and load state on the same rail. This checks their nominal voltage match.",
            available=Property(unit="V"), required=Property(unit="V"), tolerance=Property(0, "V"),
            conditions=(VerificationCondition(
                "match", Property("self.parent.available", "V"), Property("self.parent.required", "V"),
                "all(math.isfinite(v) for v in (measure, target, self.parent.tolerance.value)) "
                "and measure >= 0 and target >= 0 and 0 <= self.parent.tolerance.value "
                "and abs(measure - target) <= self.parent.tolerance.value",
            ),),
        )

    @property
    def power_consumers(self):
        # Storage and generation are excluded from this default demand boundary.
        # A specialised EPS can choose a different boundary, including losses.
        return tuple(child for child in self.components if not isinstance(child, (Battery, SolarArray)))


class EPSComponent(Component):
    subsystem_type = EPS


class SolarPowerPath(EPSComponent):
    """Explicit solar connection, including a converter or a direct path."""

    transfer_method = Property(unit="1")
    selected_transfer_method = Property("sizing.transfer_selection(transfer_method=self.transfer_method).method", "1")
    bus_voltage = Property(unit="V")
    voltage_drop = Property(unit="V")
    input_voltage_setpoint = Property(unit="V")


class Battery(EPSComponent):
    series_count = Property(unit="cell")
    parallel_count = Property(unit="string")
    cell_count = Property("self.series_count.value * self.parallel_count.value", "cell")
    cell_rated_capacity = Property(unit="Ah")
    cell_rated_energy = Property(unit="Wh")
    cell_nominal_voltage = Property(unit="V")
    cell_minimum_voltage = Property(unit="V")
    cell_maximum_voltage = Property(unit="V")
    cell_maximum_charge_current = Property(unit="A")
    cell_maximum_discharge_current = Property(unit="A")
    cell_bol_energy = Property("self.cell_rated_energy.value * 3600", "J")
    rated_capacity = Property("self.parallel_count.value * self.cell_rated_capacity.value", "Ah")
    rated_energy = Property("self.cell_count.value * self.cell_rated_energy.value", "Wh")
    nominal_voltage = Property("self.series_count.value * self.cell_nominal_voltage.value", "V")
    minimum_voltage = Property("self.series_count.value * self.cell_minimum_voltage.value", "V")
    maximum_voltage = Property("self.series_count.value * self.cell_maximum_voltage.value", "V")
```

Figure E.4: `models/eps.py` (continued).

```
221     maximum_charge_current = Property("self.parallel_count.value *
    self.cell_maximum_charge_current.value", "A")
222     maximum_discharge_current = Property("self.parallel_count.value *
    self.cell_maximum_discharge_current.value", "A")
223     cycle_coefficient = Property(unit="cycle")
224     cycle_exponent = Property(unit="1")
225     fit_dod_min = Property(unit="1")
226     fit_dod_max = Property(unit="1")
227     degradation_margin = Property(unit="1")
228     energy_retention = Property(unit="1")
229     capacity_retention = Property(unit="1")
230     bol_energy = Property("self.rated_energy.value * 3600", "J")
231     eol_energy = Property("sizing.retained_value(initial=self.bol_energy,
    retention=self.energy_retention).retained", "J")
232     eol_capacity = Property("sizing.retained_value(initial=self.rated_capacity,
    retention=self.capacity_retention).retained", "Ah")
233
234
235 class SolarArray(EPSComponent):
236     maximum_power = Property(unit="W")
237     cell_iv_curve = Property("()")
238     pv_reference_irradiance = Property(unit="W/m2")
239     power_path = Property()
240     iv_interpolation = Property("'none'", "1")
241     operating_voltage = Property(
242
    "sizing.solar_operating_voltage(transfer_method=self.power_path.value.selected_transfer_method, "
243         "**({'reference_voltage': self.power_path.value.bus_voltage, 'voltage_drop':
    self.power_path.value.voltage_drop} "
244         "if self.power_path.value.selected_transfer_method.value == 'DET' else "
245         "{'reference_voltage': self.eol_voltage if
    self.power_path.value.selected_transfer_method.value == 'MPPT' "
246         "else self.power_path.value.input_voltage_setpoint})).volts", "V")
247     operating_current = Property(
248         "self.eol_current if self.operating_voltage.value == self.eol_voltage.value else "
249         "sizing.pv_operating_point(pv_points=self.cell_iv_curve, series_count=self.series_count, "
250         "parallel_count=self.parallel_count, operating_voltage=self.operating_voltage,
    interpolation=self.iv_interpolation).current", "A")
251     operating_power = Property("self.operating_voltage.value * self.operating_current.value", "W")
252     series_count = Property(unit="cell")
253     parallel_count = Property(unit="string")
254     cell_count = Property("self.series_count.value * self.parallel_count.value", "cell")
255     cell_active_area = Property(unit="m2")
256     active_area = Property("self.cell_count.value * self.cell_active_area.value", "m2")
257     mounting_face = Property(unit="1")
258     normal_body = Property("illumination.face_normal(face=self.mounting_face).vector", "1")
259     body_face_area = Property(
260         "illumination.face_area(face=self.mounting_face, length_x=self.spacecraft.length_x, "
261         "length_y=self.spacecraft.length_y, length_z=self.spacecraft.length_z).square_metres", "m2")
262     packing_fraction = Property(unit="1")
263     usable_area = Property(unit="m2")
264     panel_area = Property("sizing.panel_area(active_area=self.active_area,
    packing_fraction=self.packing_fraction).square_metres", "m2")
```

Figure E.4: `models/eps.py` (continued).

```
265     cell_open_circuit_voltage = Property(unit="V")
266     cell_short_circuit_current = Property(unit="A")
267     open_circuit_voltage = Property(
268         "self.series_count.value *
    (sizing.pv_characteristics(pv_points=self.cell_iv_curve).open_circuit_voltage "
269         "if self.cell_iv_curve.value else self.cell_open_circuit_voltage.value)", "V")
270     short_circuit_current = Property(
271         "self.parallel_count.value *
    (sizing.pv_characteristics(pv_points=self.cell_iv_curve).short_circuit_current "
272         "if self.cell_iv_curve.value else self.cell_short_circuit_current.value)", "A")
273     sunlight_path_efficiency = Property(unit="1")
274     eclipse_path_efficiency = Property(unit="1")
275     cell_eol_voltage = Property(unit="V")
276     cell_eol_current = Property(unit="A")
277     eol_voltage = Property(
278         "self.series_count.value *
    (sizing.pv_characteristics(pv_points=self.cell_iv_curve).mpp_voltage "
279         "if self.cell_iv_curve.value else self.cell_eol_voltage.value)", "V")
280     eol_current = Property(
281         "self.parallel_count.value *
    (sizing.pv_characteristics(pv_points=self.cell_iv_curve).mpp_current "
282         "if self.cell_iv_curve.value else self.cell_eol_current.value)", "A")
283     cell_eol_power = Property(
284         "sizing.cell_power(pv_points=self.cell_iv_curve).watts if self.cell_iv_curve.value else "
285         "sizing.cell_power(eol_voltage=self.cell_eol_voltage,
    eol_current=self.cell_eol_current).watts", "W")
286     eol_power = Property("self.cell_count.value * self.cell_eol_power.value", "W")
287     bol_efficiency = Property(unit="1")
288     lifetime_retention = Property(unit="1")
289     temperature_factor = Property(unit="1")
290     eol_efficiency = Property(
291         "sizing.solar_efficiency(bol_efficiency=self.bol_efficiency,
    lifetime_retention=self.lifetime_retention, "
292         "temperature_factor=self.temperature_factor).fraction", "1")
293     irradiance = Property(unit="W/m2")
294     inherent_retention = Property(unit="1")
295     assembly_efficiency = Property(unit="1")
296     sunlight_projection = Property(
297         "illumination.orbital_projection(normal_body=self.normal_body, "
298         "pointing=self.spacecraft.AttitudeDeterminationAndControl.pointing, "
299         "pointing_face=self.spacecraft.AttitudeDeterminationAndControl.pointing_face, "
300         "clocking_angle=self.spacecraft.AttitudeDeterminationAndControl.effective_clocking_angle, "
301         "semimajor_axis=self.mission.orbit.semimajor_axis,
    eccentricity=self.mission.orbit.eccentricity, "
302         "earth_radius=self.mission.orbit.earth_radius, beta_angle=self.mission.orbit.beta_angle, "
303         "sun_projection_angle=self.mission.orbit.effective_sun_projection_angle, "
304         "period=self.mission.orbit.period).sunlight_mean", "1")
305     projected_sunlight_duration = Property("self.sunlight_projection.value *
    self.mission.orbit.sunlight_duration.value", "s")
306     generation_efficiency = Property(
307         "sizing.pv_efficiency(pv_points=self.cell_iv_curve, active_area=self.cell_active_area, "
308         "reference_irradiance=self.pv_reference_irradiance).fraction "
309         "if self.cell_iv_curve.value else self.eol_efficiency", "1")
```

Figure E.4: `models/eps.py` (continued).

```
    operating_efficiency = Property("self.generation_efficiency.value * self.operating_power.value / self.eol_power.value", "1")
    normal_power_density = Property("self.irradiance.value * self.operating_efficiency.value", "W/m2")
    eol_power_density = Property(
        "sizing.array_power_density(irradiance=self.irradiance, eol_efficiency=self.operating_efficiency, "
        "inherent_retention=self.inherent_retention, assembly_efficiency=self.assembly_efficiency, "
        "sunlight_projection=self.sunlight_projection).watts_per_square_metre", "W/m2")
    available_power = Property("self.active_area.value * self.eol_power_density.value", "W")
    source_energy = Property("self.available_power.value * self.mission.orbit.sunlight_duration.value", "J")


class RegulatorAndConverter(SolarPowerPath):
    input_voltage = Property(unit="V")
    minimum_input_voltage = Property(unit="V")
    maximum_input_voltage = Property(unit="V")
    absolute_maximum_input_voltage = Property(unit="V")
    maximum_input_current = Property(unit="A")
    output_voltage = Property(unit="V")
    maximum_output_current = Property(unit="A")
    efficiency = Property(unit="1")


class PowerCircuit(SolarPowerPath):
    maximum_voltage = Property(unit="V")
    maximum_current = Property(unit="A")


class Protection(PowerCircuit):
    """Normal operation screening at explicitly assigned sensing points."""

    trip_parameters = ("undervoltage_threshold", "overvoltage_threshold", "current_limit")
    undervoltage_threshold = Property(unit="V")  # Highest applicable UV trip threshold.
    overvoltage_threshold = Property(unit="V")  # Lowest applicable OV trip threshold.
    current_limit = Property(unit="A")          # Lowest applicable OC trip threshold.
    voltage_extrema = Property("sizing.extrema(values=self.no_trip.voltages)", "V")
    current_extrema = Property("sizing.extrema(values=self.no_trip.currents)", "A")

    def create_requirements(self):
        checks = {
            "undervoltage_threshold": ("undervoltage", "voltage_extrema", "minimum", "0 <= target < measure"),
            "overvoltage_threshold": ("overvoltage", "voltage_extrema", "maximum", "0 <= measure < target"),
            "current_limit": ("overcurrent", "current_extrema", "maximum",
                              "0 <= self.parent.context.current_extrema.value.minimum <= measure < target"),
        }
        if not self.trip_parameters or len(set(self.trip_parameters)) != len(self.trip_parameters):
            raise ValueError("Declare a nonempty set of distinct protection trip parameters.")
        conditions = []
        for parameter in self.trip_parameters:
```

Figure E.4: `models/eps.py` (continued).

```
            if parameter not in checks:
                raise ValueError(f"Unknown protection trip parameter {parameter!r}.")
            name, field, bound, comparison = checks[parameter]
            conditions.append(VerificationCondition(
                name, Property(f"self.parent.context.{field}.value.{bound}",
                               "A" if field == "current_extrema" else "V"),
                getattr(self, parameter),
                f"math.isfinite(measure) and math.isfinite(target) and {comparison}",
            ))
        self.add_requirement(
            "no_trip",
            statement="The protection shall remain inactive for the declared normal operating samples.",
            rationale="framework/scripts/sizing.py#extrema. Use the highest UV threshold and lowest OV/OC thresholds "
                      "for the selected COTS configuration and conditions. Equality fails this conservative screen. "
                      "Voltage samples refer to the detector's sensed node or individual cells. "
                      "Current samples are magnitudes for the specified protected direction. "
                      "This screen assumes an enabled, recovered device and establishes no fault response or recovery claim.",
            voltages=Property(unit="V"), currents=Property(unit="A"), conditions=tuple(conditions),
        )


class COTS(EPSComponent):
    """Product classes fix their inherited Property definitions."""

    fixed_properties = True


class CotsBattery(Battery, COTS):
    pass


class CotsSolarArray(SolarArray, COTS):
    pass


class CotsRegulatorAndConverter(RegulatorAndConverter, COTS):
    pass


class CotsPowerCircuit(PowerCircuit, COTS):
    pass


class CotsProtection(Protection, COTS):
    pass


EPS.default_components = (Battery, SolarArray, RegulatorAndConverter, PowerCircuit)
```

Figure E.4: `models/eps.py` (continued).

```
"""Explicit session imports and Property bindings for custom functions."""

import ast
import importlib
import inspect
from types import ModuleType
from typing import get_args, get_type_hints

from .properties import Property


class Function:
    """A named, annotated Python function returning an annotated NamedTuple."""

    def __init__(self, function):
        self.function = function
        self.signature = inspect.signature(function)
        self.annotations = get_type_hints(function)
        for name, parameter in self.signature.parameters.items():
            if parameter.kind not in (parameter.POSITIONAL_OR_KEYWORD, parameter.KEYWORD_ONLY):
                raise TypeError("Function inputs must have individually bindable names.")
            if name not in self.annotations:
                raise TypeError("Every Function input needs an annotation.")
            if type(None) in get_args(self.annotations[name]) and parameter.default is
  parameter.empty:
                raise TypeError("A nullable Function input needs an explicit default.")
        result = self.annotations.get("return")
        if not isinstance(result, type) or not issubclass(result, tuple) or not hasattr(result,
  "_fields"):
            raise TypeError("A Function returns an annotated NamedTuple.")
        fields = get_type_hints(result)
        if not result._fields or set(fields) != set(result._fields):
            raise TypeError("Every Function output needs a named, annotated field.")
        if any(type(None) in get_args(annotation) for annotation in fields.values()):
            raise TypeError("Function output annotations are nonoptional.")
        self.output_type = result

    @property
    def inputs(self):
        return {name: self.annotations[name] for name in self.signature.parameters}

    @property
    def outputs(self):
        return get_type_hints(self.output_type)

```

Figure E.5: `models/functions.py`.

```
    def __call__(self, *args, **kwargs):
        bound = self.signature.bind(*args, **kwargs)
        if any(not isinstance(prop, Property) for prop in bound.arguments.values()):
            raise TypeError("Every supplied Function input must be explicitly bound to a Property.")
        values = {name: prop.value for name, prop in bound.arguments.items()}
        result = self.function(**values)
        if not isinstance(result, self.output_type):
            raise TypeError(f"The Function must return {self.output_type.__name__}.")
        return result


class ModuleImport:
    """A session's `import package as alias` and its callable interfaces."""

    def __init__(self, source):
        tree = ast.parse(source)
        if len(tree.body) != 1 or not isinstance(tree.body[0], ast.Import):
            raise ValueError("Use import package as alias.")
        names = tree.body[0].names
        if len(names) != 1 or names[0].asname is None:
            raise ValueError("Use one explicit import package as alias.")
        self.package, self.alias = names[0].name, names[0].asname
        self.module = importlib.import_module(self.package)

    @property
    def source(self):
        return f"import {self.package} as {self.alias}"

    @property
    def namespace(self):
        return _ModuleView(self.module)


class _ModuleView:
    """Keep session import metadata separate from package attribute names."""

    def __init__(self, module):
        self._module = module

    def __getattr__(self, name):
        value = getattr(self._module, name)
        if inspect.isfunction(value) or inspect.isbuiltin(value):
            return value if self._module.__name__ == "math" else Function(value)
        return _ModuleView(value) if isinstance(value, ModuleType) else value
```

Figure E.5: `models/functions.py` (continued).

```python
"""Body face identifiers and prescribed pointing choices."""

from enum import Enum


class Face(str, Enum):
    POS_X = "+X"
    NEG_X = "-X"
    POS_Y = "+Y"
    NEG_Y = "-Y"
    POS_Z = "+Z"
    NEG_Z = "-Z"

    @property
    def normal(self):
        sign = 1 if self.value[0] == "+" else -1
        return tuple(sign if axis == self.value[1] else 0 for axis in "XYZ")


class Pointing(str, Enum):
    SUN_POINTING = "sun_pointing"
    NADIR = "nadir"
```

Figure E.6: `models/geometry.py`.

```
"""One configuration, its imports, orbit, domain elements and requirements."""

from collections import Counter
from types import MappingProxyType

from .elements import DomainElement, Orbit
from .functions import ModuleImport
from .objects import ModelObject
from .requirements import Requirement, VerificationReport
from .properties import Property
from .stakeholders import Stakeholder, ThirdParty

_DEFAULT_ORBIT = object()


class Mission(ModelObject):
    is_mission = True
    child_aliases = {"orbit": Orbit}
    lifetime = Property(unit="year")
    days_per_year = Property(unit="day/year")

    def __init__(self, name="mission", *, orbit=_DEFAULT_ORBIT, imports=(), **bindings):
        super().__init__(name, **bindings)
        self._imports = {}
        self.add_import("import math as math")
        self.add_import("import framework.scripts.sizing as sizing")
        self.add_import("import framework.scripts.illumination as illumination")
        for source in imports:
            self.add_import(source)
        if orbit is _DEFAULT_ORBIT:
            orbit = Orbit("orbit")
        if orbit is not None:
            self.add(orbit)
        self._initialized = True

    @property
    def imports(self):
        return MappingProxyType(self._imports)

    def add_import(self, source):
        imported = ModuleImport(source)
        if imported.alias in (self.name, "self", "mode"):
            raise ValueError("The import alias conflicts with the expression namespace.")
        old = self._imports.get(imported.alias)
        if old is not None:
            if old.package != imported.package:
                raise ValueError(f"Import alias {imported.alias!r} is already used.")
            return old
        if any(child.name == imported.alias for child in self.children):
            raise ValueError("An import alias cannot replace a Mission child.")
        self._imports[imported.alias] = imported
        return imported

    @property
    def namespace(self):
        objects = tuple(self.walk())
        counts = Counter(item.name for item in objects)
        namespace = {name: imported.namespace for name, imported in self.imports.items()}
```

Figure E.7: `models/mission.py`.

```
        namespace[self.name] = self
        for item in objects:
            if counts[item.name] == 1 and item.name not in ("self", "mode"):
                namespace.setdefault(item.name, item)
        return namespace

    @property
    def orbit(self):
        return next((child for child in self.children if isinstance(child, Orbit)), None)

    @property
    def elements(self):
        return tuple(child for child in self.children if isinstance(child, DomainElement))

    @property
    def stakeholders(self):
        return tuple(child for child in self.children if isinstance(child, Stakeholder))

    @property
    def third_parties(self):
        return tuple(child for child in self.children if isinstance(child, ThirdParty))

    def accepts(self, child):
        if child.name in (*self.imports, self.name, "self", "mode"):
            raise ValueError("The child name conflicts with the expression namespace.")
        if isinstance(child, Orbit):
            return self.orbit is None
        return isinstance(child, (DomainElement, Stakeholder, ThirdParty, Requirement))

    def get(self, path):
        parts = path.split(".")
        if parts[0] == self.name:
            current = self
        else:
            matches = [item for item in self.walk() if item.name == parts[0]]
            if len(matches) != 1:
                raise KeyError(f"{parts[0]!r} is missing or ambiguous. Use its full path.")
            current = matches[0]
        for name in parts[1:]:
            current = current.properties[name] if name in current.properties else current.child(name)
        return current

    def verify(self, *, mode=None):
        results = tuple(
            condition.evaluate(mode=mode)
            for requirement in self.walk() if isinstance(requirement, Requirement)
            for condition in requirement.conditions
        )
        return VerificationReport(self, results)

    @classmethod
    def restore(cls, name="mission", *, orbit=None, **configuration):
        return cls(name, orbit=orbit, **configuration)


Session = Mission
```

Figure E.7: `models/mission.py` (continued).

```
"""Distinct modes, electrical states and explicit allocation records.

Allocation normalization, averaging and chronology are intentionally unspecified.
"""

from .objects import ModelObject
from .properties import Property


class TimeAllocation(ModelObject):
    fraction = Property(unit="1")

    def __init__(self, name, fraction=None):
        super().__init__(name, fraction=fraction)


class OrbitAllocation(TimeAllocation):
    def validate_attachment(self):
        if not isinstance(self.parent, SpacecraftMode):
            raise TypeError("An OrbitAllocation belongs to a SpacecraftMode.")


class ModeAllocation(TimeAllocation):
    def __init__(self, name, spacecraft_mode, fraction=None):
        if not isinstance(spacecraft_mode, SpacecraftMode):
            raise TypeError("A ModeAllocation references a SpacecraftMode.")
        super().__init__(name, fraction)
        self._spacecraft_mode = spacecraft_mode

    @property
    def spacecraft_mode(self):
        return self._spacecraft_mode

    def validate_attachment(self):
        if not isinstance(self.parent, ComponentMode):
            raise TypeError("A ModeAllocation belongs to a ComponentMode.")
        if self.mission is not None:
            component = self.parent.parent
            if self.spacecraft_mode not in component.spacecraft_modes:
                raise ValueError("Allocate a mode from the component's own Space Segment.")


class StateAllocation(TimeAllocation):
    def __init__(self, name, state, fraction=None):
        if not isinstance(state, ComponentState):
            raise TypeError("A StateAllocation references a ComponentState.")
        super().__init__(name, fraction)
        self._state = state

    @property
```

Figure E.8: `models/modes.py`.

```
    def state(self):
        return self._state

    def validate_attachment(self):
        if not isinstance(self.parent, ComponentMode) or self.state.parent is not self.parent:
            raise ValueError("Allocate a state from this ComponentMode.")


class SpacecraftMode(ModelObject):
    child_aliases = {"allocation": OrbitAllocation}

    def accepts(self, child):
        return isinstance(child, OrbitAllocation) and self.allocation is None

    @property
    def allocation(self):
        return next((child for child in self.children if isinstance(child, OrbitAllocation)), None)

    def create_defaults(self):
        if self.allocation is None:
            self.add(OrbitAllocation("allocation"))


class ComponentState(ModelObject):
    voltage = Property(unit="V")
    current = Property(unit="A")
    power = Property("self.voltage.value * self.current.value", "W")


class ComponentMode(ModelObject):
    @property
    def states(self):
        return tuple(child for child in self.children if isinstance(child, ComponentState))

    @property
    def allocations(self):
        return tuple(child for child in self.children if isinstance(child, ModeAllocation))

    @property
    def state_allocations(self):
        return tuple(child for child in self.children if isinstance(child, StateAllocation))

    def accepts(self, child):
        return isinstance(child, (ComponentState, ModeAllocation, StateAllocation))

    def create_defaults(self):
        for name in ("on", "off"):
            if not any(child.name == name for child in self.children):
                self.add(ComponentState(name))
```

Figure E.8: `models/modes.py` (continued).

```
"""Named objects, owned properties and explicit containment lifecycle."""

import keyword
from types import MappingProxyType

from .properties import Property


def identifier(name):
    if not isinstance(name, str) or not name.isidentifier() or keyword.iskeyword(name):
        raise ValueError("Object and property names must be Python identifiers.")
    return name


class ModelObject:
    fixed_properties = False
    is_mission = False
    child_aliases = {}

    def __init__(self, name, **bindings):
        self._name = identifier(name)
        self._parent = None
        self._children = []
        self._initialized = False
        self._requirements_initialized = False
        definitions = {}
        for cls in reversed(type(self).__mro__):
            definitions.update({key: value for key, value in vars(cls).items() if isinstance(value,
    Property)})
        self._properties = {
            key: value.copy(self, key, frozen=self.fixed_properties)
            for key, value in definitions.items()
        }
        if self.fixed_properties and bindings:
            raise TypeError("Define COTS property values on the product class.")
        for key, value in bindings.items():
            if key in self._properties:
                self._set_property(key, value)
            elif isinstance(value, Property):
                self.define(key, value)
            else:
                raise TypeError(f"Unknown property {key!r}. Declare it with Property.")

    def __getattr__(self, name):
        properties = self.__dict__.get("_properties", {})
        if name in properties:
            return properties[name]
        for child in self.__dict__.get("_children", ()):
```

Figure E.9: `models/objects.py`.

```
48              if child.name == name:
49                  return child
50          raise AttributeError(f"{type(self).__name__} has no attribute {name!r}.")
51
52      def __setattr__(self, name, value):
53          if name in self.__dict__.get("_properties", {}):
54              self._set_property(name, value)
55          else:
56              object.__setattr__(self, name, value)
57
58      def _set_property(self, name, value):
59          current = self._properties[name]
60          current._editable()
61          if value is current:
62              return
63          if isinstance(value, Property):
64              source = Property.link(value) if value.owner is not None else value
65              current.expression = source.expression
66              current.unit = source.unit
67              current._modes = dict(source._modes)
68              current._source = source._source
69          else:
70              current.expression = value
71
72      def define(self, name, prop):
73          identifier(name)
74          if self.fixed_properties:
75              raise AttributeError("COTS properties are fixed by the product class.")
76          if name in self.__dict__ or hasattr(type(self), name) or name in self._properties or
    any(child.name == name for child in self.children):
77              raise ValueError(f"{name!r} is already defined.")
78          if not isinstance(prop, Property):
79              raise TypeError("Declare an additional property with Property.")
80          source = Property.link(prop) if prop.owner is not None else prop
81          self._properties[name] = source.copy(self, name)
82          return self._properties[name]
83
84      @property
85      def name(self):
86          return self._name
87
88      @property
89      def parent(self):
90          return self._parent
91
92      @property
93      def path(self):
94          return f"{self.parent.path}.{self.name}" if self.parent is not None else self.name
```

Figure E.9: `models/objects.py` (continued).

```
95
96      @property
97      def mission(self):
98          if self.is_mission:
99              return self
100         return self.parent.mission if self.parent is not None else None
101
102     @property
103     def properties(self):
104         return MappingProxyType(self._properties)
105
106     @property
107     def children(self):
108         return tuple(self._children)
109
110     @property
111     def requirements(self):
112         from .requirements import Requirement
113
114         return tuple(child for child in self.children if isinstance(child, Requirement))
115
116     def add_requirement(self, name, **definition):
117         """Create an owned default once, preserving authored replacements."""
118         from .requirements import Requirement
119
120         identifier(name)
121         existing = next((child for child in self.children if child.name == name), None)
122         if existing is not None:
123             if not isinstance(existing, Requirement):
124                 raise ValueError(f"Generated requirement name {name!r} conflicts with another
    object.")
125             return existing
126         definition.setdefault("context", self)
127         return self.add(Requirement(name, **definition))
128
129     def walk(self):
130         yield self
131         for child in self.children:
132             yield from child.walk()
133
134     def child(self, name):
135         for child in self.children:
136             if child.name == name:
137                 return child
138         raise KeyError(f"{self.path} has no child {name!r}.")
139
140     def accepts(self, child):
141         return False
```

Figure E.9: `models/objects.py` (continued).

```
142
143     def validate_attachment(self):
144         """Subclasses validate links after joining a parent."""
145
146     def add(self, child, *, initialize=True):
147         if isinstance(child, ModelObject) and child.parent is self:
148             return child
149         if not isinstance(child, ModelObject) or not self.accepts(child):
150             raise TypeError(f"{type(self).__name__} cannot contain {type(child).__name__}.")
151         if child.parent is not None:
152             raise ValueError("An object has one parent. Reference its properties from other objects.")
153         if self in child.walk():
154             raise ValueError("Containment cannot contain a cycle.")
155         if any(existing.name == child.name for existing in self.children):
156             raise ValueError(f"Duplicate child name {child.name!r} in {self.path}.")
157         alias_type = self.child_aliases.get(child.name)
158         if child.name in self.__dict__ or hasattr(type(self), child.name) or child.name in
    self.properties:
159             if alias_type is None or not isinstance(child, alias_type):
160                 raise ValueError(f"Child name {child.name!r} conflicts with an existing attribute.")
161         roots = (self.mission or self, child)
162         requirements_before = {req for root in roots for item in root.walk() for req in
    item.requirements}
163         states_before = {item: item._requirements_initialized for item in child.walk()}
164         child._parent = self
165         self._children.append(child)
166         try:
167             for item in child.walk():
168                 item.validate_attachment()
169                 for prop in item.properties.values():
170                     if item.mission is not None and any(mode.mission is not item.mission for mode in
    prop.modes):
171                         raise ValueError("Property bindings must use modes from this Mission.")
172             if initialize:
173                 child.initialize()
174             else:
175                 child._initialized = True
176                 for item in child.walk():
177                     item._requirements_initialized = True
178         except Exception:
179             self._children.remove(child)
180             child._parent = None
181             for root in roots:
182                 for item in root.walk():
183                     for requirement in item.requirements:
184                         if requirement not in requirements_before:
185                             item.remove(requirement)
186             for item in child.walk():
```

Figure E.9: `models/objects.py` (continued).

```
                item._requirements_initialized = states_before.get(item, False)
            raise
        return child

    def remove(self, child):
        child = self.child(child) if isinstance(child, str) else child
        self._children.remove(child)
        child._parent = None
        return child

    def initialize(self):
        if not self._initialized:
            self.create_defaults()
            self._initialized = True
        if self.mission is not None:
            for item in self.walk():
                if not item._requirements_initialized:
                    item.create_requirements()
                    item._requirements_initialized = True
        return self

    def create_defaults(self):
        """Create declared physical defaults on an explicit first add."""

    def create_requirements(self):
        """Add owned default requirements after joining a Mission."""

    @classmethod
    def restore(cls, name, **bindings):
        value = cls(name, **bindings)
        value._initialized = True
        value._requirements_initialized = True
        return value

    @property
    def verification(self):
        from .requirements import InstanceVerification

        return self.mission.verify().for_instance(self) if self.mission else
InstanceVerification(self, ())

    @property
    def verified(self):
        return self.verification.verified

    def __repr__(self):
        return f"{type(self).__name__}({self.name!r})"
```

Figure E.9: `models/objects.py` (continued).

```
"""Expression bindings, descriptive units and evaluation without a value cache."""

from contextvars import ContextVar
from dataclasses import dataclass
from enum import Enum
import math
from types import MappingProxyType


@dataclass(frozen=True)
class Unit:
    symbol: str = ""

    def __str__(self):
        return self.symbol


class EvaluationStatus(str, Enum):
    VALUE = "value"
    PENDING = "pending"
    ERROR = "error"


@dataclass(frozen=True)
class EvaluationResult:
    status: EvaluationStatus
    value: object = None
    message: str = ""


class PendingValue(ValueError):
    """A referenced property has no expression yet."""


_ACTIVE = ContextVar("model_evaluation", default=None)


def expression(source):
    """Accept one Python expression, including literal input conveniences."""
    if source is None:
        return None
    if isinstance(source, Enum):
        source = repr(source.value)
    elif type(source) in (bool, int, float):
        source = repr(source)
    if not isinstance(source, str) or "\n" in source or "\r" in source:
        raise ValueError("Supply a Python one-line expression.")
    compile(source, "<property>", "eval")
```

Figure E.10: `models/properties.py`.

```
    return source


class EvaluationContext:
    """One evaluation's call stack and observed property dependencies."""

    def __init__(self, mission=None, mode=None):
        self.mission, self.mode = mission, mode
        self.stack = []
        self.dependencies = {}

    def __enter__(self):
        self._token = _ACTIVE.set(self)
        return self

    def __exit__(self, *_):
        _ACTIVE.reset(self._token)

    def execute(self, source, owner=None, **bindings):
        namespace = dict(self.mission.namespace) if self.mission else {"math": math}
        namespace.update(self=owner, mode=self.mode, **bindings)
        # Session code is trusted Python. There is deliberately no sandbox.
        return eval(compile(expression(source), "<expression>", "eval"), namespace)

    def read(self, prop):
        if self.stack:
            edges = self.dependencies.setdefault(self.stack[-1], [])
            if prop not in edges:
                edges.append(prop)
        if prop in self.stack:
            path = " -> ".join(item.reference for item in (*self.stack, prop))
            raise ValueError(f"Property dependency cycle: {path}")
        mission = prop.owner.mission if prop.owner is not None else None
        if self.mission is None:
            self.mission = mission
        elif prop.owner is not None and mission is not self.mission:
            raise ValueError(f"{prop.reference} belongs outside this Mission.")
        if self.mode is not None and self.mission is not None:
            if getattr(self.mode, "mission", None) is not self.mission:
                raise ValueError("The evaluation mode must belong to this Mission.")
        source = prop._modes.get(self.mode, prop.expression)
        if source is None:
            raise PendingValue(f"{prop.reference} is unset.")
        self.stack.append(prop)
        try:
            linked = prop._source if self.mode not in prop._modes else None
            value = self.read(linked) if linked is not None else self.execute(source, prop.owner)
            return self.read(value) if isinstance(value, Property) else value
```

Figure E.10: `models/properties.py` (continued).

```
        finally:
            self.stack.pop()

    def paths(self, root):
        """Return each observed path from a measure to its input properties."""
        def walk(path):
            for child in self.dependencies.get(path[-1], ()):
                if child not in path:
                    extended = (*path, child)
                    yield extended
                    yield from walk(extended)
        return tuple(walk((root,)))


class Property:
    """An expression and annotation. Read .value to evaluate it."""

    def __init__(self, source=None, unit=None):
        self.owner, self.name = None, ""
        self._frozen = False
        self._source = None
        self._modes = {}
        self.expression = source
        self.unit = unit

    def __set_name__(self, owner, name):
        self.name = name

    def __get__(self, instance, owner=None):
        return self if instance is None else instance.properties[self.name]

    def __set__(self, instance, value):
        instance._set_property(self.name, value)

    def copy(self, owner, name, *, frozen=False):
        prop = Property(self.expression, self.unit)
        prop.owner, prop.name = owner, name
        prop._modes = dict(self._modes)
        prop._source = self._source
        prop._frozen = frozen
        return prop

    @classmethod
    def link(cls, source):
        prop = cls(unit=source.unit)
        prop._source = source
        return prop

```

Figure E.10: `models/properties.py` (continued).

```
    def _editable(self):
        if self._frozen:
            raise AttributeError(f"{self.reference} is fixed by its COTS class.")

    @property
    def expression(self):
        return self._source.reference if self._source is not None else self._expression

    @expression.setter
    def expression(self, source):
        self._editable()
        self._expression = expression(source)
        self._source = None

    @property
    def unit(self):
        return self._unit

    @unit.setter
    def unit(self, value):
        self._editable()
        if value is not None and not isinstance(value, (str, Unit)):
            raise TypeError("A unit is a Unit annotation or a string.")
        self._unit = value if isinstance(value, Unit) else Unit(value or "")

    @property
    def modes(self):
        return MappingProxyType(self._modes)

    def bind(self, mode, source):
        """Override this expression for an explicitly selected mode or state."""
        from .modes import ComponentMode, ComponentState, SpacecraftMode

        self._editable()
        if not isinstance(mode, (SpacecraftMode, ComponentMode, ComponentState)):
            raise TypeError("Choose a spacecraft mode, component mode or component state.")
        if self.owner is not None and self.owner.mission is not None:
            if mode.mission is not self.owner.mission:
                raise ValueError("The binding mode must belong to this Mission.")
        self._modes[mode] = expression(source)
        return self

    def unbind(self, mode):
        """Remove an override so evaluation uses the default expression."""
        self._editable()
        self._modes.pop(mode, None)
        return self

```

Figure E.10: `models/properties.py` (continued).

```
    @property
    def reference(self):
        return f"{self.owner.path}.{self.name}" if self.owner is not None else "<property>"

    @property
    def value(self):
        return self.evaluate()

    def evaluate(self, *, mission=None, mode=None):
        active = _ACTIVE.get()
        if active is not None:
            if mission is not None and mission is not active.mission:
                raise ValueError("Nested evaluation uses the active Mission.")
            if mode is not None and mode is not active.mode:
                raise ValueError("Nested evaluation uses the active mode.")
            return active.read(self)
        with EvaluationContext(mission, mode) as context:
            return context.read(self)

    @property
    def verification(self):
        from .requirements import InstanceVerification

        mission = self.owner.mission if self.owner is not None else None
        return mission.verify().for_instance(self) if mission else InstanceVerification(self, ())

    @property
    def verified(self):
        return self.verification.verified

    def __repr__(self):
        return f"Property({self.reference!r}, expression={self.expression!r},
unit={str(self.unit)!r})"

    def __format__(self, spec):
        value = format(self.value, spec)
        unit = str(self.unit)
        return f"{value} {unit}" if unit else value

    def assess(self, **options):
        try:
            return EvaluationResult(EvaluationStatus.VALUE, self.evaluate(**options))
        except PendingValue as error:
            return EvaluationResult(EvaluationStatus.PENDING, message=str(error))
        except Exception as error:
            return EvaluationResult(EvaluationStatus.ERROR, message=f"{type(error).__name__}:
{error}")
```

Figure E.10: `models/properties.py` (continued).

```
"""Read-only COTS queries against a component's owned requirements."""

from copy import deepcopy
from dataclasses import dataclass
import math
from numbers import Real

from .elements import Component
from .properties import EvaluationStatus
from .requirements import VerificationReport


@dataclass(frozen=True)
class CotsRecommendation:
    candidate: Component
    report: VerificationReport
    sort_value: object = None


def recommend_cots(component, candidates, *, sort_by=None, reverse=False, limit=None, mode=None):
    """Substitute candidate Property definitions in an isolated Mission.

    Candidates are detached, fixed Component instances. Only the generic
    component's direct requirements are checked. Every owned requirement must
    have conditions and pass. With no requirements, the query is unconstrained.
    Property expressions use the generic component as self. Existing COTS
    selections are omitted. No installation or assembly search is performed.
    Sorting requires a finite real Property value and preserves ties in input
    order. Return the original candidates alongside their trial reports.
    """
    if not isinstance(component, Component) or component.fixed_properties:
        raise TypeError("Choose a generic Component.")
    mission = component.mission
    if mission is None:
        raise ValueError("Attach the component to a Mission before querying COTS.")
    if mode is not None and getattr(mode, "mission", None) is not mission:
        raise ValueError("The evaluation mode must belong to this Mission.")
    if limit is not None and (type(limit) is not int or limit < 0):
        raise ValueError("The recommendation limit must be a nonnegative integer or None.")
    candidates = tuple(dict.fromkeys(candidates))
    if any(not isinstance(item, Component) or not item.fixed_properties for item in candidates):
        raise TypeError("Supply fixed COTS Component instances.")
```

Figure E.11: `models/recommendations.py`.

```
    if any(item.parent is not None for item in candidates):
        raise ValueError("Supply detached COTS candidates.")
    if sort_by is not None and (not isinstance(sort_by, str) or not any(
        sort_by in item.properties for item in (component, *candidates)
    )):
        raise ValueError(f"Unknown sort Property {sort_by!r}.")

    matches = []
    for candidate in candidates:
        if not component.accepts(candidate):
            continue
        memo = {id(imported): imported for imported in mission.imports.values()}
        trial, product = deepcopy((mission, candidate), memo)
        owner, selected_mode = memo[id(component)], memo.get(id(mode))
        for installed in owner.cots:
            owner.remove(installed)
        for name, prop in product.properties.items():
            definition = prop.copy(None, name)
            if name in owner.properties:
                setattr(owner, name, definition)
            else:
                owner.define(name, definition)
        reports = tuple(requirement.verify(mode=selected_mode) for requirement in owner.requirements)
        if any(report.verified is not True for report in reports):
            continue
        sort_value = None
        if sort_by is not None:
            if sort_by not in product.properties:
                continue
            result = owner.properties[sort_by].assess(mode=selected_mode)
            sort_value = result.value
            if (result.status is not EvaluationStatus.VALUE or isinstance(sort_value, bool)
                    or not isinstance(sort_value, Real)
                    or not (isinstance(sort_value, int) or math.isfinite(sort_value))):
                continue
        report = VerificationReport(trial, tuple(result for entry in reports for result in
entry.results))
        matches.append(CotsRecommendation(candidate, report, sort_value))
    if sort_by is not None:
        matches.sort(key=lambda match: match.sort_value, reverse=reverse)
    return tuple(matches[:limit])
```

Figure E.11: `models/recommendations.py` (continued).

```
"""Boolean conditions, separate evaluation status and transitive failure traces."""

from dataclasses import dataclass

from .objects import ModelObject
from .properties import EvaluationContext, EvaluationStatus, PendingValue, Property
from .properties import expression as expression_source


@dataclass(frozen=True)
class VerificationFailure:
    requirement: object
    condition: object
    path: tuple
    measure: object
    target: object

    @property
    def instance(self):
        return self.path[-1].owner

    def explain(self):
        path = " -> ".join(prop.reference for prop in self.path)
        return f"{self.condition.path}: {self.condition.expression} failed (measure={self.measure!r}, target={self.target!r}) via {path}"


@dataclass(frozen=True)
class ConditionResult:
    condition: object
    status: EvaluationStatus
    value: object = None
    measure: object = None
    target: object = None
    paths: tuple = ()
    message: str = ""

    @property
    def requirement(self):
        return self.condition.parent

    @property
    def failures(self):
        if self.value is not False or self.status is not EvaluationStatus.VALUE:
            return ()
        return tuple(VerificationFailure(self.requirement, self.condition, path, self.measure, self.target) for path in self.paths)


@dataclass(frozen=True)
class InstanceVerification:
```

Figure E.12: `models/requirements.py`.

```
    instance: object
    results: tuple

    @property
    def verified(self):
        if any(result.value is False for result in self.results):
            return False
        if self.results and all(result.value is True for result in self.results):
            return True
        return None

    @property
    def status(self):
        if any(result.status is EvaluationStatus.ERROR for result in self.results):
            return EvaluationStatus.ERROR
        if not self.results or any(result.status is EvaluationStatus.PENDING for result in
self.results):
            return EvaluationStatus.PENDING
        return EvaluationStatus.VALUE

    @property
    def failures(self):
        return tuple(failure for result in self.results for failure in result.failures if
failure.instance is self.instance or failure.path[-1] is self.instance)


@dataclass(frozen=True)
class VerificationReport:
    mission: object
    results: tuple

    def for_instance(self, instance):
        if isinstance(instance, str):
            instance = self.mission.get(instance)
        related = tuple(result for result in self.results if any(path[-1].owner is instance or
path[-1] is instance for path in result.paths))
        return InstanceVerification(instance, related)

    @property
    def verified(self):
        return InstanceVerification(self.mission, self.results).verified

    @property
    def status(self):
        return InstanceVerification(self.mission, self.results).status

    @property
    def failures(self):
        return tuple(failure for result in self.results for failure in result.failures)


```

Figure E.12: `models/requirements.py` (continued).

```
class VerificationCondition(ModelObject):
    measure = Property()
    target = Property()

    def __init__(self, name, measure, target, expression):
        super().__init__(name, measure=measure, target=target)
        self.expression = expression

    @property
    def expression(self):
        return self._expression

    @expression.setter
    def expression(self, source):
        source = expression_source(source)
        if source is None:
            raise ValueError("Supply a boolean Python expression.")
        self._expression = source

    def evaluate(self, *, mode=None):
        context = EvaluationContext(self.mission, mode)
        measure, target, value, message, paths = None, None, None, "", ()
        with context:
            try:
                try:
                    measure = context.read(self.measure)
                finally:
                    paths = context.paths(self.measure)
                target = context.read(self.target)
                value = context.execute(self.expression, self, measure=measure, target=target)
                if type(value) is not bool:
                    raise TypeError("A VerificationCondition must return a boolean.")
                status = EvaluationStatus.VALUE
            except PendingValue as error:
                status, value, message = EvaluationStatus.PENDING, None, str(error)
            except Exception as error:
                status, value, message = EvaluationStatus.ERROR, None, f"{type(error).__name__}:
    {error}"
        return ConditionResult(self, status, value, measure, target, paths, message)


class Requirement(ModelObject):
    def __init__(self, name, *, statement="", rationale="", text=None, context=None, conditions=(),
    **bindings):
        if text is not None:
            if statement:
                raise TypeError("Specify statement or its text alias, not both.")
            statement = text
        if not isinstance(statement, str) or not isinstance(rationale, str):
            raise TypeError("Statement and rationale are text.")
        if context is not None and not isinstance(context, ModelObject):
```

Figure E.12: `models/requirements.py` (continued).

```
147             raise TypeError("A requirement context is a model object.")
148         super().__init__(name, **bindings)
149         self._context = context
150         self.statement = statement
151         self.rationale = rationale
152         for condition in conditions:
153             self.add(condition)
154 
155     @property
156     def context(self):
157         """Calculation context, separate from containment and verification coverage."""
158         return self._context
159 
160     def validate_attachment(self):
161         if self.mission is not None and self.context is not None and self.context.mission is not
    self.mission:
162             raise ValueError("The requirement context must belong to its Mission.")
163 
164     @property
165     def text(self):
166         """Compatibility alias for the statement template."""
167         return self.statement
168 
169     @text.setter
170     def text(self, value):
171         self.statement = value
172 
173     def render_statement(self, *, mode=None):
174         """Render prose independently of verification, using owned condition names."""
175         fields = {condition.name: condition for condition in self.conditions}
176         fields.update(self=self, mode=mode)
177         with EvaluationContext(self.mission, mode):
178             return self.statement.format_map(fields)
179 
180     @property
181     def conditions(self):
182         return self.children
183 
184     def accepts(self, child):
185         return isinstance(child, VerificationCondition)
186 
187     def verify(self, *, mode=None):
188         return VerificationReport(self.mission, tuple(condition.evaluate(mode=mode) for condition in
    self.conditions))
189 
190     @property
191     def verification(self):
192         return self.verify()
```

Figure E.12: `models/requirements.py` (continued).

```
"""Concrete segment and subsystem classifications, with visible defaults."""

from .elements import GroundSegment, SpaceSegment, Subsystem
from .eps import EPS
from .properties import Property
from .requirements import VerificationCondition


class CommandAndDataHandling(Subsystem):
    pass


class Communication(Subsystem):
    pass


class AttitudeDeterminationAndControl(Subsystem):
    pointing = Property(unit="1")
    pointing_face = Property(unit="1")
    clocking_angle = Property(unit="deg")
    effective_clocking_angle = Property("0 if self.pointing.value == 'sun_pointing' else
  self.clocking_angle", "deg")


class GuidanceNavigationAndControl(Subsystem):
    pass


class StructuresAndMechanisms(Subsystem):
    pass


class Thermal(Subsystem):
    pass


class Propulsion(Subsystem):
    pass


class Payload(Subsystem):
    pass


class CubeSatBus(SpaceSegment):
    units = Property(unit="U")
    length_x = Property("illumination.cubesat_dimensions(units=self.units).x", "m")
    length_y = Property("illumination.cubesat_dimensions(units=self.units).y", "m")
    length_z = Property("illumination.cubesat_dimensions(units=self.units).z", "m")
```

Figure E.13: `models/spacecraft.py`.

```
    default_subsystems = (
        EPS,
        CommandAndDataHandling,
        Communication,
        AttitudeDeterminationAndControl,
        GuidanceNavigationAndControl,
        StructuresAndMechanisms,
        Thermal,
        Propulsion,
        Payload,
    )

    def create_requirements(self):
        limits = (
            (
                "mass", "kg", "2", "kg/U",
                "CubeSat Design Specification Rev 14.1, Table 1 gives typical maxima of 2 kg/U. "
                "Confirm the applicable dispenser and launch provider limits.",
            ),
            (
                "volume", "m3", "0.001", "m3/U",
                "Nominal allocation of 0.001 m3/U from the 10 cm cube definition. "
                "This checks accounted volume. Bounding dimensions, packing and dispenser fit require
    separate checks.",
            ),
        )
        for quantity, unit, per_u, per_u_unit, rationale in limits:
            name = f"{quantity}_limit"
            condition = VerificationCondition(
                "maximum", getattr(self, quantity),
                Property("self.parent.units.value * self.parent.maximum_per_u.value", unit),
                "type(self.parent.units.value) in (int, float) and self.parent.units.value > 0 "
                "and 0 < target and math.isfinite(target) and 0 <= measure <= target",
            )
            self.add_requirement(
                name,
                statement=f"The complete {{self.context.name}} {quantity} shall not exceed
    {{maximum.target}}.",
                rationale=rationale,
                units=self.units,
                maximum_per_u=Property(per_u, per_u_unit),
                conditions=(condition,),
            )


class GroundStation(GroundSegment):
    pass
```

Figure E.13: `models/spacecraft.py` (continued).

```
"""Stakeholder classifications and third parties."""

from .objects import ModelObject
from .requirements import Requirement


class Stakeholder(ModelObject):
    def accepts(self, child):
        return isinstance(child, Requirement)


class RegulatoryAgency(Stakeholder):
    pass


class MissionStakeholder(Stakeholder):
    pass


class LaunchProvider(Stakeholder):
    pass


class Partner(Stakeholder):
    pass


class ThirdParty(ModelObject):
    pass
```

Figure E.14: `models/stakeholders.py`.

```python
"""Fixed geometry and full-orbit illumination summaries.

Freimann et al. (2019), Sec. II.i: max(0, panel normal dot Sun direction).
Qureshi (2020), Sec. 2.2: separate panel projection and Earth shadow.
The Keplerian orbit and cylindrical shadow match sizing.py. All changing
positions, shadow boundaries and projections remain internal to this script.
"""

import math
from typing import NamedTuple, Tuple

from framework.models.geometry import Face, Pointing
from . import sizing

Vector = Tuple[float, float, float]
_TAU = 2 * math.pi


class Dimensions(NamedTuple):
    x: float
    y: float
    z: float


class Normal(NamedTuple):
    vector: Vector


class Projection(NamedTuple):
    sunlight_mean: float
    projected_sunlight_duration: float


def cubesat_dimensions(units: float) -> Dimensions:
    """Nominal 10 cm unit layouts in m. These are allocation boxes, excluding rails."""
    layouts = {1: (1, 1, 1), 1.5: (1, 1, 1.5), 2: (1, 1, 2),
               3: (1, 1, 3), 6: (2, 1, 3), 12: (2, 2, 3)}
    if type(units) not in (int, float) or units not in layouts:
        raise ValueError("Supported nominal layouts are 1U, 1.5U, 2U, 3U, 6U and 12U. Supply other dimensions explicitly.")
    return Dimensions(*(0.1 * count for count in layouts[units]))


def face_normal(face: str) -> Normal:
    """Outward unit normal in the body frame."""
    return Normal(Face(face).normal)

```

Figure E.15: `scripts/illumination.py`.

```

def face_area(face: str, length_x: float, length_y: float, length_z: float) -> sizing.Area:
    """Gross face area of a rectangular body, in m2."""
    sizing._finite(length_x, length_y, length_z)
    if min(length_x, length_y, length_z) <= 0:
        raise ValueError("Body dimensions must be positive.")
    lengths = (length_x, length_y, length_z)
    axis = "XYZ".index(Face(face).value[1])
    area = math.prod(length for index, length in enumerate(lengths) if index != axis)
    sizing._finite(area)
    return sizing.Area(area)


def _dot(left, right):
    return sum(a * b for a, b in zip(left, right))


def _unit(vector):
    if len(vector) != 3:
        raise ValueError("A panel normal needs three body-frame components.")
    sizing._finite(*vector)
    scale = max(abs(value) for value in vector)
    if scale == 0:
        raise ValueError("A panel normal must be nonzero.")
    scaled = tuple(value / scale for value in vector)
    norm = math.sqrt(_dot(scaled, scaled))
    return tuple(value / norm for value in scaled)


def _nadir_normal(normal, pointing_face, clocking_angle):
    """Normal in radial-outward, along-track, orbit-normal coordinates.

    The pointing face is aligned with -radial. At zero clocking, the next
    cyclic positive body axis (Y for +/-X, Z for +/-Y, X for +/-Z) aligns
    with along-track. Clocking is right-handed about the pointing direction.
    """
    target = pointing_face.normal
    axis = "XYZ".index(pointing_face.value[1])
    reference = tuple(1 if index == (axis + 1) % 3 else 0 for index in range(3))
    cross = (target[1] * reference[2] - target[2] * reference[1],
             target[2] * reference[0] - target[0] * reference[2],
             target[0] * reference[1] - target[1] * reference[0])
    a, b, c = _dot(normal, target), _dot(normal, reference), _dot(normal, cross)
    angle = math.radians(clocking_angle % 360)
    return (-a, b * math.cos(angle) - c * math.sin(angle),
            -b * math.sin(angle) - c * math.cos(angle))

```

Figure E.15: `scripts/illumination.py` (continued).

```
94
95 def _eccentric_anomaly(true_anomaly, eccentricity):
96     turns = math.floor((true_anomaly + math.pi) / _TAU)
97     angle = true_anomaly - turns * _TAU
98     return math.atan2(math.sqrt((1 - eccentricity) * (1 + eccentricity)) * math.sin(angle),
99                       eccentricity + math.cos(angle)) + turns * _TAU
100
101
102 def _positive_integral(a, b, c, start, end):
103     """Exact integral of max(0, a cos(E) + b sin(E) + c)."""
104     cuts = [start, end]
105     amplitude = math.hypot(a, b)
106     if abs(c) < amplitude:
107         phase = math.atan2(b, a)
108         offset = math.acos(-c / amplitude)
109         for root in (phase - offset, phase + offset):
110             for turn in range(math.floor((start - root) / _TAU), math.ceil((end - root) / _TAU) + 1):
111                 angle = root + turn * _TAU
112                 if start < angle < end:
113                     cuts.append(angle)
114     integral = 0.0
115     cuts.sort()
116     for left, right in zip(cuts, cuts[1:]):
117         middle, span = (left + right) / 2, right - left
118         value = a * math.cos(middle) + b * math.sin(middle)
119         if value + c > 0:
120             integral += max(0.0, 2 * math.sin(span / 2) * value + c * span)
121     return integral
122
123
124 def orbital_projection(semimajor_axis: float, eccentricity: float, earth_radius: float,
125                        beta_angle: float, sun_projection_angle: float, period: float,
126                        normal_body: Vector, pointing: str, pointing_face: str,
127                        clocking_angle: float) -> Projection:
128     """One unobstructed section, averaged over sunlight in one complete orbit.
129
130     NADIR fixes the declared face toward Earth and its clocking in the local
131     orbital frame. SUN_POINTING keeps the declared face toward the Sun.
132     Neither law models slew, pointing error or attitude dynamics.
133     No initial phase or epoch is needed for these complete-orbit averages.
134     """
135     sizing._orbit(semimajor_axis, eccentricity, earth_radius)
136     sizing._illumination_inputs(semimajor_axis, earth_radius, beta_angle, period)
137     sizing._finite(sun_projection_angle, clocking_angle)
138     normal = _unit(normal_body)
139     pointing, pointing_face = Pointing(pointing), Face(pointing_face)
140     circular = eccentricity == 0 or abs(beta_angle) == 90
```

Figure E.15: `scripts/illumination.py` (continued).

```
    if circular:
        durations = sizing.circular_illumination(semimajor_axis, earth_radius, beta_angle, period)
    else:
        durations = sizing.eccentric_illumination(
            semimajor_axis, eccentricity, earth_radius, beta_angle, sun_projection_angle, period)
    if pointing is Pointing.SUN_POINTING:
        mean = max(0.0, min(1.0, _dot(normal, pointing_face.normal)))
        return Projection(mean, mean * durations.sunlight)

    radial, along_track, orbit_normal = _nadir_normal(normal, pointing_face, clocking_angle)
    beta, sun_angle = math.radians(beta_angle), math.radians(sun_projection_angle % 360)
    cos_beta = 0.0 if abs(beta_angle) == 90 else math.cos(beta)
    # c(nu) = A cos(nu) + B sin(nu) + C. With dt = (1-e cos(E))/n dE,
    # the projected time integrand is max(0, a cos(E) + b sin(E) + c).
    a_true = cos_beta * (radial * math.cos(sun_angle) + along_track * math.sin(sun_angle))
    b_true = cos_beta * (radial * math.sin(sun_angle) - along_track * math.cos(sun_angle))
    c_true = orbit_normal * math.sin(beta)
    a = a_true - eccentricity * c_true
    b = math.sqrt((1 - eccentricity) * (1 + eccentricity)) * b_true
    c = c_true - eccentricity * a_true

    shadow = []
    if durations.eclipse > 0:
        if eccentricity == 0:
            half_width = math.pi * durations.eclipse / period
            shadow = [(sun_angle + math.pi - half_width, sun_angle + math.pi + half_width)]
        else:
            shadow = [(sun_angle + math.pi + 2 * math.atan(entry),
                       sun_angle + math.pi + 2 * math.atan(exit))
                      for entry, exit in sizing._shadow_intervals(
                          semimajor_axis, eccentricity, earth_radius, beta_angle,
sun_projection_angle)]
    # Integrate only the sunlit arcs, split at each zero of panel projection.
    integral, start = 0.0, sun_angle
    for entry, exit in (*shadow, (sun_angle + _TAU, sun_angle + _TAU)):
        integral += _positive_integral(a, b, c, _eccentric_anomaly(start, eccentricity),
                                       _eccentric_anomaly(entry, eccentricity))
        start = exit
    projected_duration = period * integral / _TAU
    mean = projected_duration / durations.sunlight
    sizing._finite(mean, projected_duration)
    if not -1e-12 <= mean <= 1 + 1e-12:
        raise ArithmeticError("Integrated sunlight projection lies outside [0, 1].")
    mean = min(1.0, max(0.0, mean))
    return Projection(mean, mean * durations.sunlight)
```

Figure E.15: `scripts/illumination.py` (continued).

```python
"""Preliminary EPS sizing from final_report/design-proposal.tex.

Keplerian orbit, spherical Earth, cylindrical shadow and a fixed Sun per orbit.
Public angle inputs are in degrees to match Orbit's annotations.
Electrical extrema describe supplied normal operating cases, without transients.
Optional PV samples retain their selected temperature and degradation conditions.
"""

import math
from typing import NamedTuple, Optional


class OrbitalPeriod(NamedTuple):
    seconds: float


class Illumination(NamedTuple):
    eclipse: float
    sunlight: float


class Energy(NamedTuple):
    joules: float


class Power(NamedTuple):
    watts: float


class CellCount(NamedTuple):
    fractional: float
    required: int


class MissionCycles(NamedTuple):
    cycles: float


class DepthOfDischarge(NamedTuple):
    fraction: float


class RetainedValue(NamedTuple):
    retained: float


class Area(NamedTuple):
    square_metres: float


class Efficiency(NamedTuple):
    fraction: float


class PowerDensity(NamedTuple):
    watts_per_square_metre: float

```

Figure E.16: `scripts/sizing.py`.

```
58
59 class Extrema(NamedTuple):
60     minimum: float
61     maximum: float
62
63
64 class PVCharacteristics(NamedTuple):
65     mpp_voltage: float
66     mpp_current: float
67     maximum_power: float
68     open_circuit_voltage: float
69     short_circuit_current: float
70
71
72 class Voltage(NamedTuple):
73     volts: float
74
75
76 class TransferSelection(NamedTuple):
77     method: str
78
79
80 class PVOperatingPoint(NamedTuple):
81     voltage: float
82     current: float
83     power: float
84
85
86 def transfer_selection(transfer_method: str) -> TransferSelection:
87     """Validate an explicitly selected solar extraction method."""
88     if transfer_method not in ("DET", "MPPT", "FIXED_VOLTAGE"):
89         raise ValueError("Select DET, MPPT or FIXED_VOLTAGE for the solar power path.")
90     return TransferSelection(transfer_method)
91
92
93 def solar_operating_voltage(transfer_method: str, reference_voltage: float,
94                             voltage_drop: float = 0) -> Voltage:
95     """DET uses bus voltage plus drop. Other methods use their selected voltage.
96
97     MPPT is an ideal MPP reference. Tracking, startup and conversion feasibility
98     require separate checks. A DET drop is supplied at the assessed current.
99     """
100    transfer_selection(transfer_method)
101    _finite(reference_voltage, voltage_drop)
102    if reference_voltage <= 0 or voltage_drop < 0:
103        raise ValueError("Require a positive reference voltage and a nonnegative path drop.")
104    if transfer_method != "DET" and voltage_drop != 0:
105        raise ValueError("A bus-to-array voltage drop applies only to DET.")
106    voltage = reference_voltage + voltage_drop
107    _finite(voltage)
108    return Voltage(voltage)
109
110
111 def pv_operating_point(pv_points: tuple, series_count: int, parallel_count: int,
112                        operating_voltage: float, interpolation: str = "none") -> PVOperatingPoint:
113     """Current at an imposed array voltage under the selected curve conditions.
114
```

Figure E.16: `scripts/sizing.py` (continued).

```
    Exact samples are used by default. Linear interpolation requires explicit
    acceptance. No extrapolation, mismatch or reverse-blocking model is supplied.
    """
    points = tuple(tuple(point) for point in pv_points)
    characteristics = pv_characteristics(points)
    if any(type(count) is not int or count <= 0 for count in (series_count, parallel_count)):
        raise ValueError("Require positive integer series and parallel counts.")
    if interpolation not in ("none", "linear"):
        raise ValueError("Select none or linear for PV interpolation.")
    _finite(operating_voltage)
    if not 0 <= operating_voltage <= series_count * characteristics.open_circuit_voltage:
        raise ValueError("The operating voltage lies outside the supplied PV characteristic.")
    points = tuple((series_count * voltage, parallel_count * current) for voltage, current in points)
    for voltage, current in points:
        _finite(voltage, current)
        if operating_voltage == voltage:
            power = voltage * current
            _finite(power)
            return PVOperatingPoint(voltage, current, power)
    if interpolation == "none":
        raise ValueError("The operating voltage needs a sampled point or explicitly accepted linear
interpolation.")
    for (left, current_left), (right, current_right) in zip(points, points[1:]):
        if left < operating_voltage < right:
            fraction = (operating_voltage - left) / (right - left)
            current = current_left + fraction * (current_right - current_left)
            power = operating_voltage * current
            _finite(current, power)
            return PVOperatingPoint(operating_voltage, current, power)
    raise ValueError("The operating voltage is not covered by the supplied PV characteristic.")


def extrema(values: tuple) -> Extrema:
    """Finite extrema of explicitly supplied cases on one electrical path."""
    values = tuple(values)
    if not values:
        raise ValueError("Supply at least one operating value.")
    _finite(*values)
    return Extrema(min(values), max(values))


def load_current_extrema(powers: tuple, minimum_voltage: float,
                         maximum_voltage: float) -> Extrema:
    """Independent P/V corners. Powers include losses at the assessed source."""
    power = extrema(powers)
    _finite(minimum_voltage, maximum_voltage)
    if power.minimum < 0 or not 0 < minimum_voltage <= maximum_voltage:
        raise ValueError("Require nonnegative powers and an ordered positive voltage interval.")
    result = Extrema(power.minimum / maximum_voltage, power.maximum / minimum_voltage)
    _finite(*result)
    return result


def pv_characteristics(pv_points: tuple) -> PVCharacteristics:
    """Sampled MPP and endpoints of one complete, ordered I-V characteristic.

    Points are (voltage in V, current in A). No interpolation or extrapolation
```

Figure E.16: `scripts/sizing.py` (continued).

```
    is performed. Sparse data must include the manufacturer's MPP point.
    """
    points = tuple(tuple(point) for point in pv_points)
    if len(points) < 3 or any(len(point) != 2 for point in points):
        raise ValueError("Supply at least three (voltage, current) PV points including MPP and both endpoints.")
    for voltage, current in points:
        _finite(voltage, current)
        if voltage < 0 or current < 0:
            raise ValueError("PV voltages and currents must be nonnegative.")
    if any(right[0] <= left[0] for left, right in zip(points, points[1:])):
        raise ValueError("PV voltage samples must be strictly increasing.")
    if points[0][0] != 0 or points[0][1] <= 0 or points[-1][1] != 0:
        raise ValueError("PV samples must span short circuit (0, Isc) to open circuit (Voc, 0).")
    powers = tuple(voltage * current for voltage, current in points)
    _finite(*powers)
    index = max(range(len(points)), key=powers.__getitem__)
    if powers[index] <= 0:
        raise ValueError("The PV characteristic must include a positive power point.")
    voltage, current = points[index]
    return PVCharacteristics(voltage, current, powers[index], points[-1][0], points[0][1])


def pv_efficiency(pv_points: tuple, active_area: float,
                  reference_irradiance: float) -> Efficiency:
    """Reference MPP efficiency for the selected cell or purchased panel."""
    _finite(active_area, reference_irradiance)
    if active_area <= 0 or reference_irradiance <= 0:
        raise ValueError("PV active area and reference irradiance must be positive.")
    incident_power = active_area * reference_irradiance
    _finite(incident_power)
    if incident_power <= 0:
        raise ValueError("Incident PV power is below floating point range.")
    efficiency = pv_characteristics(pv_points).maximum_power / incident_power
    if not 0 < efficiency <= 1:
        raise ValueError("The selected PV characteristic must give an efficiency in (0, 1].")
    return Efficiency(efficiency)


def _finite(*values):
    if not all(math.isfinite(value) for value in values):
        raise ValueError("Sizing inputs and results must be finite.")


def _orbit(semimajor_axis, eccentricity, earth_radius):
    _finite(semimajor_axis, eccentricity, earth_radius)
    if earth_radius <= 0 or not 0 <= eccentricity < 1:
        raise ValueError("Require a positive Earth radius and 0 <= eccentricity < 1.")
    if semimajor_axis * (1 - eccentricity) <= earth_radius:
        raise ValueError("The orbit's perigee must lie above the spherical Earth.")


def _illumination_inputs(semimajor_axis, earth_radius, beta_angle, period):
    _finite(semimajor_axis, earth_radius, beta_angle, period)
    if not -90 <= beta_angle <= 90 or period <= 0 or semimajor_axis <= earth_radius or earth_radius <= 0:
```

Figure E.16: `scripts/sizing.py` (continued).

```
        raise ValueError("Require a positive period, radius above Earth and beta in [-90, 90] degrees.")


def orbit_period(semimajor_axis: float, eccentricity: float, earth_radius: float,
                 gravitational_parameter: float) -> OrbitalPeriod:
    _orbit(semimajor_axis, eccentricity, earth_radius)
    _finite(gravitational_parameter)
    if gravitational_parameter <= 0:
        raise ValueError("The gravitational parameter must be positive.")
    period = 2 * math.pi * semimajor_axis * math.sqrt(semimajor_axis / gravitational_parameter)
    _finite(period)
    if period <= 0:
        raise ValueError("The orbital period is below floating point range.")
    return OrbitalPeriod(period)


def circular_illumination(semimajor_axis: float, earth_radius: float,
                          beta_angle: float, period: float) -> Illumination:
    _illumination_inputs(semimajor_axis, earth_radius, beta_angle, period)
    if abs(beta_angle) == 90:
        return Illumination(0.0, period)
    beta = math.radians(beta_angle)
    radius_ratio = earth_radius / semimajor_axis
    shadow_ratio = abs(math.sin(beta)) / radius_ratio
    # Treat a contact within floating point roundoff as a zero-duration eclipse.
    if shadow_ratio >= 1 - 2e-15:
        return Illumination(0.0, period)
    sine = radius_ratio * math.sqrt((1 - shadow_ratio) * (1 + shadow_ratio)) / math.cos(beta)
    eclipse = period * (math.asin(min(1.0, sine)) / math.pi)
    return Illumination(eclipse, period - eclipse)


def _polynomial(coefficients, value):
    result = 0.0
    for coefficient in reversed(coefficients):
        result = result * value + coefficient
    return result


def _bisect(coefficients, left, right):
    initial = _polynomial(coefficients, left)
    for _ in range(128):
        middle = (left + right) / 2
        if middle == left or middle == right:
            break
        value = _polynomial(coefficients, middle)
        if value == 0:
            return middle
        if (value > 0) == (initial > 0):
            left, initial = middle, value
        else:
            right = middle
    return (left + right) / 2


def _roots(coefficients, left, right):
```

Figure E.16: `scripts/sizing.py` (continued).

```
    """Isolate real polynomial roots through derivative roots, without a grid."""
    coefficients = list(coefficients)
    while len(coefficients) > 1 and coefficients[-1] == 0:
        coefficients.pop()
    if len(coefficients) == 1:
        return []
    if len(coefficients) == 2:
        root = -coefficients[0] / coefficients[1]
        return [root] if left <= root <= right else []
    derivative = [index * coefficient for index, coefficient in enumerate(coefficients)][1:]
    points = [left, *_roots(derivative, left, right), right]
    roots = [point for point in points if _polynomial(coefficients, point) == 0]
    for lower, upper in zip(points, points[1:]):
        if _polynomial(coefficients, lower) * _polynomial(coefficients, upper) < 0:
            roots.append(_bisect(coefficients, lower, upper))
    unique = []
    for root in sorted(roots):
        if not unique or root != unique[-1]:
            unique.append(root)
    return unique


def _mean_anomaly(t, eccentricity, cos_lambda, sin_lambda):
    # Resolve nu = lambda + pi + 2 atan(t) without losing small shadow arcs
    # by first adding them to pi, particularly near an eccentric apogee.
    cos_x, sin_x = (1 - t * t) / (1 + t * t), 2 * t / (1 + t * t)
    cos_nu = -(cos_lambda * cos_x - sin_lambda * sin_x)
    sin_nu = -(sin_lambda * cos_x + cos_lambda * sin_x)
    anomaly = math.atan2(
        math.sqrt((1 - eccentricity) * (1 + eccentricity)) * sin_nu,
        eccentricity + cos_nu,
    )
    return anomaly - eccentricity * math.sin(anomaly)


def _shadow_intervals(semimajor_axis, eccentricity, earth_radius, beta_angle, sun_projection_angle):
    """Night-side shadow arcs in t = tan((nu - lambda - pi)/2). Inputs are validated by callers."""
    angle = math.radians(sun_projection_angle % 360)
    beta = math.radians(beta_angle)
    p = semimajor_axis * (1 - eccentricity) * (1 + eccentricity)
    q = (p / earth_radius) ** 2
    s = math.sin(beta) ** 2
    cos_lambda, sin_lambda = math.cos(angle), math.sin(angle)
    a, b, c = 1 - eccentricity * cos_lambda, 2 * eccentricity * sin_lambda, 1 + eccentricity *
 cos_lambda
    # Set x = nu - lambda - pi on the night side and t = tan(x/2).
    # Multiplication by (1+t^2)^2 turns the shadow boundary into this quartic.
    coefficients = [q * s - a * a, -2 * a * b, q * (4 - 2 * s) - b * b - 2 * a * c, -2 * b * c, q * s
 - c * c]
    _finite(*coefficients)
    scale = max(abs(value) for value in coefficients)
    coefficients = [value / scale for value in coefficients]
    boundaries = [-1.0, *(root for root in _roots(coefficients, -1.0, 1.0) if -1 < root < 1), 1.0]
    return [(entry, exit) for entry, exit in zip(boundaries, boundaries[1:])
            if _polynomial(coefficients, (entry + exit) / 2) < 0]


```

Figure E.16: `scripts/sizing.py` (continued).

```
def eccentric_illumination(semimajor_axis: float, eccentricity: float, earth_radius: float,
                           beta_angle: float, sun_projection_angle: float,
                           period: float) -> Illumination:
    _orbit(semimajor_axis, eccentricity, earth_radius)
    _illumination_inputs(semimajor_axis, earth_radius, beta_angle, period)
    _finite(sun_projection_angle)
    if abs(beta_angle) == 90:
        return Illumination(0.0, period)
    angle = math.radians(sun_projection_angle % 360)
    cos_lambda, sin_lambda = math.cos(angle), math.sin(angle)
    mean_span = 0.0
    for entry, exit in _shadow_intervals(semimajor_axis, eccentricity, earth_radius, beta_angle,
    sun_projection_angle):
        start = _mean_anomaly(entry, eccentricity, cos_lambda, sin_lambda)
        end = _mean_anomaly(exit, eccentricity, cos_lambda, sin_lambda)
        mean_span += (end - start) % (2 * math.pi)
    eclipse = min(period, max(0.0, period * (mean_span / (2 * math.pi))))
    return Illumination(eclipse, period - eclipse)


def load_energy(average_power: float, duration: float) -> Energy:
    _finite(average_power, duration)
    if average_power < 0 or duration < 0:
        raise ValueError("Average load power and duration must be nonnegative.")
    energy = average_power * duration
    _finite(energy)
    return Energy(energy)


def minimum_panel_power(sunlight_energy: float, eclipse_energy: float,
                        sunlight_efficiency: float, eclipse_efficiency: float,
                        sunlight_duration: float) -> Power:
    _finite(sunlight_energy, eclipse_energy, sunlight_efficiency, eclipse_efficiency,
    sunlight_duration)
    if sunlight_energy < 0 or eclipse_energy < 0 or sunlight_duration <= 0:
        raise ValueError("Require nonnegative load energies and positive sunlight duration.")
    if not 0 < sunlight_efficiency <= 1 or not 0 < eclipse_efficiency <= 1:
        raise ValueError("Energy path efficiencies must lie in (0, 1].")
    power = (sunlight_energy / sunlight_efficiency + eclipse_energy / eclipse_efficiency) /
    sunlight_duration
    _finite(power)
    return Power(power)


def cell_power(eol_voltage: Optional[float] = None, eol_current: Optional[float] = None,
               pv_points: tuple = ()) -> Power:
    """Use either scalar EOL MPP ratings or an explicitly selected I-V curve."""
    if pv_points:
        if eol_voltage is not None or eol_current is not None:
            raise ValueError("Select PV points or scalar MPP ratings, rather than both.")
        return Power(pv_characteristics(pv_points).maximum_power)
    if eol_voltage is None or eol_current is None:
        raise ValueError("Supply both EOL MPP ratings or a complete PV characteristic.")
    _finite(eol_voltage, eol_current)
    if eol_voltage <= 0 or eol_current <= 0:
        raise ValueError("Cell EOL maximum power point voltage and current must be positive.")
    power = eol_voltage * eol_current
```

Figure E.16: `scripts/sizing.py` (continued).

```
    _finite(power)
    return Power(power)


def cell_count(required_power: float, eol_cell_power: float) -> CellCount:
    _finite(required_power, eol_cell_power)
    if required_power < 0 or eol_cell_power <= 0:
        raise ValueError("Require nonnegative panel power and positive EOL cell power.")
    count = required_power / eol_cell_power
    _finite(count)
    return CellCount(count, math.ceil(count))


def mission_cycles(lifetime: float, days_per_year: float, orbits_per_day: float) -> MissionCycles:
    _finite(lifetime, days_per_year, orbits_per_day)
    if lifetime <= 0 or days_per_year <= 0 or orbits_per_day <= 0:
        raise ValueError("Mission lifetime, days per year and orbits per day must be positive.")
    cycles = lifetime * days_per_year * orbits_per_day
    _finite(cycles)
    return MissionCycles(cycles)


def depth_of_discharge(required_cycles: float, cycle_coefficient: float, cycle_exponent: float,
                       fit_dod_min: float, fit_dod_max: float) -> DepthOfDischarge:
    _finite(required_cycles, cycle_coefficient, cycle_exponent, fit_dod_min, fit_dod_max)
    if required_cycles <= 0 or cycle_coefficient <= 0 or cycle_exponent >= 0:
        raise ValueError("Require positive cycle counts and coefficient, and a negative cycle-life
  exponent.")
    if not 0 < fit_dod_min <= fit_dod_max <= 1:
        raise ValueError("The characterized DoD interval must lie in (0, 1].")
    if required_cycles > cycle_coefficient * fit_dod_min ** cycle_exponent:
        raise ValueError("Required cycles exceed the characterized DoD interval.")
    dod = (required_cycles / cycle_coefficient) ** (1 / cycle_exponent)
    return DepthOfDischarge(max(fit_dod_min, min(fit_dod_max, dod)))


def battery_energy(eclipse_energy: float, depth_of_discharge: float,
                   degradation_margin: float) -> Energy:
    _finite(eclipse_energy, depth_of_discharge, degradation_margin)
    if eclipse_energy < 0 or not 0 < depth_of_discharge <= 1 or degradation_margin < 0:
        raise ValueError("Require nonnegative energy and margin, and DoD in (0, 1].")
    energy = eclipse_energy / depth_of_discharge * (1 + degradation_margin)
    _finite(energy)
    return Energy(energy)


def retained_value(initial: float, retention: float) -> RetainedValue:
    _finite(initial, retention)
    if initial < 0 or not 0 <= retention <= 1:
        raise ValueError("Require a nonnegative rating and retention in [0, 1].")
    return RetainedValue(initial * retention)


def parallel_strings(required_total: float, per_cell: float, series_count: int) -> CellCount:
    """Check-bound for an entered series count. Both quantities use the same unit."""
    _finite(required_total, per_cell)
    if required_total < 0 or per_cell <= 0 or type(series_count) is not int or series_count <= 0:
```

Figure E.16: `scripts/sizing.py` (continued).

```
        raise ValueError("Require a nonnegative total, positive cell rating and positive integer series count.")
    fractional = required_total / per_cell / series_count
    _finite(fractional)
    return CellCount(fractional, math.ceil(fractional))


def panel_area(active_area: float, packing_fraction: float) -> Area:
    _finite(active_area, packing_fraction)
    if active_area < 0 or not 0 < packing_fraction <= 1:
        raise ValueError("Require nonnegative active area and packing in (0, 1].")
    area = active_area / packing_fraction
    _finite(area)
    return Area(area)


def solar_efficiency(bol_efficiency: float, lifetime_retention: float, temperature_factor: float) -> Efficiency:
    _finite(bol_efficiency, lifetime_retention, temperature_factor)
    if not 0 <= bol_efficiency <= 1 or not 0 <= lifetime_retention <= 1 or temperature_factor < 0:
        raise ValueError("Efficiencies and retention lie in [0, 1]. The temperature factor is nonnegative.")
    efficiency = bol_efficiency * lifetime_retention * temperature_factor
    if not 0 <= efficiency <= 1:
        raise ValueError("Corrected solar efficiency must lie in [0, 1].")
    return Efficiency(efficiency)


def array_power_density(irradiance: float, eol_efficiency: float, inherent_retention: float,
                        assembly_efficiency: float, sunlight_projection: float) -> PowerDensity:
    """Sunlight mean density at constant irradiance, efficiency and losses."""
    _finite(irradiance, eol_efficiency, inherent_retention, assembly_efficiency, sunlight_projection)
    if irradiance < 0 or not 0 <= sunlight_projection <= 1:
        raise ValueError("Require nonnegative irradiance and sunlight projection in [0, 1].")
    if not all(0 <= factor <= 1 for factor in (eol_efficiency, inherent_retention, assembly_efficiency)):
        raise ValueError("Solar efficiency and loss factors must lie in [0, 1].")
    density = irradiance * eol_efficiency * inherent_retention * assembly_efficiency * sunlight_projection
    _finite(density)
    return PowerDensity(density)


def minimum_array_area(required_power: float, power_density: float) -> Area:
    _finite(required_power, power_density)
    if required_power < 0 or power_density <= 0:
        raise ValueError("Require nonnegative power and positive available power density.")
    area = required_power / power_density
    _finite(area)
    return Area(area)


def tumbling_array_area(effective_area: float, packing_fraction: float) -> Area:
    """Nogrady's six-face, half-illuminated, 50 degree incidence approximation."""
    return panel_area(2 * effective_area / math.cos(math.radians(50)), packing_fraction)
```

Figure E.16: `scripts/sizing.py` (continued).